

Advanced Databases

(DSC 301)

Lecture 1

Introduction to the Course & SQL

Revision

Dr. Ahmed Elsayed Yakoub

2025 - 2026

Course Info.

- Lectures: Wednesday (12-2, 2-4, 4-6)
- Instructor: Dr. Ahmed Elsayed
 - Office hours: Wednesday 11:00 - 12:00
 - Contact: Dr.ahmedyakoub@gmail.com
- Assistant Instructor/TA: ???
- Textbook:

[1] Elmasri, Ramez. Fundamentals of database systems. Pearson Education India, 2008.

[2] Coronel, Carlos, Steven Morris, and Peter Rob. . Cengage Learning., 2011.

Assessment Scheme

- Midterm Exam: 30%
- Lab Work: 20 %
- Final Exam: 50 %
- Total: 100 %

Course Syllabus

Ref. Topic Ser.

Introduction to the Course & SQL Revision 1

[1] Functional Dependencies and Normalization for Relational Databases 2

[1] Disk Storage, Basic File Structures, and Hashing 3

[1] Indexing Structures for Files 4

[1,2] Algorithms for Query Processing and Optimization 5

[2] Transaction Management 6

[2] Distributed Database Management Systems 7

NoSQL Databases: An Overview 8

.....

The components of SQL :

As a language, SQL has two main components:

1. a data definition language (DDL) for defining the database structure, implementing the data structure and constraints components of the relation data model.
2. Data manipulation language (DML) for CRUD operations on the database, implementing the operations component of the relational data model.

3 CRUD operations: SQL DML

- In this section you will learn more about how tables can be manipulated using SQL to create, retrieve, update and delete data (CRUD operations).

After completing this section you should be able to:

- use the SQL INSERT, UPDATE and DELETE statements to add, update and delete rows in tables respectively
- use the SQL SELECT statement to answer queries about the data stored in a single database table

- explain the function and the order of processing of the SQL SELECT statement.

3.1 SQLSELECT statement • The general form of the SELECT statement is:

SELECT [DI STI NCT] * | <col umn l i st >

FROM <t abl e l i st >

[WHERE <condi t i on>]

[GROUP BY <col umn l i st > [HAVI NG <condi t i on>]] [ORDER BY <col umn l i st > [ASC| DESC]

]

[LI MI T <number>]

3.2SQLSELECTstatement processing

• The order of processing of SELECT statement clauses is as follows:

1 . FROM

2. WHERE

3. GROUP BY

4. HAVI NG

5. SELECT

6. ORDER BY

7. LI MI T

Coping withmissing or invalid data elements

• We have to be careful when handling columns that can take null • Any or that includes a null will evaluate to null (meaning‘unknown’).

• When a WHERE (or HAVING) clause is processed, if any of the columns referenced by the condition are null for a row, then that row is not selected.

• the functions IS NULL and IS NOT NULL check whether a column (or expression) is null or otherwise.

- Example: the following SQL will 'display the details of patients whose weight has not been recorded' .

```
SELECT *  
FROM pat i ent  
WHERE wei ght I S NULL;
```

3.3 Aggregation

- SQL aggregate functions compute a from a .
- The following example calculates some summary statistics about the contents of the patient table:

```
SELECT COUNT( * ) ,  
COUNT( DI STI NCT pat i ent _name) ,  
COUNT( wei ght ) ,  
MI N( wei ght ) ,  
MAX( wei ght ) ,  
CAST( AVG( wei ght ) AS DECI MAL( 4, 1 ) ) FROM pat i ent
```




Watch the activity 9.6
animation for this query

8

Aggregation

```
SELECT COUNT( *),  
COUNT( DI STI NCT pati ent _name),  
COUNT( wei ght ),  
MI N( wei ght) ,  
MAX( wei ght) ,  
CAST( AVG( wei ght) AS DECI MAL( 4, 1 ) )  
FROM pati ent
```

COUNT(*)	COUNT(DISTINCT patient_name)	COUNT(weight)	MIN(weight)	MAX(weight)	AVG(weight)
17	15	15	49.2	91.4	67.5

- Note:
- COUNT(DISTINCT patient_name) returns the number of unique patient names
- COUNT(weight) returns the number of rows where the weight column has a value, i.e. the number of patients whose weight has been recorded
- MIN(weight) returns the minimum weight recorded
- MAX(weight) returns the maximum weight recorded
- CAST(AVG(weight) AS DECIMAL(4,1)) returns the average weight recorded where the CAST function outputs this average to one decimal place.

Data Maintenance Queries

- INSERT
- UPDATE
- DELETE

INSERT

1. Add a new row into the EMPLOYEE table with the supplied values for all columns.

INSERT INTO EMPLOYEE

VALUES (1234,'Karolyn','Jones','733 Jackie Lane, Baldwin Harbor, NY

11510','516-223-8183','Senior VP', 95000, 'MGR')

INSERT

2. Add a new row into the EMPLOYEE table with the supplied values for most of the columns, leaving some column values as NULL.

INSERT INTO EMPLOYEE

VALUES (3344,'Robert','Moses','142 Crestview Drive, Long Branch, NJ

08835', NULL, NULL, 40000,'STF')

INSERT

3. Add new rows into the MANAGER, selecting managers from the EMPLOYEE table.

INSERT INTO MANAGER

SELECT *

FROM EMPLOYEE

WHERE EmployeeCode ='MGR'

UPDATE

1. Change last name in a specified row in EMPLOYEE table with supplied value.

UPDATE EMPLOYEE

SET LastName ='Raj'

WHERE EmployeeNo = 1234

UPDATE

2. Increase the salary of all managers by 5%.

UPDATE EMPLOYEE

SET Salary = Salary * 1.05

WHERE EmployeeCode ='MGR'

UPDATE

• 3. Modify multiple columns in a specified row in EMPLOYEE table with supplied values.

UPDATE EMPLOYEE

SET EmployeePosition ='Asst. Supervisor', Salary = 60000,

EmployeeCode ='SPR'

WHERE EmployeeNo = 3344

DELETE

1. Delete all rows from the MANAGER table. DELETE FROM MANAGER

DELETE

2. Delete one specific row from the EMPLOYEE table.

DELETE FROM EMPLOYEE

WHERE EmployeeNo = 3344

DELETE

3. Delete from EMPLOYEE table those rows for staff earning less than 25000.

DELETE FROM EMPLOYEE

WHERE EmployeeCode = 'STF' and Salary < 25000

Retrieval Queries

- Simple queries: mostly deriving results from single database tables.
- Advanced queries: the results need to be obtained from more than one table.
- Queries involving set operations
- queries within queries (subqueries or nested queries) • enables a complex query to be broken down into simple queries to produce the desired result.

Simple queries

- Based on the data model for a university we considered in earlier chapters. As you are already familiar with this data model, let us use it to illustrate the following query types. Go back and review the

following figures from earlier chapters:

Figure 7-22 ERD: university teaching aspects

Figure 9-22 University ERD: transformation into relational data model.

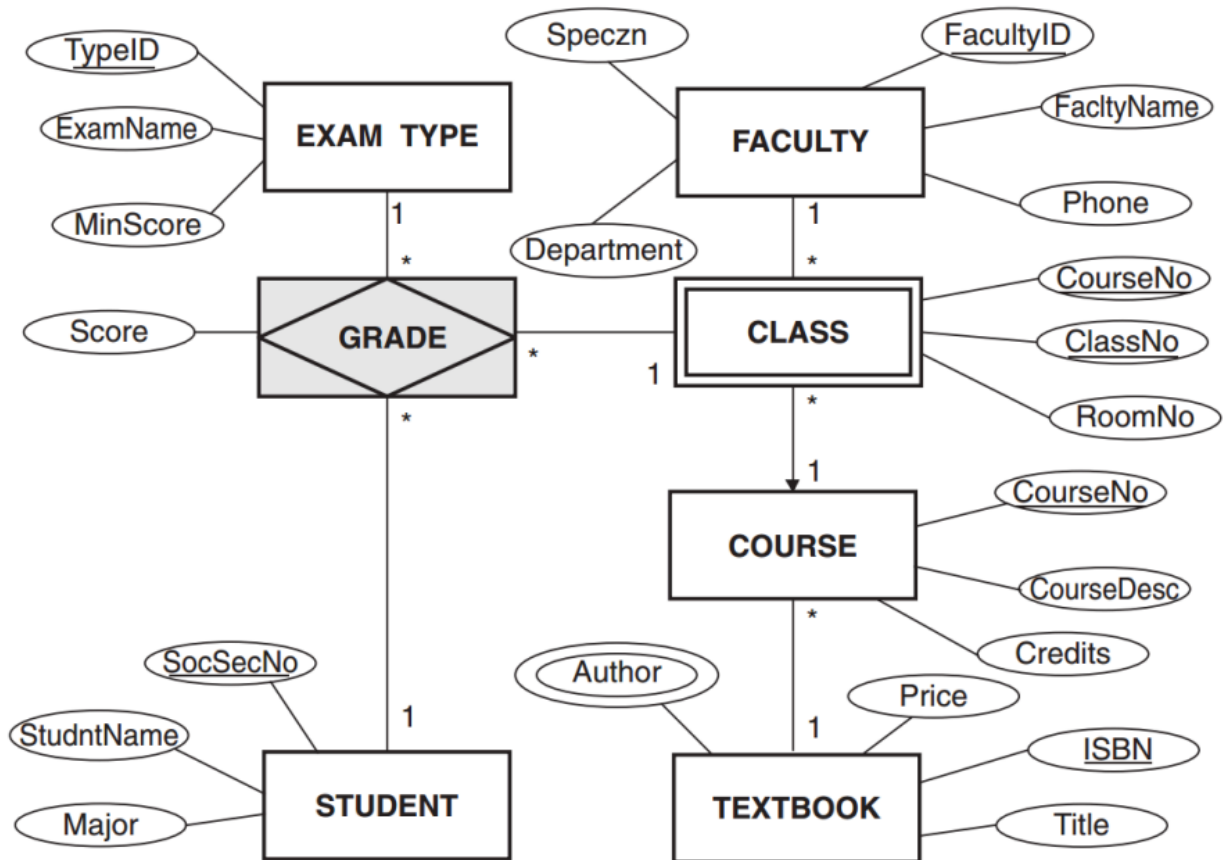


Figure 7-22 ERD: university teaching aspects.

Simple select with simple condition 1. Find the name of the course with course number 10971.

```
SELECT CourseDesc
```

```
FROM COURSE
```

```
WHERE CourseNo = '10971'
```

2. Find the names of students with computer science major.

```
SELECT StudntName
```

```
FROM STUDENT
```

WHERE Major = 'Computer Science'

Simple select using wild card '*' and simple condition

3. List all data for faculty member John Saunders.

SELECT *

FROM FACULTY

WHERE FacultyName = 'John Saunders'

Simple select using 'distinct '

4. Find all the majors for which students have enrolled.

SELECT DISTINCT Major

FROM STUDENT

Simple application of aggregate functions (Max and Min)

5. Find the maximum and minimum price of prescribed textbooks.

SELECT MAX (Price), MIN (Price)

FROM TEXTBOOK

Simple select with attribute expression and a simple condition

6. Add 5 points to minimum score for mid-term examination.

SELECT MinScore, (MinScore+5)

FROM EXAMTYPE

WHERE TypeId = 'MIDTERM'

Select statement with string constants, count() function and a simple condition

7. Print the total number of faculty members in the English department.

```
SELECT "The total number of faculty members in", Department,"is", COUNT (FACULTY)
FROM FACULTY
WHERE Department ='English'
```

Simple select with simple condition

8. Find the number of credits for the course with course number 10971.

```
SELECT Credits
FROM COURSE
WHERE CourseNo ='10971'
```

Select with groupby, having, sum() function& a where condition

9. List students by social security number in a given class with total score greater than given number.

```
SELECT SocSecNo, ClassNo, SUM (Score) FROM GRADE
WHERE ClassNo ='11223344'
GROUP BY SocSecNo, ClassNo
ORDER BY SocSecNo, ClassNo
HAVING SUM (Score) > 60
```

Select with IS NULL

10. Find the students who have not yet declared a major.

```
SELECT SocSecNo, StudntName
FROM STUDENT
WHERE Major IS NULL
```

Select with LIKE condition

11. Find the names and phone numbers of faculty members with specialization in data systems.

```
SELECT FacultyName, Phone
```

```
FROM FACULTY
```

```
WHERE Specialization LIKE 'Data%'
```

ADVANCED SQL QUERIES

Schema

These examples of advanced queries are based on the relational data model for a medical center as shown in Figure 13-11.

In these queries, observe how subqueries are used to obtain the desired results.

PATIENT (PatientNo, PatientName, DOB, AdmissionDate, DischargeDate)

DOCTOR (DoctorID, DoctorName, DoctorPhone)

SERVICE (ServiceCode, ServiceDesc)

TREATMENT (PatientNo, DoctorID, TreatmentDesc)

Foreign Key: **PatientNo** REFERENCES **PATIENT**

DoctorID REFERENCES **DOCTOR**

BILLING (PatientNo, ServiceCode, BilledAmount)

Foreign Key: **PatientNo** REFERENCES **PATIENT**

ServiceCode REFERENCES **SERVICE**

Figure 13-11 Relational data model: medical center.

Join implemented in where clause with simple condition

1. List the services billed to patient whose number is 224466.

```
SELECT SERVICE.ServiceCode, ServiceDesc FROM SERVICE, BILLING
WHERE SERVICE.ServiceCode = BILLING.ServiceCode AND PatientNo ='224466'
```

Uncorrelated nested query

2. List the services billed to patient whose number is 224466. (Alternative query.)

```
SELECT SERVICE.ServiceCode, ServiceDesc FROM SERVICE
WHERE ServiceCode IN
(SELECT ServiceCode FROM BILLING
WHERE PatientNo ='224466')
```

Uncorrelated nested query 2'. If the BILLING schema where:

BILLING (PatientNo, ServiceCode, BilledAmount), List the service billed to patient whose number is 224466. (Alternative query.)

```
SELECT SERVICE.ServiceCode, ServiceDesc FROM SERVICE
WHERE ServiceCode =
(SELECT ServiceCode FROM BILLING
WHERE PatientNo ='224466')
```

Uncorrelated nested query with ">" connector

3. List those patients who have been charged more than the average for service SCAN.

```
SELECT DISTINCT PatientNo
FROM BILLING
WHERE ServiceCode ='SCAN'
```

```

AND BilledAmount >
(SELECT AVG (BilledAmount)
FROM BILLING
WHERE ServiceCode ='SCAN' )

```

Union with uncorrelated nested query and NOT IN connector

6. Obtain a list of all billings including those patients for whom no billings have been done until now.

```

SELECT BILLING.PatientNo, PatientName, BilledAmount FROM BILLING, PATIENT
WHERE BILLING.PatientNo = PATIENT.PatientNo UNION
SELECT PatientNo, PatientName, 0
FROM PATIENT
WHERE PatientNo NOT IN
(SELECT PatientNo
FROM BILLING)

```

3Representingrelationships between tables

- A foreign key creates a relationship between a and a .
- In the and tables below, the column of the table () – the foreign key, matches the primary key column of the table () – the primary key.
- This represents a relationship between the and tables – a doctor is responsible for several patients

3Representingrelationships between

doctor (doctor_id, doctor_name, date_of_birth)

doctor_id doctor_name date_of_birth

d06 Gibson 1954-02-24

d07 Paxton 1960-05-23

d09 Tamblin 1972-12-22

tables

Referencing

table

Referenced table

patient (patient_id, patient_name, date_of_birth, gender, height, weight, doctor_id)

patient_id	patient_name	date_of_birth	gender	height	weight	doctor_id
p001	Thornton	1980-01-22	F	162.3	71.6	d06
p007	Tennent	1980-04-01	M	176.8	70.9	d07
p008	James	1980-07-08	M	167.9	70.5	d07
p009	Kay	1980-09-25	F	164.7	53.2	d06
p015	Harris	1980-12-04	M	180.6	64.3	d06
p031	Rubinstein	1980-12-23	F	null	null	null
p037	Boswell	1981-06-11	F	null	null	null

Indicates absence of

relationship

(also absence of d09)

3.1Referential integrity

- The referential integrity constraint enforces the integrity of the primary keys and foreign keys

- the value of a foreign key in the must either be or be one of the values of the primary key in the.

Referential integrity

Enforced by the DBMS :

- when a row containing an invalid foreign key value is inserted in the
- when a foreign key in the is updated to an invalid value
- when a row with a referenced primary key is deleted from the
- when a referenced primary key is updated in the .

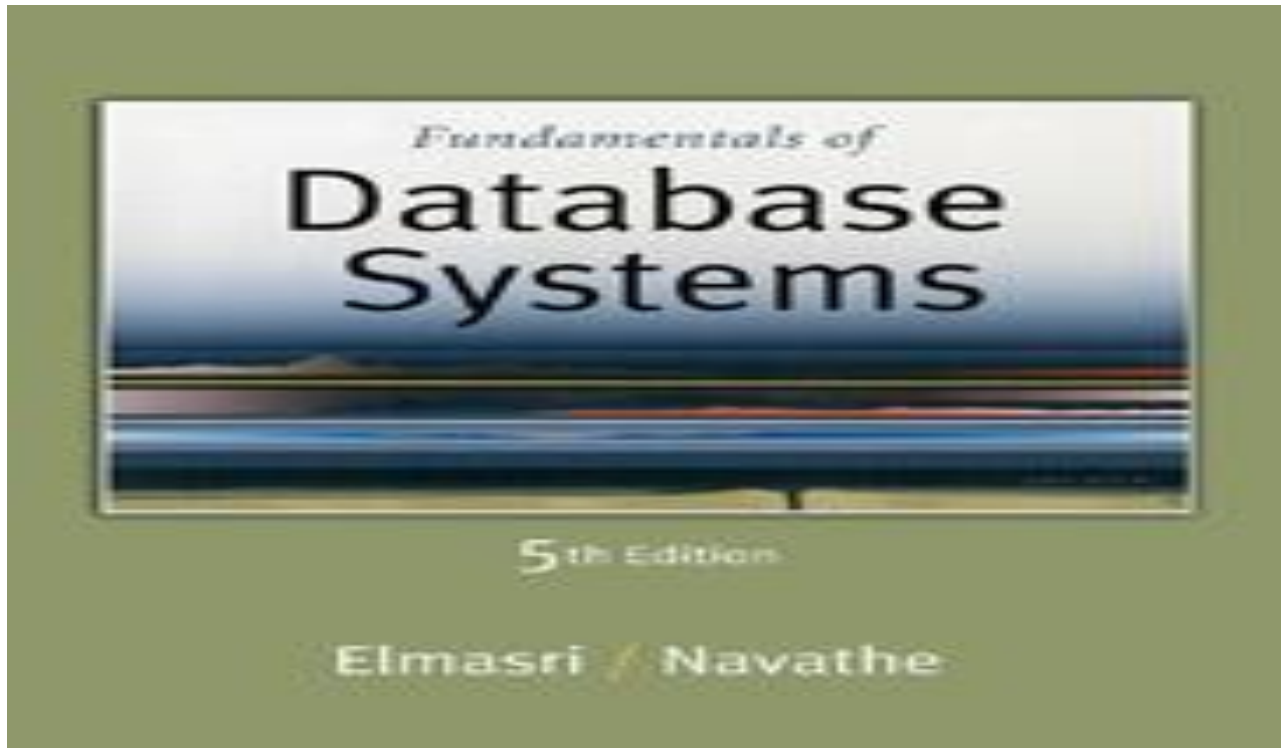
Referential integrity

- SQL allows us to specify referential actions:
- RESTRICT (default): prevent the referenced primary key being deleted or updated
- SET NULL: sets the value of the referencing foreign key to
- SET DEFAULT: sets the value of the referencing foreign key to a specified default value
- CASCADE: deletes the dependent rows in the referencing table.

3.2 Joins

- inner join and outer join operations allow us to: • Realize relationships between tables (e.g. which doctor is responsible for which patients), and to
- Identify the absence of a relationship between specific rows of the tables (e.g. which doctors are not responsible for any patients, and which patients are not under the care of a doctor).

.....



Chapter 10

Functional Dependencies and Normalization for Relational Databases

Chapter Outline

- v 1 Informal Design Guidelines for Relational Databases v 1.1 Semantics of the Relation Attributes
- v 1.2 Redundant Information in Tuples and Update Anomalies v 1.3 Null Values in Tuples
- v 2 Functional Dependencies (FDs)
- v 2.1 Definition of FD
- v 2.2 Inference Rules for FDs

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe Slide 10- 3

Chapter Outline

- v 3 Normal Forms Based on Primary Keys
- v 3.1 Normalization of Relations

v 3.2 Practical Use of Normal Forms

v 3.3 Definitions of Keys and Attributes Participating in Keys v 3.4 First Normal Form

v 3.5 Second Normal Form

v 3.6 Third Normal Form

1 Informal Design Guidelines for Relational Databases (1)

v What is relational database design?

v The grouping of attributes to form "good" relation schemas

v Two levels of relation schemas

v The logical "user view" level

v The storage "base relation" level

v Design is concerned mainly with base relations v What are the criteria for "good" base relations?

Informal Design Guidelines for Relational Databases (2)

v We first discuss informal guidelines for good relational design

v Then we discuss formal concepts of functional dependencies and normal forms

v - 1NF (First Normal Form)

v - 2NF (Second Normal Form)

v - 3NF (Third Normal Form)

1.1 Semantics of the Relation Attributes

v GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).

v Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation v Only foreign keys should be used to refer to other entities v Entity and relationship attributes should be kept apart as much as possible.

v Bottom Line:

Figure 10.1 A simplified COMPANY

relational database schema

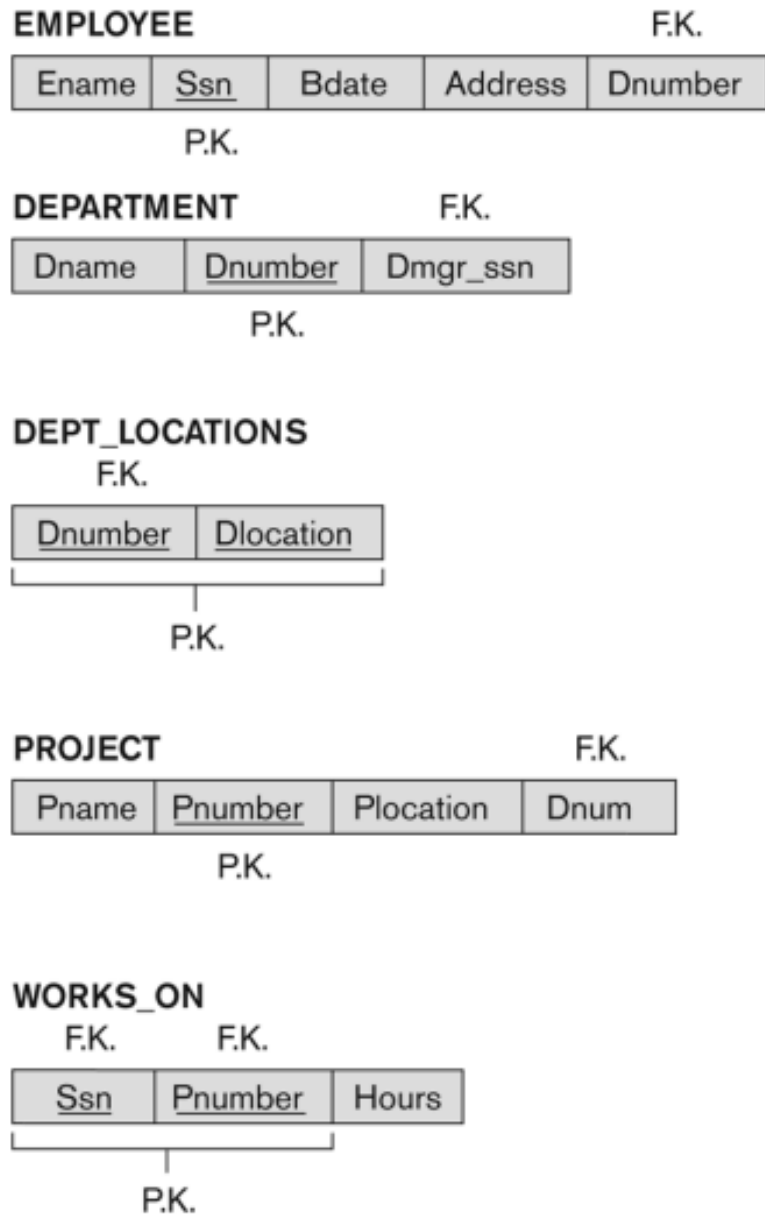


Figure 10.1

A simplified COMPANY relational database schema.

1.2 Redundant Information in Tuples and Update Anomalies

- v Information is stored redundantly v Wastes storage
- v Causes problems with update anomalies v Insertion anomalies
- v Deletion anomalies
- v Modification anomalies

EXAMPLE OF AN UPDATE ANOMALY

v Consider the relation:

v EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

v Update Anomaly:

v Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.

EXAMPLE OF AN INSERT ANOMALY

v Consider the relation:

v EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

v Insert Anomaly:

v Cannot insert a project unless an employee is assigned to it.

v Conversely

v Cannot insert an employee unless he/she is assigned to a project.

EXAMPLE OF AN DELETE ANOMALY

v Consider the relation:

v EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

v Delete Anomaly:

v When a project is deleted, it will result in deleting all the employees who work on that project.

v Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Figure 10.3 Two relation schemas suffering from update anomalies

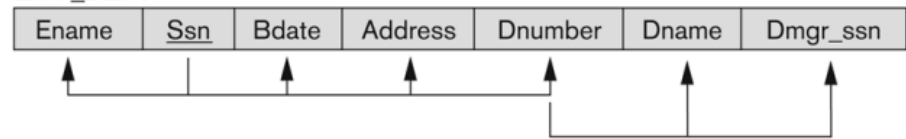
Figure 10.3

Two relation schemas suffering from update anomalies.

(a) EMP_DEPT and
(b) EMP_PROJ.

(a)

EMP_DEPT



(b)

EMP_PROJ

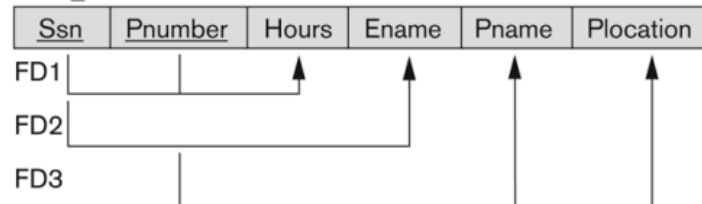


Figure 10.4 Example States for EMP_DEPT and EMP_PROJ

Figure 10.4

Example states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 10.2. These may be stored as base relations for performance reasons.

EMP_DEPT							Redundancy	
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn		
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555		
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555		
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321		
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321		
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555		
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555		
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321		
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555		

EMP_PROJ						Redundancy		Redundancy	
Ssn	Pnumber	Hours	Ename	Pname	Plocation				
123456789	1	32.5	Smith, John B.	ProductX	Bellaire				
123456789	2	7.5	Smith, John B.	ProductY	Sugarland				
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston				
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire				
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland				
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland				
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston				
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford				
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston				
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford				
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford				
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford				
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford				
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford				
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston				
888665555	20	Null	Borg, James E.	Reorganization	Houston				

Guideline to Redundant Information in Tuples and Update Anomalies

v GUIDELINE 2:

v Design a schema that does not suffer from the insertion, deletion and update anomalies.

v If there are any anomalies present, then note them so that applications can be made to take them into account.

1.3 Null Values in Tuples

GUIDELINE 3:

Relations should be designed such that their tuples will have as few NULL values as possible

Attributes that are NULL frequently could be placed in separate relations (with the primary key) Reasons for nulls:

Attribute not applicable or invalid

Attribute value unknown (may exist)

Value known to exist, but unavailable

2.1 Functional Dependencies (1)

Functional dependencies (FDs)

Are used to specify the "goodness" of relational designs

And keys are used to define normal forms for relations

Are constraints that are derived from the and of the data attributes A set of attributes X a set of attributes Y if the value of X determines a unique value for Y

Functional Dependencies (2)

$X \rightarrow Y$ holds if whenever two tuples have the same value for X, they the same value for Y

For any two tuples t_1 and t_2 in any relation instance $r(R)$: If $t_1[X]=t_2[X]$, $t_1[Y]=t_2[Y]$

$X \rightarrow Y$ in R specifies a on all relation instances $r(R)$

Written as $X \rightarrow Y$; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow:). FDs are derived from the real-world constraints on the attributes

Examples of FD constraints (1)

Social security number determines employee name $SSN \rightarrow ENAME$

Project number determines project name and location

$PNUMBER \rightarrow \{PNAME, PLOCATION\}$

Employee ssn and project number determines the hours per week that the employee works on the project

$\{SSN, PNUMBER\} \rightarrow HOURS$

Examples of FD constraints (2)

- ✓ An FD is a property of the attributes in the schema R
- ✓ The constraint must hold on relation instance $r(R)$
- ✓ If K is a key of R, then K functionally determines all attributes in R
- ✓ (since we never have two distinct tuples with $t1[K]=t2[K]$)

3 Normal Forms Based on Primary Keys

- ✓ 3.1 Normalization of Relations
- ✓ 3.2 Practical Use of Normal Forms
- ✓ 3.3 Definitions of Keys and Attributes Participating in Keys
- ✓ 3.4 First Normal Form
- ✓ 3.5 Second Normal Form
- ✓ 3.6 Third Normal Form

3.1 Normalization of Relations (1)

- ✓ Normalization:
- ✓ The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- ✓ Normal form:
- ✓ Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

3.2 Practical Use of Normal Forms

- ✓ Normalization is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- ✓ The practical utility of these normal forms becomes questionable when the constraints on which they are based are or to
- ✓ The database designers normalize to the highest possible normal form
- ✓ (usually up to 3NF, BCNF or 4NF)
- ✓ Denormalization:

- ∨ The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

3.3 Definitions of Keys and Attributes Participating in Keys (1)

- ∨ A superkey of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes $S \subseteq R$ with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$

- ∨ A key K is a superkey with the

that removal of any attribute from K will cause K not to be a superkey any more.

Definitions of Keys and Attributes

Participating in Keys (2)

- ∨ If a relation schema has more than one key, each is called a candidate key.

- ∨ One of the candidate keys is designated to be the primary key, and the others are called secondary keys.

- ∨ A Prime attribute must be a member of candidate key

- ∨ A Nonprime attribute is not a prime attribute— that is, it is not a member of any candidate key.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe Slide 10- 25

3.2 First Normal Form

- ∨ Disallows

- ∨ composite attributes

- ∨ multivalued attributes

- ∨ nested relations; attributes whose values for an are non-atomic

- ∨ Considered to be part of the definition of relation

Figure 10.8 Normalization into 1NF

(a)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
-------	----------------	----------	------------

(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 10.8

Normalization into 1NF.
 (a) A relation schema that is not in 1NF. (b) Example state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Figure 10.9 Normalization nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

Figure 10.9

Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

3.3 Second Normal Form (1)

v Uses the concepts of FDs, primary key

v Definitions

v Prime attribute: An attribute that is member of the primary key K

✓ Full functional dependency: a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more ✓ Examples:

✓ $\{SSN, PNUMBER\} \rightarrow HOURS$ is a full FD since neither $SSN \rightarrow HOURS$ nor $PNUMBER \rightarrow HOURS$ hold

✓ $\{SSN, PNUMBER\} \rightarrow ENAME$ is not a full FD (it is called a partial dependency) since $SSN \rightarrow ENAME$ also holds

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe Slide 10- 29

Second Normal Form (2)

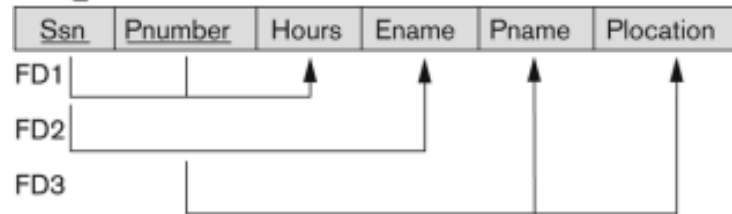
✓ A relation schema R is in second normal form (2NF) if every non-prime attribute A in R is fully functionally dependent on the primary key

✓ R can be decomposed into 2NF relations via the process of 2NF normalization

Figure 10.10 Normalizing into 2NF and 3NF

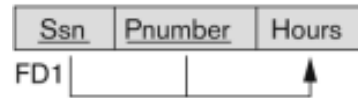
(a)

EMP_PROJ

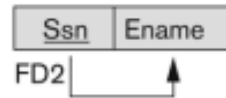


2NF Normalization

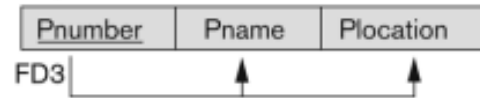
EP1



EP2

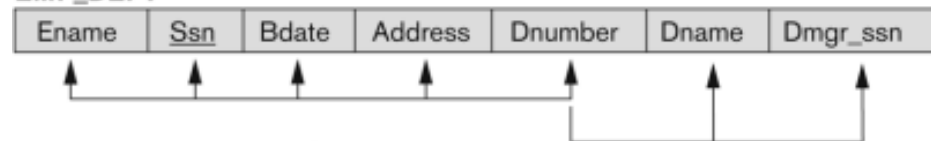


EP3



(b)

EMP_DEPT



3NF Normalization

ED1



ED2



Figure 10.10

Normalizing into 2NF and 3NF.
(a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

3.4 Third Normal Form (1)

v Definition:

v Transitive functional dependency: a FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$

v Examples:

v $SSN \rightarrow DMGRSSN$ is a transitive FD

v Since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow$

$DMGRSSN$ hold

v $SSN \rightarrow ENAME$ is non-transitive

v Since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow ENAME$

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe Slide 10- 32

Third Normal Form (2)

v A relation schema R is in third normal form (3NF) if it is in 2NF no non-prime attribute A in R is transitively dependent on the primary key

v R can be decomposed into 3NF relations via the process of 3NF normalization

v NOTE:

v In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.

v When Y is a candidate key, there is no problem with the transitive dependency .

v E.g., Consider EMP (SSN, Emp#, Salary).

v Here, $SSN \rightarrow Emp\# \rightarrow Salary$ and Emp# is a candidate key.

Normal Forms Defined Informally

v 1st normal form

v All attributes depend on the key

v 2nd normal form

v All attributes depend on the whole key v 3rd normal form

v All attributes depend on nothing but the key

Chapter Outline

v Informal Design Guidelines for Relational Databases

v Functional Dependencies (FDs)

v Definition, Inference Rules, Equivalence of Sets of FDs, Minimal Sets of FDs

v Normal Forms Based on Primary Keys.

.....

DB Normalization Exercises

Exercise 1: drugs prescribed

<u>Patient id</u>	Patient_name	doctor_id	doctor_name	date	drug_code	drug_name	dosage	duration
p001	Thronton	d06	Gibson	15-May-2014	T02378	Tramadol	50 mg 3 x day	As required
				15-May-2014	O17663	Omeprazole	40 mg 1 x day	Daily
				23-May-2014	S33558	Simvastatin	40 mg 1 x day	Daily
				15-Jun-2014	A12458	Amitriptyline	10 mg 5 x day	As required
p007	Tennet	d07	Paxton	01-Jun-2014	C31319	Ciprofloxacin	500 mg 2 x day	20 days

				01-Jun-2014	T05223	Tamsulosin	40 mg 1 x day	20 days
				01-Jul-2014	S33558	Simvastatin	20 mg 1 x day	6 weeks

Each row of the table represents a patient and the columns represent details of the patient, their doctor, and the drugs prescribed (as a repeating group of columns).

As each patient is identified by their patient identifier, we have chosen patient_id as the primary key.

2 Moving to First Normal Form (1NF)

A relation is in First Normal Form (1NF) if each attribute contains only atomic values, that is, it has no repeating groups* of values.

* A repeating group of data is defined as any attribute or group of attributes that may occur with multiple [i.e. different] values for a single value of the primary key.

3 Remove repeating groups

Patient_perscription

<u>Patient id</u>	<u>date</u>	<u>drug code</u>	drug_name	dosage	duration
p001	15-May-2014	T02378	Tramadol	50 mg 3 x day	As required
p001	15-May-2014	O17663	Omeprazole	40 mg 1 x day	Daily

p001	23-May-2014	S33558	Simvastatin	40 mg 1 x day	Daily
p001	15-Jun-2014	A12458	Amitriptyline	10 mg 5 x day	As required
p007	01-Jun-2014	C31319	Ciprofloxacin	500 mg 2 x day	20 days
p007	01-Jun-2014	T05223	Tamsulosin	40 mg 1 x day	20 days
p007	01-Jul-2014	S33558	Simvastatin	20 mg 1 x day	6 weeks

patient_doctor

<u>Patient_id</u>	Patient_name	doctor_id	doctor_name
p001	Thronton	d06	Gibson
p007	Tennet	d07	Paxton

4 patient_prescription

patient_id	date	drug_code	drug_name	dosage	duration
p001	15-May-2014	T02378	Tramadol	50 mg 3 x day	As required
p001	15-May-2014	O17663	Omeprazole	40 mg 1 x day	Daily
p001	23-May-2014	S33558	Simvastatin	40 mg 1 x day	Daily
p001	15-Jun-2014	A12458	Amitriptyline	10 mg 5 x day	As required
p007	01-Jun-2014	C31319	Ciprofloxacin	500 mg 2 x day	20 days
p007	01-Jun-2014	T05223	Tamsulosin	40 mg 1 x day	20 days
p007	01-Jul-2014	S33558	Simvastatin	20 mg 1 x day	6 weeks

patient_prescription (patient_id, date, drug_code, drug_name, dosage, duration)

patient_doctor

patient_id	patient_name	doctor_id	doctor_name
p001	Thornton	d06	Gibson
p007	Tennent	d07	Paxton

patient_doctor (patient_id, patient_name, doctor_id, doctor_name)

5 Moving to Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if it is in 1NF and every non-primary key attribute of the relation is dependent on the whole primary key, that is, without partial key dependencies.

- To represent the data in 2NF we remove any attributes that only depend on part of the primary key to separate relations, and choose a primary key for each new relation.
- This step only applies to relations that have a composite primary key. We have to decide whether any attributes in such relations are **functionally dependent** on only part of the composite primary key.

6 Moving to Second Normal Form (2NF)

patient_prescription

patient_id	date	drug_code	drug_name	dosage	duration
p001	15-May-2014	T02378	Tramadol	50 mg 3 x day	As required
p001	15-May-2014	O17663	Omeprazole	40 mg 1 x day	Daily
p001	23-May-2014	S33558	Simvastatin	40 mg 1 x day	Daily
p001	15-Jun-2014	A12458	Amitriptyline	10 mg 5 x day	As required
p007	01-Jun-2014	C31319	Ciprofloxacin	500 mg 2 x day	20 days
p007	01-Jun-2014	T05223	Tamsulosin	40 mg 1 x day	20 days
p007	01-Jul-2014	S33558	Simvastatin	20 mg 1 x day	6 weeks

patient_prescription (patient_id, date, drug_code, drug_name, dosage, duration)

This relation is not in 2NF

patient_id	date	drug_code	dosage	duration
p001	15-May-2014	T02378	50 mg 3 x day	As required
p001	15-May-2014	O17663	40 mg 1 x day	Daily
p001	23-May-2014	S33558	40 mg 1 x day	Daily
p001	15-Jun-2014	A12458	10 mg 5 x day	As required
p007	01-Jun-2014	C31319	500 mg 2 x day	20 days
p007	01-Jun-2014	T05223	40 mg 1 x day	20 days
p007	01-Jul-2014	S33558	20 mg 1 x day	6 weeks

prescription (patient_id, date, drug_code, dosage, duration)

drug_code	drug_name
T02378	Tramadol
O17663	Omeprazole
S33558	Simvastatin
A12458	Amitriptyline
C31319	Ciprofloxacin
T05223	Tamsulosin

drug (drug_code, drug_name)

8 Moving to Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it is in 2NF and every non-primary key attribute of the relation is wholly dependent on the whole primary key, and not by any non-primary key attribute.

- To represent the data in 3NF we remove any attributes that are not directly dependent on the primary key to separate relations, and choose a primary key for each new relation.

9 Moving to Third Normal Form (3NF)

doctor_id	doctor_name
-----------	-------------

d06	Gibson
-----	--------

d07	Paxton
-----	--------

patient_doctor

patient_id	patient_name	doctor_id	doctor_name
------------	--------------	-----------	-------------

p001	Thornton	d06	Gibson
------	----------	-----	--------

p007	Tennent	d07	Paxton
------	---------	-----	--------

FK

patient_id	patient_name	doctor_id
------------	--------------	-----------

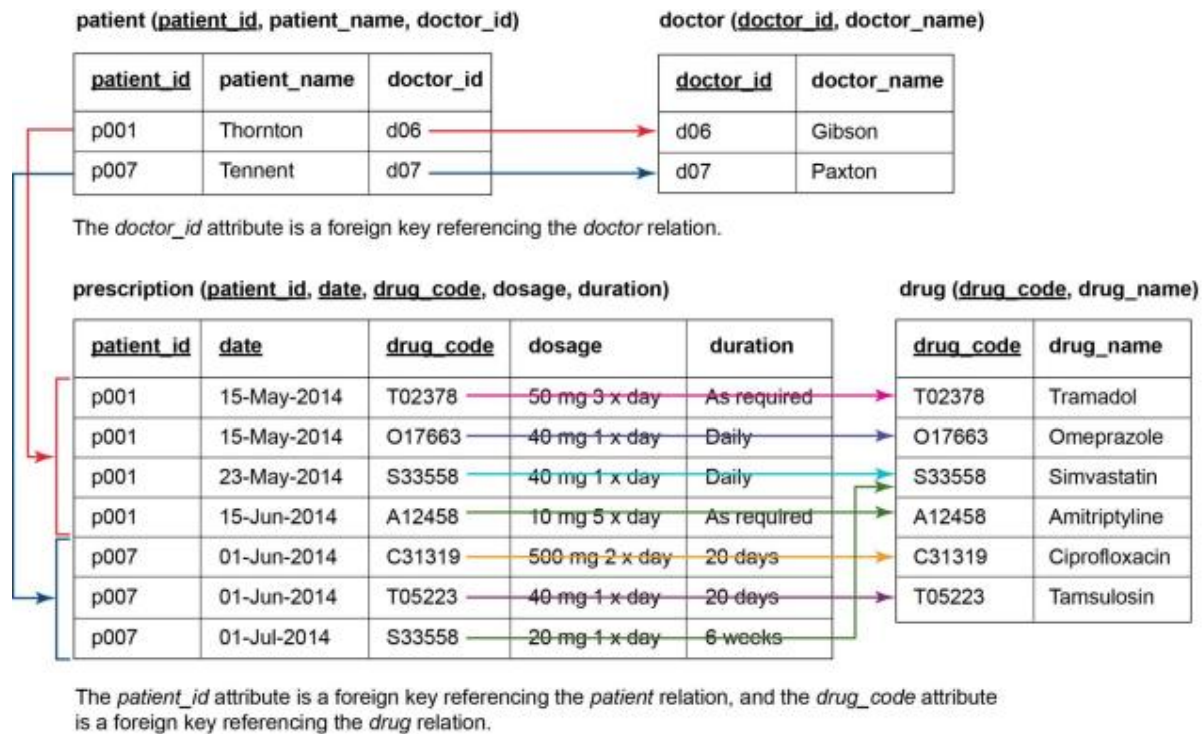
p001	Thornton	d06
------	----------	-----

p007	Tennent	d07
------	---------	-----

doctor (doctor_id, doctor_name) patient (patient_id, patient_name, doctor_id) 10

Normalised relations in 3NF

The final set of normalised relations is shown below.



11

Exercise 2: The purchase of art in a hotel

Normalise this relation to 3NF, showing the new relations . Make sure you identify the primary key for each normalized relation by underlining it

ART_PURCHASE (customer_id, art_id, art_theme, art_price, artist_code, artist_name, customer_name, customer_address, customer_phone, purchase_date)

Suppose that the following functional dependencies hold:

customer_id determines **customer_name**, **customer_address**, and **customer_phone**

art_id determines **art_theme**, **art_price**, **artist_code** and **artist_name** **artist_code** determines **artist_name**

customer_id and **art_id** determines **purchase_date**

Normalised relations in 3NF

– ART_PURCHASE (customer_id, art_id, purchase_date)

– CUSTOMER (customer_id, customer_name, customer_address, customer_phone) – ART (art_id, art_theme, art_price, artist_code) – ARTIST (artist_code, artist_name).

Chapter 14 Indexing Structures for Files

Chapter Outline

- Types of Single-level Ordered Indexes ■ Primary Indexes
- Clustering Indexes
- Secondary Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 3**

Indexes as Access Paths

- A single-level index is an auxiliary file that makes it more efficient to search for a record in the data file.
- The index is usually specified on one field of the file (although it could be specified on several fields)
- One form of an index is a file of entries <**field value, pointer to record**>, which is ordered by field value
- The index is called an access path on the field.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 4**

Indexes as Access Paths (contd.)

- The index file usually occupies considerably less disk blocks than the data file because its entries are much smaller
- A binary search on the index yields a pointer to the file record

■ Indexes can also be characterized as dense or sparse ■ A **dense index** has an index entry for every search key value (and hence every record) in the data file. ■ A **sparse (or nondense) index**, on the other hand, has index entries for only some of the search values

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 5**

Indexes as Access Paths (contd.)

■ Example: Given the following data file EMPLOYEE(NAME, SSN, ADDRESS, JOB, SAL, ...)

■ Suppose that:

■ record size $R=150$ bytes block size $B=512$ bytes $r=30000$ records ■ Then, we get:

■ blocking factor $Bfr = B \div R = 512 \div 150 = 3$ records/block

■ number of file blocks $b = (r/Bfr) = (30000/3) = 10000$ blocks

■ For an index on the SSN field, assume the field size $V_{SSN}=9$ bytes, assume the record pointer size $P_R=7$ bytes. Then:

■ index entry size $R_I = (V_{SSN} + P_R) = (9+7) = 16$ bytes

■ index blocking factor $Bfr_I = B \div R_I = 512 \div 16 = 32$ entries/block ■ number of index blocks $b_I = (r/Bfr_I) = (30000/32) = 938$ blocks ■ binary search needs $\log_2 b_I = \log_2 938 = 10$ block accesses + 1 ■ This is compared to an average linear search cost of:

■ $(b/2) = 10000/2 = 5000$ block accesses

■ If the file records are ordered, the binary search cost would be: ■ $\log_2 b = \log_2 10000 = 14$ block accesses

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 6**

Types of Single-Level Indexes

■ Primary Index

■ Defined on an ordered data file

■ The data file is ordered on a **key field**

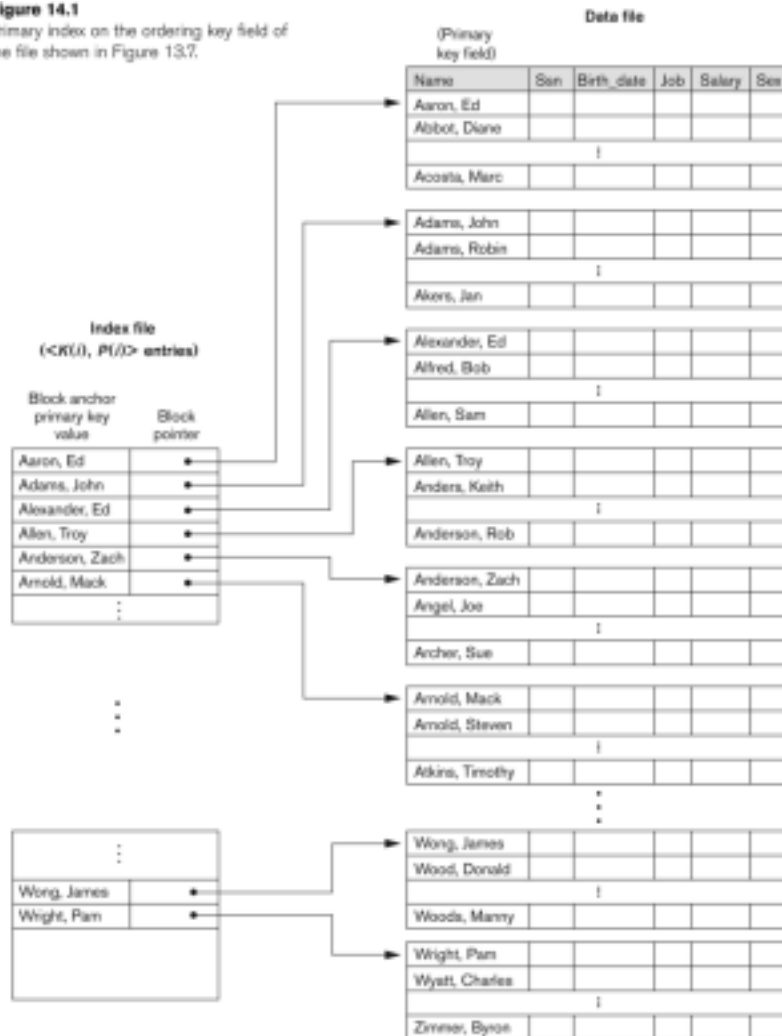
■ Includes one index entry *for each block* in the data file; the index entry has the key field value for the *first record* in the block, which is called the *block anchor*

■ A similar scheme can use the *last record* in a block. ■ A primary index is a nondense (sparse) index, since it includes an entry for each disk block of the data file and the keys of its anchor record rather than for every search value.

Primary index on the ordering key field

Figure 14.1

Primary index on the ordering key field of the file shown in Figure 13.7.



Types of Single-Level Indexes

■ Clustering Index

■ Defined on an ordered data file

■ The data file is ordered on a *non-key field* unlike primary index, which requires that the ordering field of the data file have a distinct value for each record.

■ Includes one index entry *for each distinct value* of the field; the index entry points to the first data block that contains records with that field value.

■ It is another example of *nondense* index where Insertion and Deletion is relatively straightforward with a clustering index.

A clustering index
on the
DEPTNUMBER
ordering non-key
field of an
EMPLOYEE file.

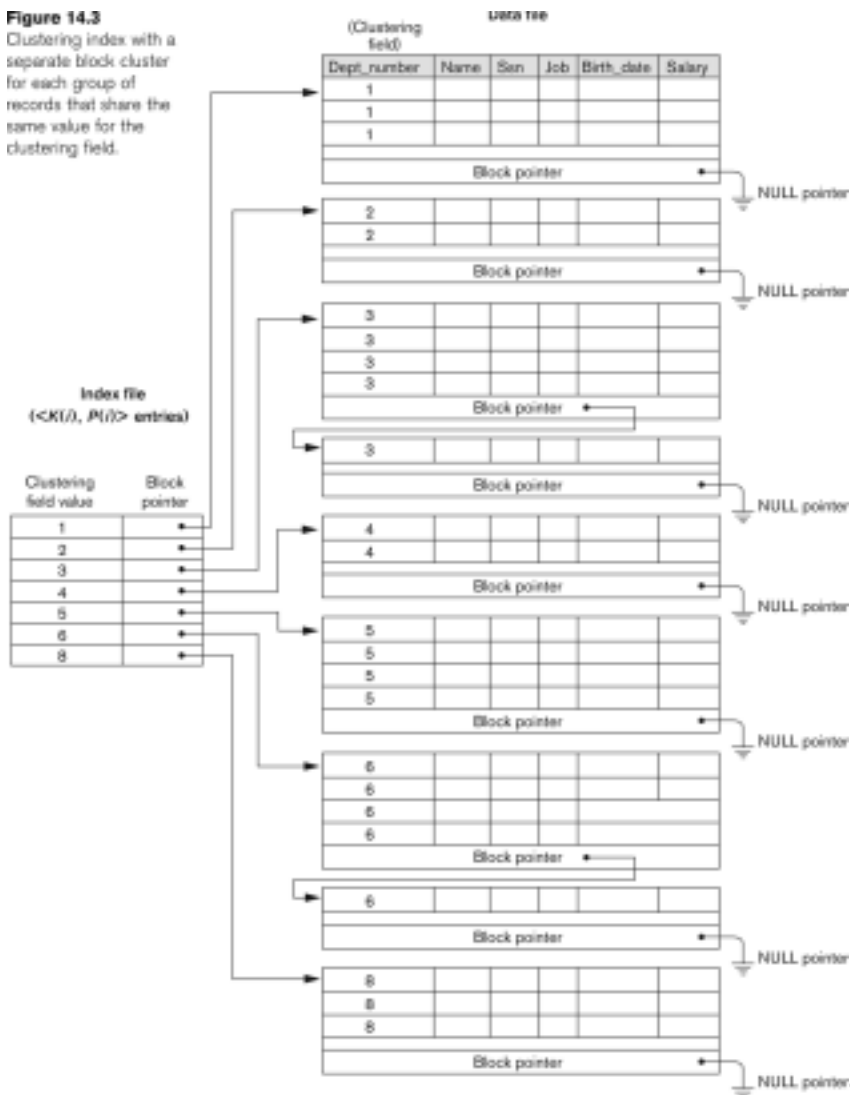


■ FIGURE 14.2

Another Clustering Index Example

Figure 14.3

Clustering index with a separate block cluster for each group of records that share the same value for the clustering field.



Slide 14- 11

Types of Single-Level Indexes

■ Secondary Index

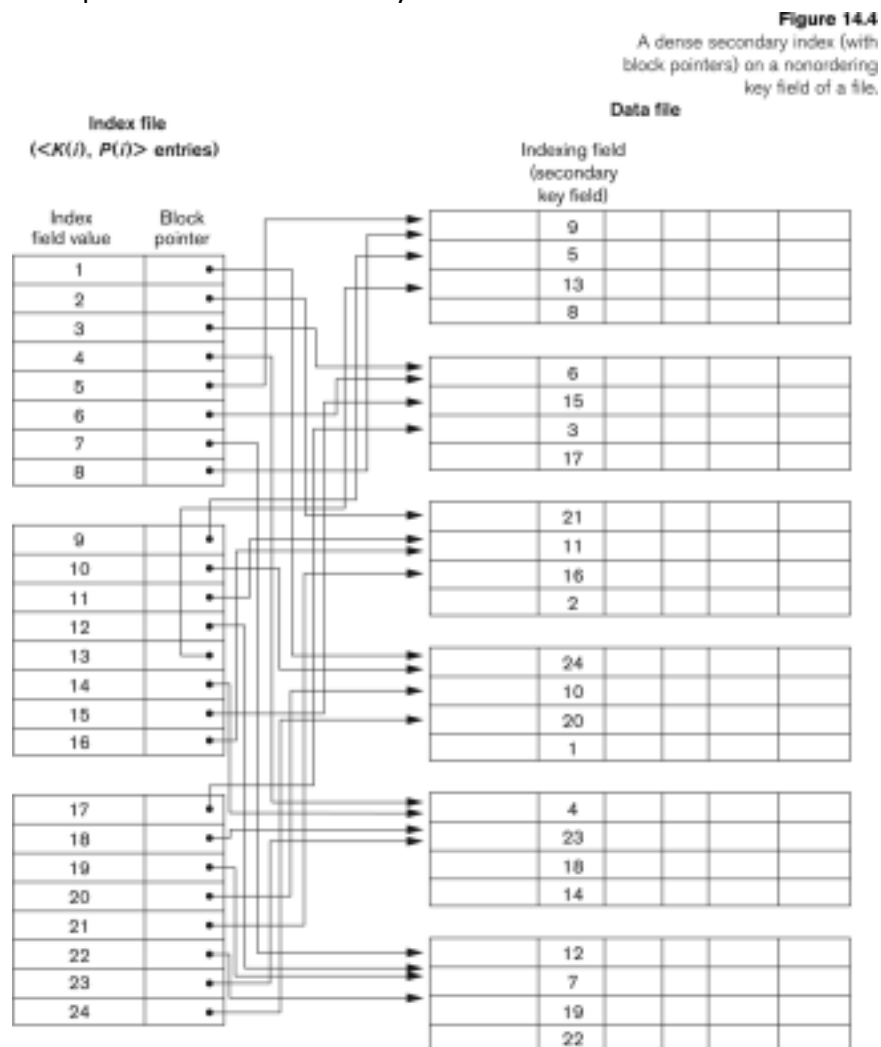
■ A secondary index provides a secondary means of accessing a file for which some primary access already exists.

■ The secondary index may be on a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.

■ The index is an ordered file with two fields.

- The first field is of the same data type as some **non-ordering field** of the data file that is an indexing field.
- The second field is either a **block pointer** or a record pointer. ■ There can be *many* secondary indexes (and hence, indexing fields) for the same file.
- Includes one entry *for each record* in the data file; hence, it is a *dense index*

Example of a Dense Secondary Index



An Example of a Secondary Index

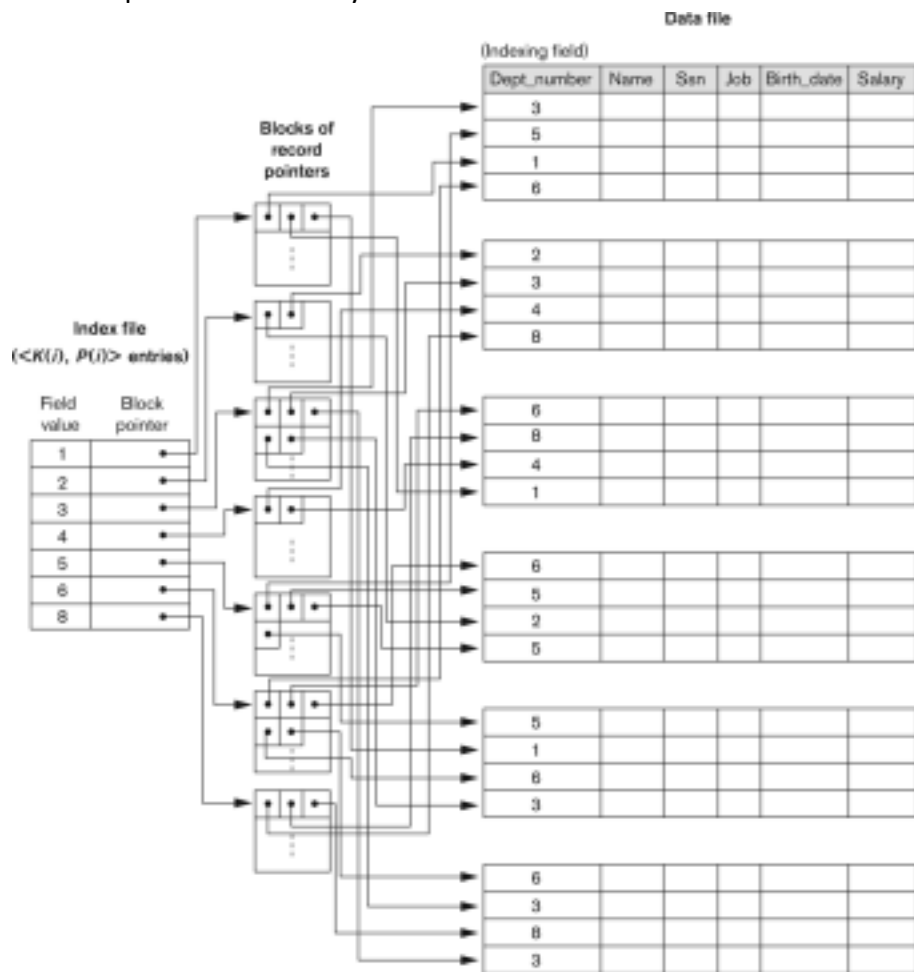


Figure 14.5

A secondary index (with record pointers) on a nonkey field implemented using one level of indirection so that index entries are of fixed length and have unique field values.

Properties of Index Types

TABLE 14.2 PROPERTIES OF INDEX TYPES

TYPE OF INDEX	NUMBER OF (FIRST-LEVEL) INDEX ENTRIES	DENSE OR NONDENSE	BLOCK ANCHORING ON THE DATA FILE
Primary	Number of blocks in data file	Nondense	Yes
Clustering	Number of distinct index field values	Nondense	Yes/no ^a
Secondary (key)	Number of records in data file	Dense	No
Secondary (nonkey)	Number of records ^b or Number of distinct index field values ^c	Dense or Nondense	No

^aYes if every distinct value of the ordering field starts a new block; no otherwise.

^bFor option 1.

^cFor options 2 and 3.

Multi-Level Indexes

- Because a single-level index is an ordered file, we can create a primary index *to the index itself*;
- In this case, the original index file is called the *first-level index* and the index to the index is called the *second-level index*.
- We can repeat the process, creating a third, fourth, ..., top level until all entries of the *top level* fit in one disk block ■ A multi-level index can be created for any type of first level index (primary, secondary, clustering) as long as the first-level index consists of *more than one* disk block

A Two-level Primary Index

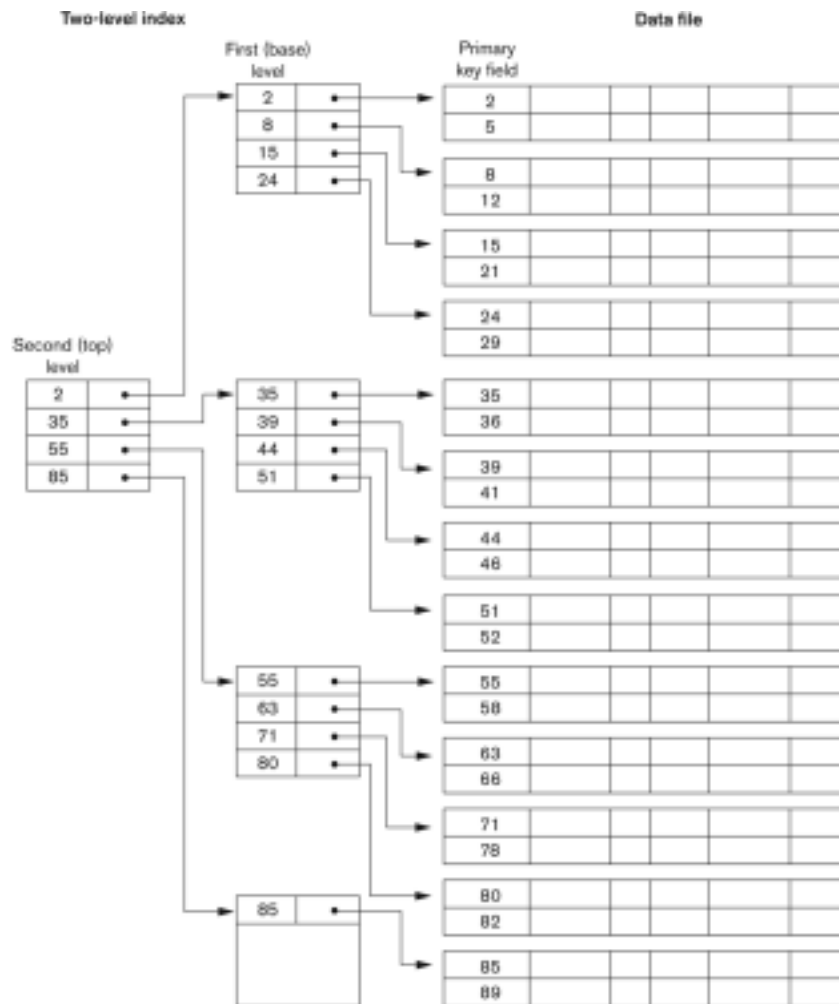


Figure 14.8
A two-level primary index resembling ISAM (Index Sequential Access Method) organization.

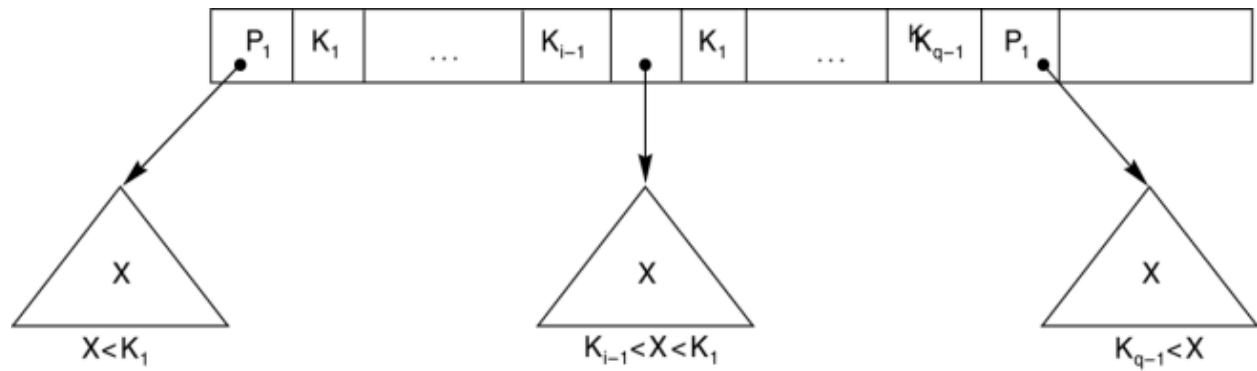
Multi-Level Indexes

■ Such a multi-level index is a form of *search tree* ■ However, insertion and deletion of new index entries is a severe problem because every level of the index is an *ordered file*.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 18**

A Node in a Search Tree with Pointers to Subtrees below It

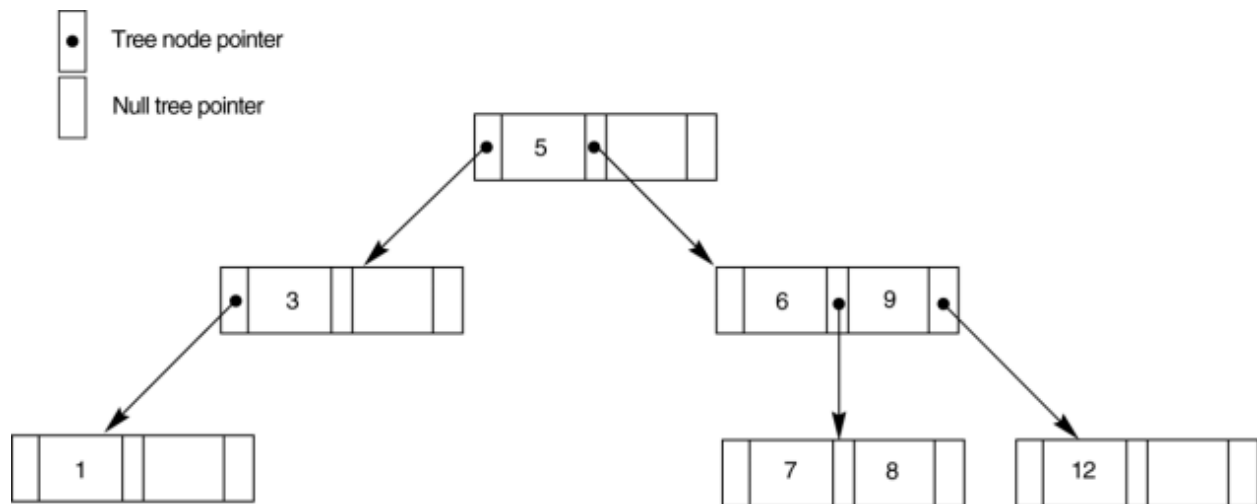
■ FIGURE 14.8



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 19**

FIGURE 14.9

A search tree of order $p = 3$.



Dynamic Multilevel Indexes Using B Trees and B+-Trees

■ Most multi-level indexes use B-tree or B+-tree data structures because of the insertion and deletion problem ■ This leaves space in each tree node (disk block) to allow for new index entries

■ These data structures are variations of search trees that allow efficient insertion and deletion of new search values. ■ In B-Tree and B+-Tree data structures, each node corresponds to a disk block

■ Each node is kept between half-full and completely full

Dynamic Multilevel Indexes Using B Trees and B+-Trees (contd.)

■ An insertion into a node that is not full is quite efficient

■ If a node is full the insertion causes a split into two nodes

- Splitting may propagate to other tree levels
- A deletion is quite efficient if a node does not become less than half full

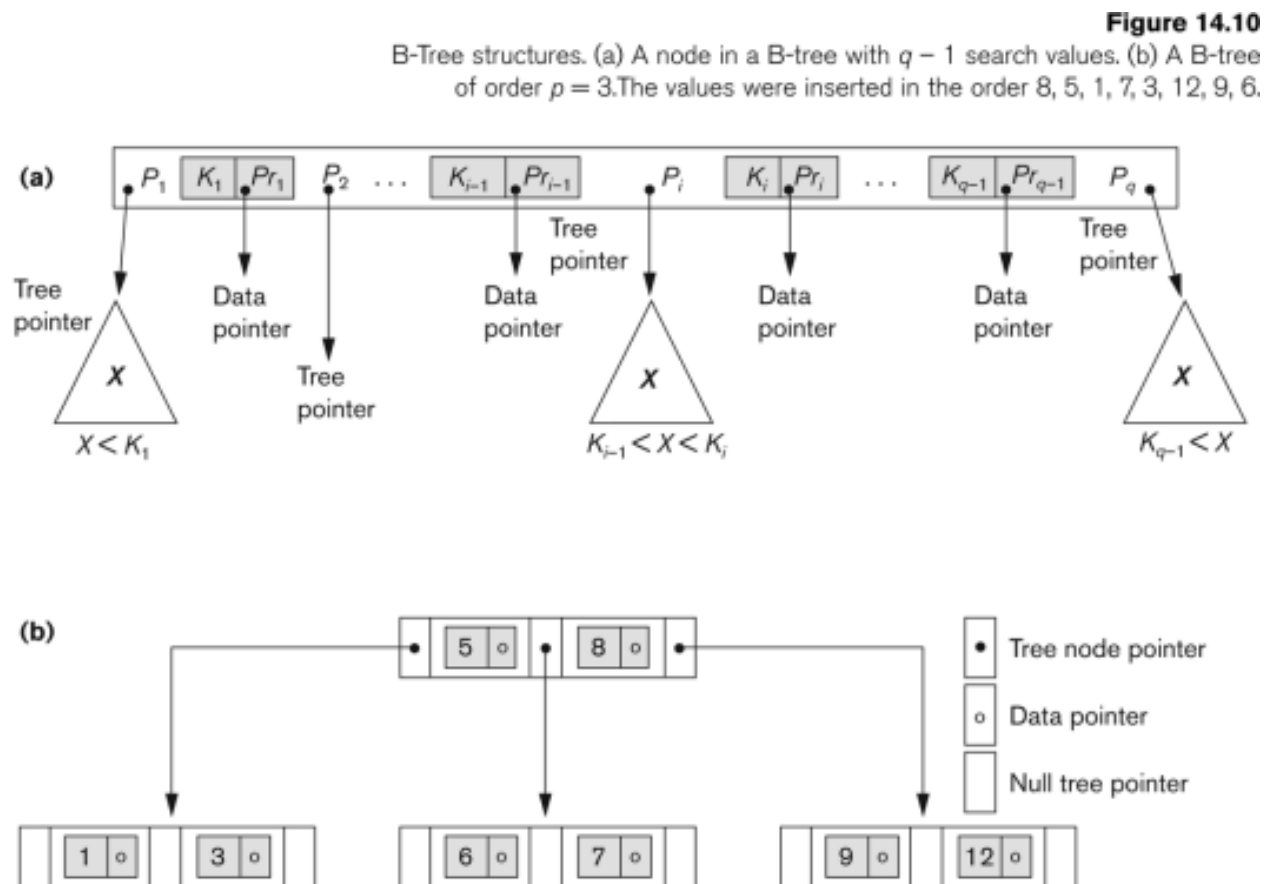
- If a deletion causes a node to become less than half full, it must be merged with neighboring nodes

Difference between B-tree and B+-tree

- In a B-tree, pointers to data records exist at all levels of the tree
- In a B+-tree, all pointers to data records exist at the leaf-level nodes
- A B+-tree can have less levels (or higher capacity of search values) than the corresponding B-tree

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 14- 23**

B-tree Structures

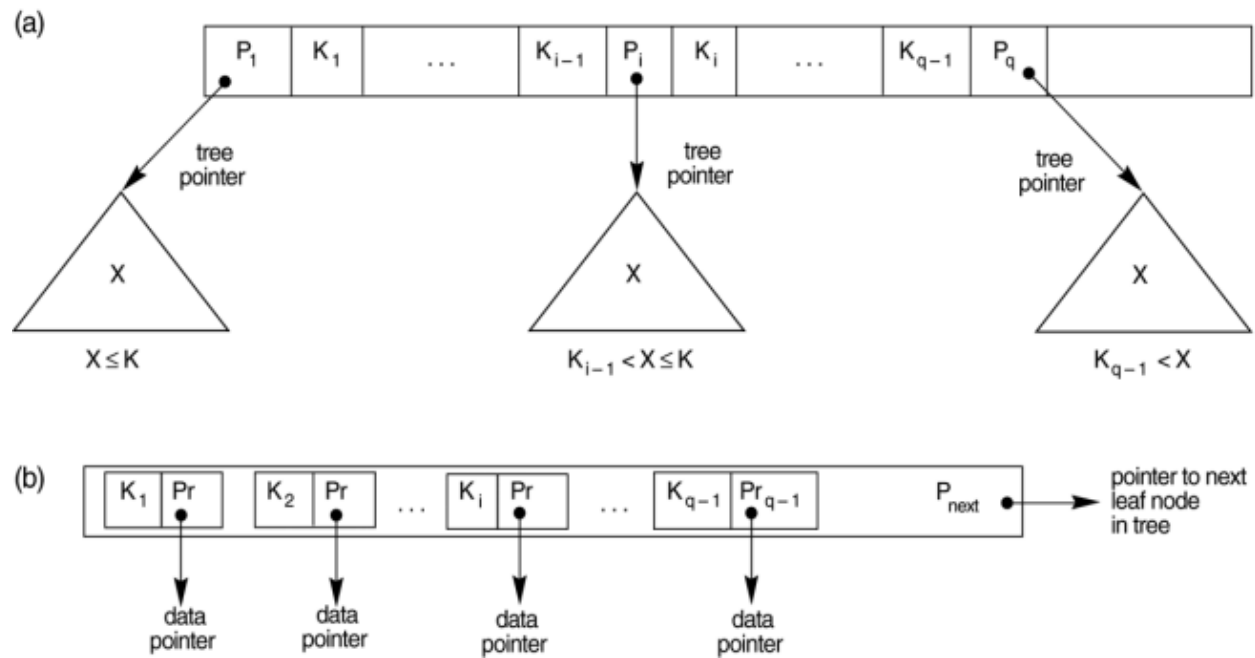


The Nodes of a B+-tree

■ FIGURE 14.11 The nodes of a B+-tree

■ (a) Internal node of a B+-tree with $q - 1$ search values.

■ (b) Leaf node of a B+-tree with $q - 1$ search values and $q - 1$ data pointers.

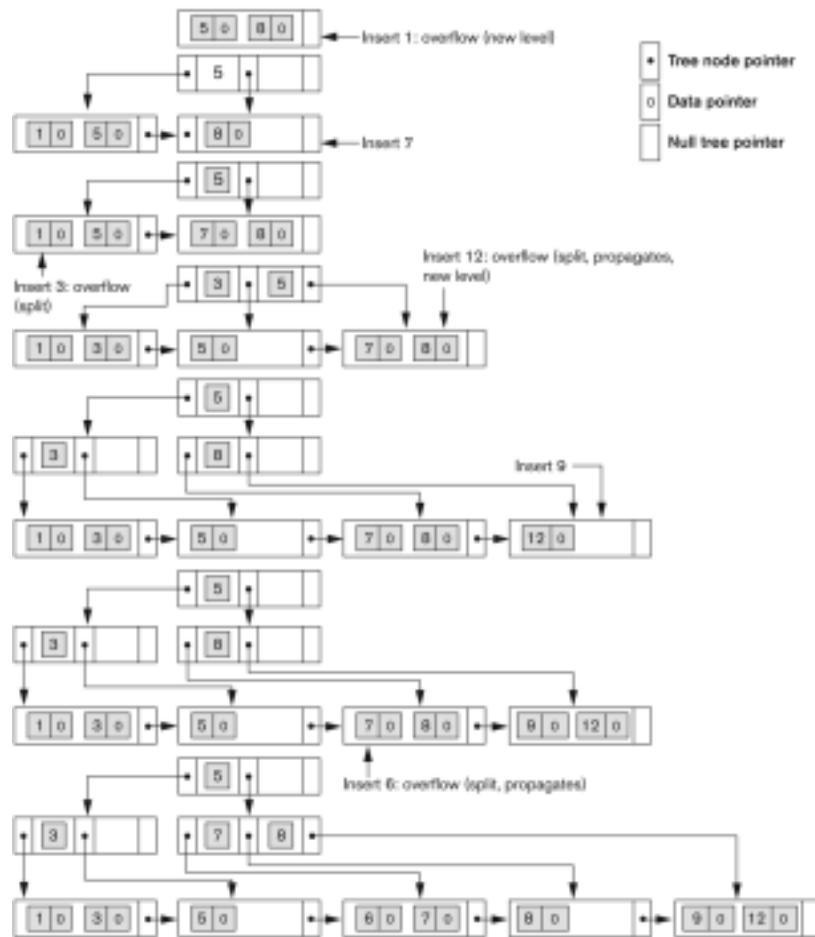


An Example of an Insertion in a B+-tree

Figure 14.12

An example of insertion in a B+-tree with $p = 3$ and $p_{\text{leaf}} = 2$.

Insertion sequence: 8, 5, 1, 7, 3, 12, 9, 6



An Example of a Deletion in a B+-tree

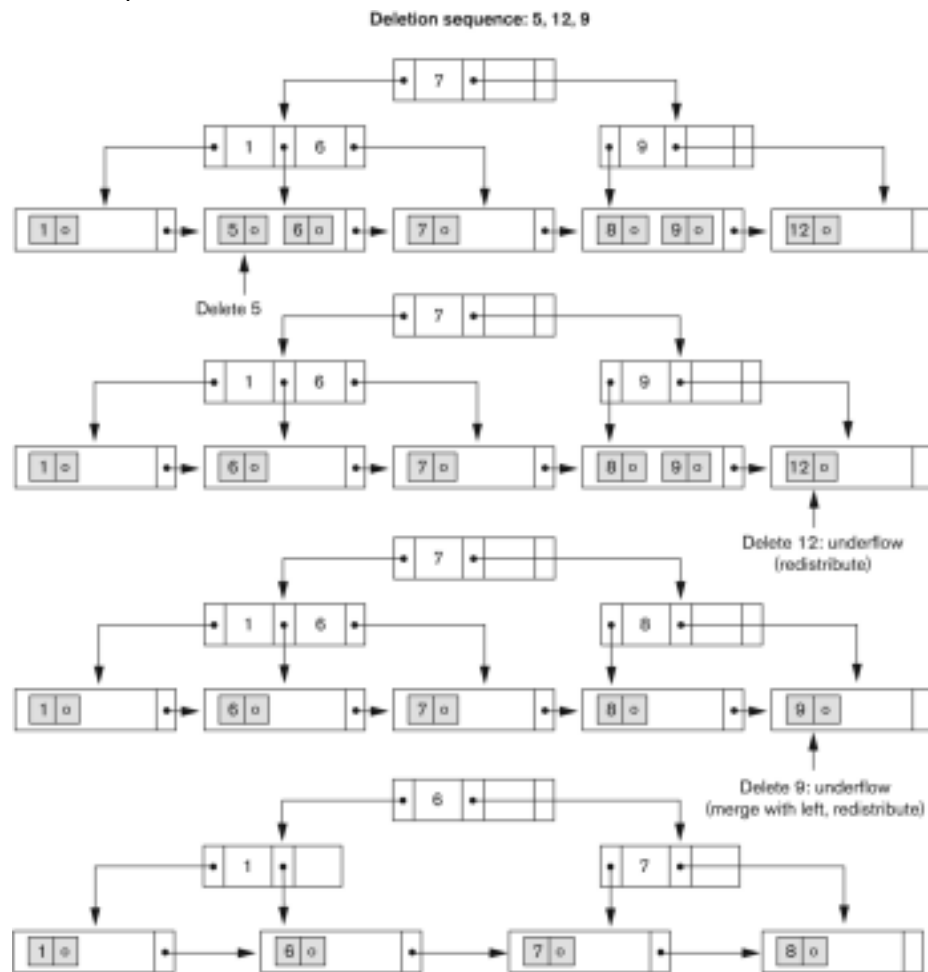


Figure 14.13
An example of deletion from a B+-tree.

Summary

- Types of Single-level Ordered Indexes ■ Primary Indexes
- Clustering Indexes
- Secondary Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys

Chapter 15

Query Processing and Optimization

Query processing

- We will now consider the efficiency of SQL query execution in general by studying query processing and optimization.

Query Processing Steps

- Scanning. Carefully examine the text of the query and identify the language components such as SQL keywords, names of relations, names of attributes, built-in functions, and operators.
- Parsing. Verify that the query syntax conforms to SQL syntax grammar.
- Validating. Check and validate relation and attribute names and verify the semantics of the query.
- Internal representation. Create an internal representation or data structure for the query in the form of a query tree or query graph. Usually this data structure consists of translations of query components into relational algebra operations forming a query tree or query graph.

Query Processing Steps

- Optimizing. Examine , choose an optimal strategy, and produce an execution plan.
- Code generation. Generate code for executing the plan produced by the optimizer.
- Runtime processing. Run the query code, compiled or interpreted, to produce the query result.

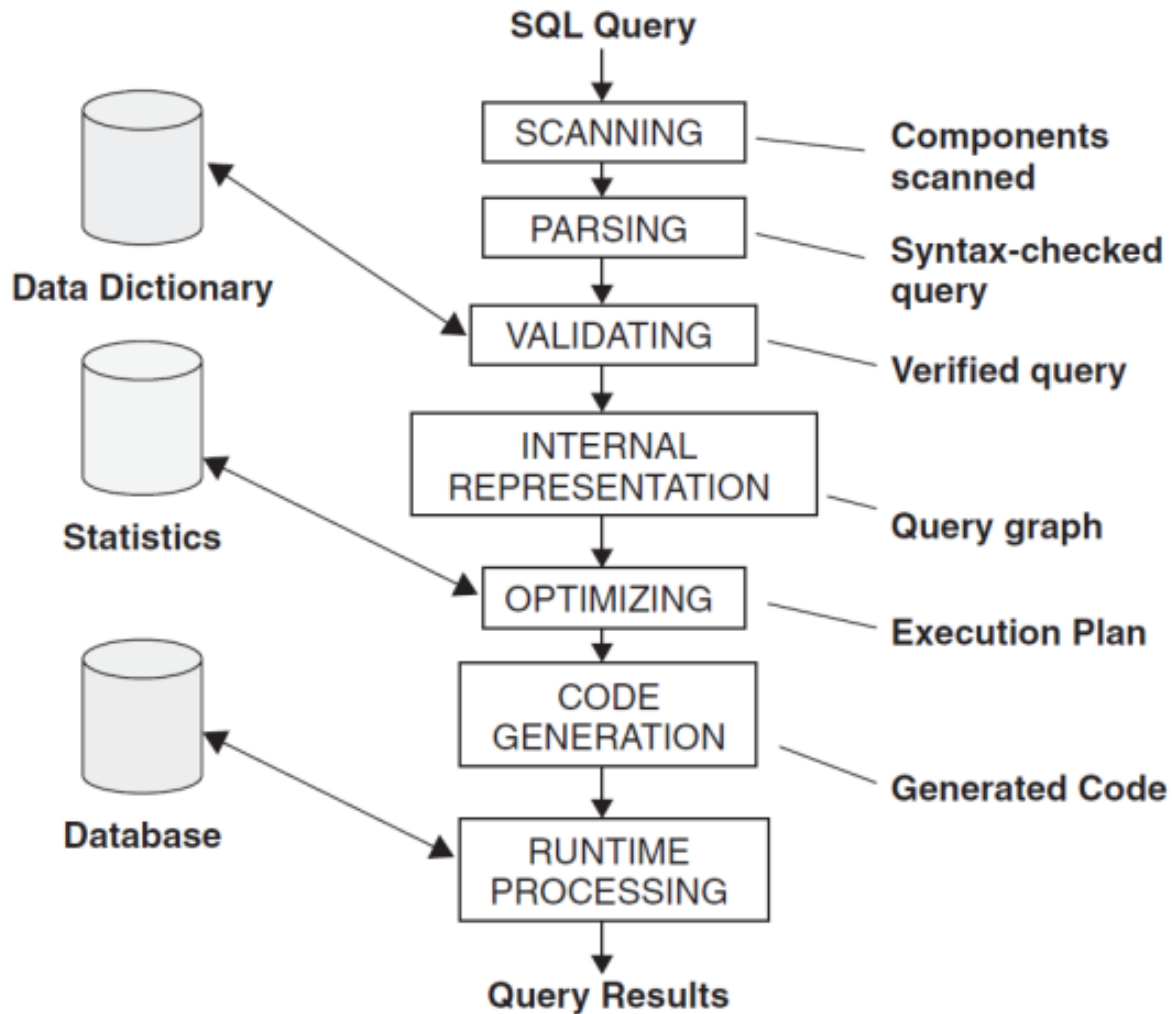


Figure 13-15 Query processing steps.

0. Introduction to Query Processing (1)

■ Two internal representations of a query: ■ **Query Tree**

■ **Query Graph**

■ **Query optimization:**

■ The process of choosing a suitable execution strategy for processing a query.

Translating SQL Queries into Relational Algebra (2)

■ **Example:**

■ For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate.

■ Relation algebra:

$\pi_{PNUMBER, DNUM, LNAME, ADDRESS, BDATE}$

$((\sigma_{PLOCATION='STAFFORD'}(PROJECT)))$

$DNUM=DNUMBER(DEPARTMENT) \text{ MGRSSN=SSN}(EMPLOYEE))$

■ SQL query:

Q2: SELECT P.NUMBER,P.DNUM,E.LNAME, E.ADDRESS, E.BDATE

FROM PROJECT AS P,DEPARTMENT AS D, EMPLOYEE AS E

WHERE P.DNUM=D.DNUMBER AND

D.MGRSSN=E.SSN AND

P.PLOCATION='STAFFORD';

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 15- 8**

Translating SQL Queries into Relational Algebra (2)

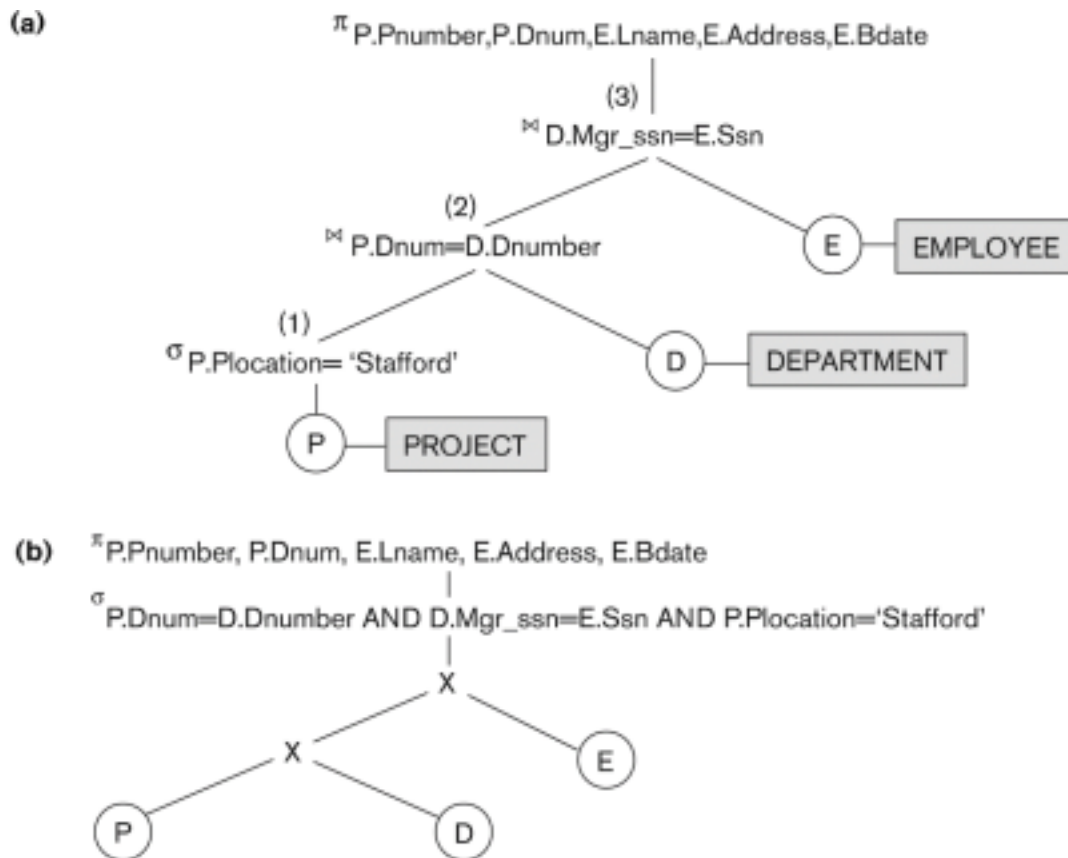


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Translating SQL Queries into Relational Algebra (2)

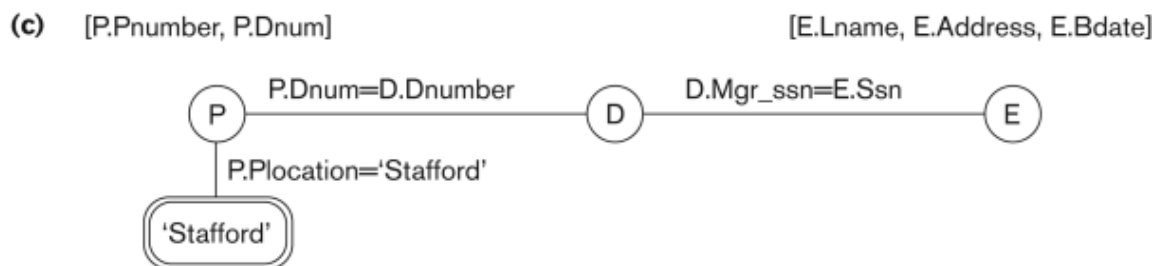


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Optimizer Choices

- Rule-based optimizer
- Preset rules and points

- Rules assign a fixed cost to each operation ■ Cost-based optimizer
- Algorithms based on statistics about objects being accessed
- Adds up processing cost, I/O costs, resource costs to derive total cost

Using Heuristics in Query Optimization(1)

- Process for heuristics optimization

1. The parser of a high-level query generates an initial internal representation;
 2. Apply heuristics rules to optimize the internal representation.
 3. A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.
- The main heuristic is to apply first the operations that reduce the size of intermediate results.
 - E.g., Apply SELECT and PROJECT operations before applying the JOIN or other binary operations.

Using Heuristics in Query Optimization (2)

- **Query tree:**

- A tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as **leaf nodes** of the **tree**, and represents the relational algebra operations as internal nodes.
- An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation. ■ **Query graph:**
- A graph data structure that corresponds to a relational calculus expression. It does *not* indicate an order on which operations to perform first. There is only a *single* graph corresponding to each query.

Using Heuristics in Query Optimization (3)

- Example:

- For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate.

■ Relation algebra:

$\pi_{PNUMBER, DNUM, LNAME, ADDRESS, BDATE}$

$((\sigma_{PLOCATION='STAFFORD'}(PROJECT)))$

$DNUM=DNUMBER(DEPARTMENT) \text{ MGRSSN=SSN}(EMPLOYEE))$

■ SQL query:

Q2: SELECT P.NUMBER,P.DNUM,E.LNAME, E.ADDRESS, E.BDATE

FROM PROJECT AS P,DEPARTMENT AS D, EMPLOYEE AS E

WHERE P.DNUM=D.DNUMBER AND

D.MGRSSN=E.SSN AND

P.PLOCATION='STAFFORD';

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 15- 14**

Using Heuristics in Query Optimization (4)

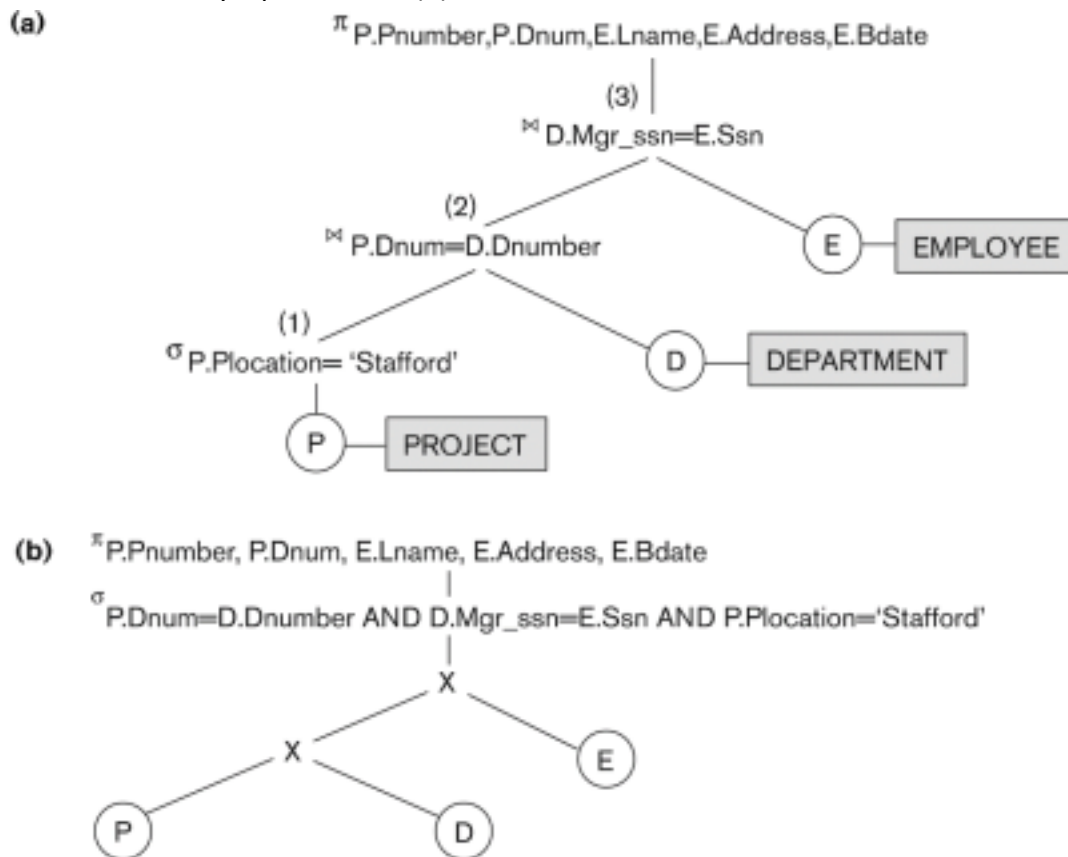


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Using Heuristics in Query Optimization (5)

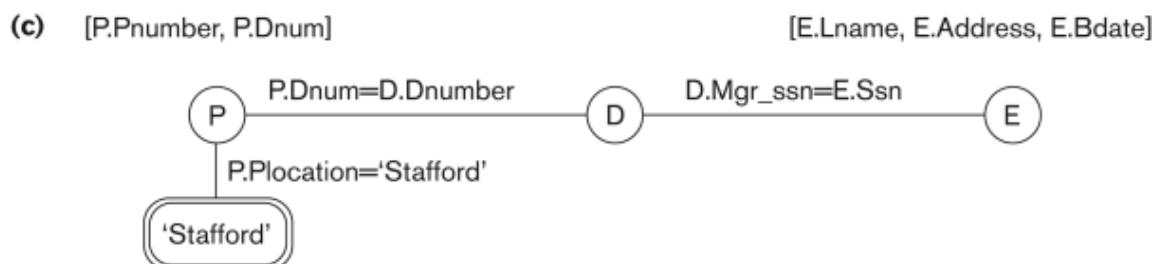


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Using Heuristics in Query Optimization (6)

■ Heuristic Optimization of Query Trees:

■ The same query could correspond to many different relational algebra expressions — and hence many different query trees.

■ The task of heuristic optimization of query trees is to find a **final query tree** that is efficient to execute.

■ Example:

Q: SELECT LNAME

FROM EMPLOYEE, WORKS_ON, PROJECT WHERE PNAME = 'AQUARIUS' AND

PNMUBER=PNO AND ESSN=SSN

AND BDATE > '1957-12-31';

Using Heuristics in Query Optimization (7)

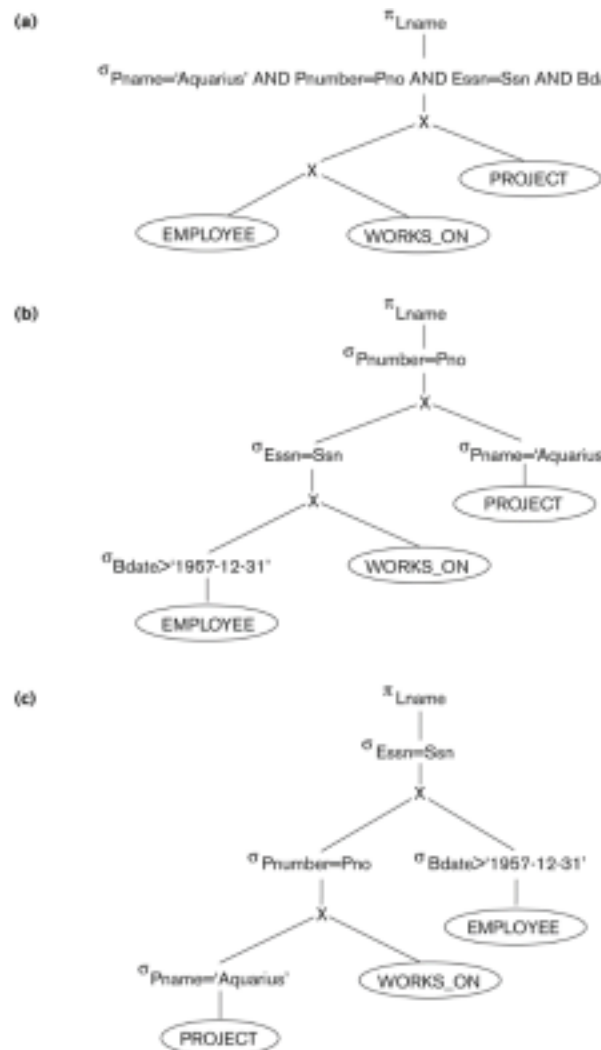


Figure 15.5

Steps in converting a query tree during heuristic optimization.

(a) Initial (canonical) query tree for SQL query Q.

(b) Moving SELECT operations down the query tree.

(c) Applying the more restrictive SELECT operation first.

(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.

(e) Moving PROJECT operations down the query tree.

Using Heuristics in Query Optimization (8)

(d)



(e)

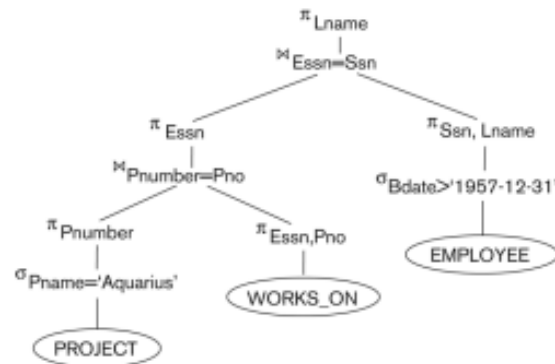


Figure 15.5

Steps in converting a query tree during heuristic optimization.

- (a) Initial (canonical) query tree for SQL query Q.
- (b) Moving SELECT operations down the query tree.
- (c) Applying the more restrictive SELECT operation first.
- (d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.
- (e) Moving PROJECT operations down the query tree.

Using Heuristics in Query Optimization (9)

■ General Transformation Rules for Relational Algebra Operations: 1. Cascade of σ : A conjunctive selection condition can be broken up into a cascade (sequence) of individual σ operations:

$$\sigma_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } cn}(R) = \sigma_{c1}(\sigma_{c2}(\dots(\sigma_{cn}(R))\dots))$$

2. Commutativity of σ : The σ operation is commutative: ■ $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$

3. Cascade of π : In a cascade (sequence) of π operations, all but the last one can be ignored:

$$\pi_{List1}(\pi_{List2}(\dots(\pi_{Listn}(R))\dots)) = \pi_{List1}(R)$$

4. Commuting σ with π : If the selection condition c involves only the attributes $A1, \dots, An$ in the projection list, the two operations can be commuted:

$$\pi_{A1, A2, \dots, An}(\sigma_c(R)) = \sigma_c(\pi_{A1, A2, \dots, An}(R))$$

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe **Slide 15- 20**

Using Heuristics in Query Optimization (10)

■ General Transformation Rules for Relational Algebra Operations (contd.):

5. Commutativity of (and x): The operation is commutative as is the x operation:

■ $R \bowtie S = S \bowtie R$; $R \times S = S \times R$

6. Commuting σ with (or x): If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R—the two operations can be commuted as follows: ■ $\sigma_c(R \bowtie S) = (\sigma_c(R)) \bowtie S$

■ Alternatively, if the selection condition c can be written as (c1 and c2), where condition c1 involves only the attributes of R and condition c2 involves only the attributes of S, the operations commute as follows: ■ $\sigma_c(R \bowtie S) = (\sigma_{c1}(R)) \bowtie (\sigma_{c2}(S))$

Using Heuristics in Query Optimization (11)

■ General Transformation Rules for Relational Algebra Operations (contd.):

7. Commuting π with (or x): Suppose that the projection list is $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, where $A_1, \dots, A_n$ are attributes of R and $B_1, \dots, B_m$ are attributes of S. If the join condition c involves only attributes in L, the two operations can be commuted as follows:

■ $\pi_L(R \bowtie S) = (\pi_{A_1, \dots, A_n}(R)) \bowtie (\pi_{B_1, \dots, B_m}(S))$ ■ If the join condition C contains additional attributes not in L, these must be added to the projection list, and a final π operation is needed.

Using Heuristics in Query Optimization (12)

■ General Transformation Rules for Relational Algebra Operations (contd.):

8. Commutativity of set operations: The set operations $\cup$ and $\cap$ are commutative but “−” is not.

9. Associativity of $\cup$, $\cap$, $\cup$, and $\cap$: These four operations are individually associative; that is, if θ stands for any one of these four operations (throughout the expression), we have

■ $(R \theta S) \theta T = R \theta (S \theta T)$

10. Commuting σ with set operations: The σ operation commutes with $\cup$, $\cap$, and “−”. If θ stands for any one of these three operations, we have

■ $\sigma_c(R \theta S) = (\sigma_c(R)) \theta (\sigma_c(S))$

Using Heuristics in Query Optimization (13)

■ General Transformation Rules for Relational Algebra Operations (contd.):

■ The π operation commutes with $\cup$.

$$\pi_L(R \cup S) = (\pi_L(R)) \cup (\pi_L(S))$$

■ Converting a (σ, x) sequence into : If the condition c of a σ that follows a x Corresponds to a join condition, convert the (σ, x) sequence into a as follows:

$$(\sigma_c(R \times S)) = (R \bowtie_c S)$$

■ Other transformations

Using Heuristics in Query Optimization (15)

■ Summary of Heuristics for Algebraic Optimization: 1. The main heuristic is to apply first the operations that reduce the size of intermediate results.

2. Perform select operations as early as possible to reduce the number of tuples and perform project operations as early as possible to reduce the number of attributes. (This is done by moving select and project operations as far down the tree as possible.)

3. The select and join operations that are most restrictive should be executed before other similar operations. (This is done by reordering the leaf nodes of the tree among themselves and adjusting the rest of the tree

appropriately.)

Using Heuristics in Query Optimization (16)

■ Query Execution Plans

■ An **execution plan** for a relational algebra query consists of a combination of the relational algebra query tree and information about the access methods to be used for each relation as well as the methods to be used in computing the relational operators stored in the tree.

■ **Materialized evaluation**: the result of an operation is stored as a temporary relation.

■ **Pipelined evaluation**: as the result of an operator is produced, it is forwarded to the next operator in sequence.

Using Selectivity and Cost Estimates in Query Optimization (1)

■ **Cost-based query optimization**:

■ Estimate and compare the costs of executing a query using different execution strategies and choose the strategy with the lowest cost estimate. ■ (Compare to heuristic query optimization)

■ Issues

- Cost function

- Number of execution strategies to be considered

Using Selectivity and Cost Estimates in Query Optimization (2)

- Cost Components for Query Execution 1. Access cost to secondary storage

- 2. Storage cost

- 3. Computation cost

- 4. Memory usage cost

- 5. Communication cost

- Note: Different database systems may focus on different cost components.

Using Selectivity and Cost Estimates in Query Optimization (3)

- Catalog Information Used in Cost Functions ■ Information about the size of a file

- number of records (tuples) (r),

- record size (R),

- number of blocks (b)

- blocking factor (bfr)

- Information about indexes and indexing attributes of a file ■ Number of levels (x) of each multilevel index

- Number of first-level index blocks (bl_1)

- Number of distinct values (d) of an attribute

- Selectivity (sl) of an attribute

- Selection cardinality (s) of an attribute. ($s = sl * r$)

Overview of Query Optimization in Oracle

- Oracle DBMS V8

- **Rule-based query optimization:** the optimizer chooses execution plans based on heuristically ranked operations. ■ (Currently it is being phased out)

- **Cost-based query optimization:** the optimizer examines alternative access paths and operator algorithms and chooses the execution plan with lowest estimate cost.

- The query cost is calculated based on the estimated usage of resources such as I/O, CPU and memory needed.
 - Application developers could specify hints to the ORACLE query optimizer.
 - The idea is that an application developer might know more information about the data.
-

Database Systems:

Design, Implementation, and Management

13th Edition

Chapter 11 Database Performance Tuning and Query Optimization

Objectives

In this chapter, you will learn:

- Basic database performance-tuning concepts
- How a DBMS processes SQL queries
- About the importance of indexes in query processing

Database Systems, 13th Edition ²

Objectives (cont'd.)

- About the types of decisions the query optimizer has to make
- Some common practices used to write efficient SQL code
- How to formulate queries and tune the DBMS for optimal performance

Database Systems, 13th Edition ³

Database Performance-Tuning Concepts

- One of the main functions of a database system is to provide timely answers to end users. End users interact with the DBMS through the use of queries to generate information, using the following sequence:

1. The end-user (client-end) application generates a query.
2. The query is sent to the DBMS (server end).

3. The DBMS (server end) executes the query.

4. The DBMS sends the resulting data set to the end-user (client-end) application.

- End users expect their **queries to return results as quickly as possible**.

- How do you know that the performance of a database is good? Good database performance is hard to evaluate.

- Regardless of end-user perceptions, ***the goal of database performance is to execute queries as fast as possible***. • Therefore, database performance must be closely **monitored** and **regularly tuned**.

Database Systems, 13th Edition ⁴

Database Performance-Tuning Concepts

- Database performance tuning refers to a set of activities and procedures designed to reduce the response time of the database system—that is, to ensure that an end-user query is processed by the DBMS in the minimum amount of time.

- The time required by a query to return a result set depends on many factors, which tend to be wide-ranging and to vary among environments and among vendors. In general, **the performance of a typical DBMS is constrained by three main factors: CPU processing power, available primary memory (RAM), and input/output (hard disk and network) throughput**.

- Fine-tuning the performance of a system requires a holistic approach. That is, ***all factors must be checked to ensure that each one operates at its optimum level and has sufficient resources to minimize the occurrence of bottlenecks***.

Database Systems, 13th Edition ⁵

Database Performance-Tuning Concepts

- **Good database performance starts with good database design. *No amount of finetuning will make a poorly designed database perform as well as a well-designed database.***

TABLE
11.1

General Guidelines for Better System Performance

	SYSTEM RESOURCES	CLIENT	SERVER
Hardware	CPU	The fastest possible Dual-core CPU or higher	The fastest possible Multiple processors (quad-core technology)
	RAM	The maximum possible	The maximum possible
	Hard disk	Fast SATA/EIDE hard disk with sufficient free hard disk space	Multiple high-speed, high-capacity hard disks (SCSI/SATA/Firewire/Fibre Channel) in RAID configuration
	Network	High-speed connection	High-speed connection
Software	Operating system	Fine-tuned for best client appli- cation performance	Fine-tuned for best server application performance
	Network	Fine-tuned for best throughput	Fine-tuned for best throughput
	Application	Optimize SQL in client application	Optimize DBMS server for best performance

Database Systems, 13th Edition ⁶

Performance Tuning:

Client and Server

- In general, database performance-tuning activities can be divided into those on the client side and those on the server side.
- • On the **client side**, the objective is to generate a SQL query that returns the correct answer in the least amount of time, using the minimum amount of resources at the server end. The activities required to achieve that goal are commonly referred to as **SQL performance tuning**.
- • On the **server side**, the DBMS environment must be properly configured to respond to clients' requests in the fastest way possible, while making optimum use of existing resources. The activities required to achieve that goal are commonly referred to as **DBMS performance tuning**.

Database Systems, 13th Edition ⁷

DBMS Architecture

- All data in database are stored in **data files**. A data file can contain **rows from a single table**, or it can **contain rows from many different tables**.
- Data files
 - Automatically expand in predefined increments known as **extents**. For example, if more space is required, the DBA can define that each new extent will be in 10 KB or 10 MB increments.
 - **Data files are generally grouped in file groups or table spaces**. A table space or file group is a **logical grouping of several data files that store data with similar characteristics**. For example,

you might have a *system* table space where the **data dictionary** table data is stored, a *user data* table space to store the **user-created tables**, an *index* table space to hold all **indexes**, and a **temporary table space to do temporary sorts, grouping, and so on**. Each time you create a new database, the **DBMS automatically creates a minimum set of table spaces**.

Database Systems, 13th Edition ⁸

DBMS Architecture

- The **data cache, or buffer cache**, is a **shared, reserved memory area** that stores the **most recently accessed data blocks in RAM**. The data read from the data files is stored in the data cache after the data has been read or before the data is written to the data files. The data cache **also caches system catalog data and the contents of the indexes**.
- The **SQL cache, or procedure cache**, is a **shared, reserved memory area** that stores **the most recently executed SQL statements or PL/SQL procedures**, including triggers and functions. The SQL cache **does not store the SQL written by the end user**. Rather, the SQL cache **stores a “processed” version of the SQL** that is ready for execution by the DBMS.

Database Systems, 13th Edition ⁹

DBMS Architecture

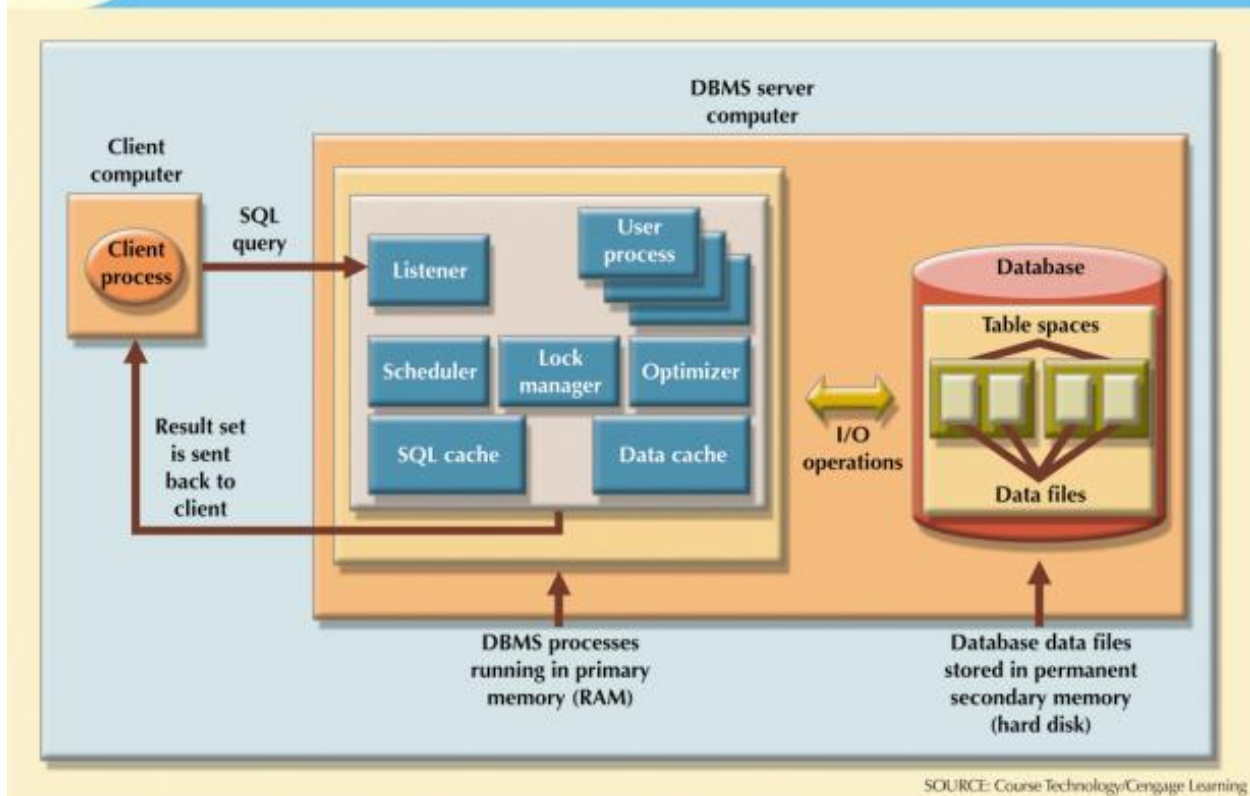
- To work with the data, the **DBMS must retrieve the data from permanent storage and place it in RAM**. In other words, the data is retrieved from the **data files and placed in the data cache**.
- • To move data from permanent storage (data files) to RAM (data cache), the **DBMS issues I/O requests and waits for the replies**. An **input/output (I/O) request is a lowlevel data access operation that reads or writes data to and from computer devices**, such as memory, hard disks, video, and printers. Note that an **I/O disk read operation retrieves an entire physical disk block, generally containing multiple rows, from permanent storage to the data cache, even if you will use only one attribute from only one row**. The **physical disk block size depends on the operating system** and could be 4K, 8K, 16K, 32K, 64K, or even larger.
- Working with data in the **data cache is many times faster than working with data in the data files**

Database Systems, 13th Edition ¹⁰

DBMS Architecture

- Most performance-tuning activities focus on minimizing the number of I/O operations because using I/O operations is many times slower than reading data from the data cache.

FIGURE 11.1 Basic DBMS architecture



DBMS Architecture (cont'd.)

- Input/output request: low-level data access operation to/from computer devices
- Data cache is faster than data in data files – DBMS does not wait for hard disk to retrieve data
- Majority of performance-tuning activities focus on minimizing I/O operations
- Typical DBMS processes:
 - Listener, user, scheduler, lock manager, optimizer

DBMS Architecture (cont'd.)

- **Listener**. The listener process listens for clients' requests and handles the processing of the SQL requests to other DBMS processes. Once a request is received, the listener passes the request to the appropriate user process.
- **User**. The DBMS creates a user process to manage each client session. Therefore, when you log on to the DBMS, you are assigned a user process.
- **Scheduler**. The scheduler process organizes the concurrent execution of SQL requests.
- **Lock manager**. This process manages all locks placed on database objects, including disk pages.
- **Optimizer**. The optimizer process analyzes SQL queries and finds the most efficient way to access the data.

Database Systems, 13th Edition ¹⁴

Database Query Optimization Modes

- Automatic query optimization
 - DBMS finds the most cost-effective access path without user intervention
- Manual query optimization
 - Requires that the optimization be selected and scheduled by the end user or programmer
- Static query optimization
 - Takes place at compilation time. In other words, the best optimization strategy is selected when the query is compiled by the DBMS. This approach is common when SQL statements are embedded in procedural programming languages such as C# or Visual Basic .NET.
- Dynamic query optimization
 - Takes place at execution time. Therefore, access strategy is dynamically determined by the DBMS at run time, using the most up to-date information about the database.

Database Systems, 13th Edition ¹⁵

Database Query Optimization Modes (cont'd.)

- Statistically based query optimization algorithm
 - Uses statistical information about the database
 - Dynamic statistical generation mode. The DBMS automatically evaluates and updates the statistics after each data access operation.

- Manual statistical generation mode. The statistics must be updated periodically through a user-selected utility such as IBM's RUNSTAT command, which is used by DB2 DBMSs. • Rule-based query optimization algorithm
- Based on a set of user-defined rules to determine the best query access strategy.

Database Systems, 13th Edition ¹⁶

Database Statistics

- Measurements about database objects and available resources:
 - Tables
 - Indexes
 - Number of processors used
 - Processor speed
 - Temporary space available
- Database statistics can be **gathered manually** by the DBA or **automatically by the DBMS**. For example, many DBMS vendors support the **ANALYZE** command in SQL to gather statistics. In addition, many vendors have their own routines to gather statistics.
- Make critical decisions about improving query processing efficiency

Database Systems, 13th Edition ¹⁷

TABLE 11.2 Sample Database Statistics Measurements

DATABASE OBJECT	SAMPLE MEASUREMENTS
Tables	Number of rows, number of disk blocks used, row length, number of columns in each row, number of distinct values in each column, maximum value in each column, minimum value in each column, and columns that have indexes
Indexes	Number and name of columns in the index key, number of key values in the index, number of distinct key values in the index key, histogram of key values in an index, and number of disk pages used by the index
Environment Resources	Logical and physical disk block size, location and size of data files, and number of extents per data file

Database Systems, 13th Edition ¹⁸

Query Processing

- DBMS processes queries in three phases

–Parsing

- DBMS parses the query and chooses the most efficient access/execution plan

–Execution

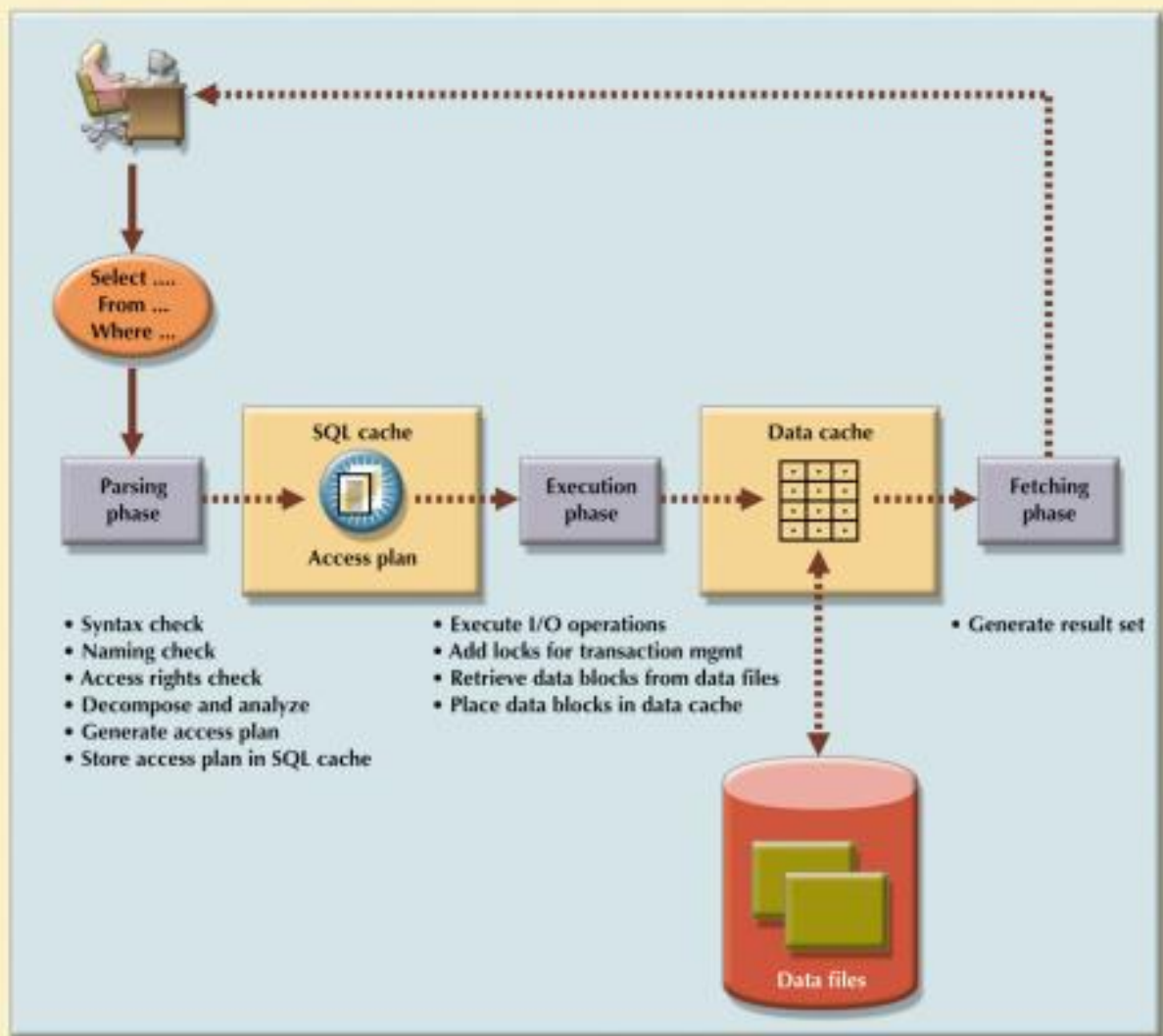
- DBMS executes the query using chosen execution plan –Fetching

- DBMS fetches the data and sends the result back to the client

The processing of SQL **DDL** statements (such as CREATE TABLE) is different from the processing required by **DML** statements. The difference is that a **DDL** statement actually **updates the data dictionary tables or system catalog**, while a **DML** statement (SELECT, INSERT, UPDATE, or DELETE) mostly **manipulates end-user data**.

FIGURE
11.2

Query processing



SOURCE: Course Technology/Cengage Learning

Database Systems, 13th Edition ²⁰

SQL Parsing Phase

- Break down query into smaller units
- Transform original SQL query into slightly different version of original SQL code
- Fully equivalent
- Optimized query results are always the same as original query
- More efficient
- Optimized query will almost always execute faster than original query

Database Systems, 13th Edition ²¹

SQL Parsing Phase (cont'd.)

- Query optimizer analyzes SQL query and finds most efficient way to access data
- Validated for syntax compliance
- Validated against data dictionary
- Tables and column names are correct
- User has proper access rights
- Analyzed and decomposed into components – Optimized
- Prepared for execution

Database Systems, 13th Edition ²²

SQL Parsing Phase (cont'd.)

- Access plans are DBMS-specific
- Translate client's SQL query into a series of complex I/O operations
- Required to read the data from the physical data files and generate result set
- DBMS checks if access plan already exists for query in SQL cache
- DBMS reuses the access plan to save time • If not, optimizer evaluates various plans – Chosen plan placed in SQL cache

Database Systems, 13th Edition ²³

**TABLE
11.3**

Sample DBMS Access Plan I/O Operations

OPERATION	DESCRIPTION
Table scan (full)	Reads the entire table sequentially, from the first row to the last, one row at a time (slowest)
Table access (row ID)	Reads a table row directly, using the row ID value (fastest)
Index scan (range)	Reads the index first to obtain the row IDs and then accesses the table rows directly (faster than a full table scan)
Index access (unique)	Used when a table has a unique index in a column
Nested loop	Reads and compares a set of values to another set of values, using a nested loop style (slow)
Merge	Merges two data sets (slow)
Sort	Sorts a data set (slow)

Database Systems, 13th Edition ²⁴

SQL Execution Phase

SQL Fetching Phase

- All I/O operations indicated in access plan are executed
- Locks acquired
- Data retrieved and placed in data cache – Transaction management commands processed •
- Rows of resulting query result set are returned to client
- DBMS may use temporary table space to store temporary data

Database Systems, 13th Edition ²⁵

Query Processing Bottlenecks

- Delay introduced in the processing of an I/O operation that slows the system
- CPU
- RAM
- Hard disk
- Network
- Application code

Database Systems, 13th Edition ²⁶

Indexes and Query Optimization

- Indexes

- Crucial in speeding up data access
- Facilitate searching, sorting, and using aggregate functions as well as join operations – Ordered set of values that contains index key and pointers
- More efficient to use index to access table than to scan all rows in table sequentially

Database Systems, 13th Edition ²⁷

Indexes and Query Optimization (cont'd.)

- Data sparsity: number of different values a column could possibly have
- Indexes implemented using:
 - **Hash indexes**: This type of index is good for simple and fast lookup operations based on **equality conditions**—for example, LNAME="Scott" and FNAME="Shannon".
 - **B-tree indexes**: This is the **default and most common type of index used in databases**. The B-tree index is **used mainly in tables in which column values repeat a relatively small number of times**.
 - **Bitmap indexes**: These indexes are **used mostly in data warehouse applications** in tables with a large number of rows in which a small number of column values repeat many times. Bitmap indexes tend to **use less space than B-tree indexes** because they **use bits instead of bytes** to store their data.
- DBMSs determine best type of index to use

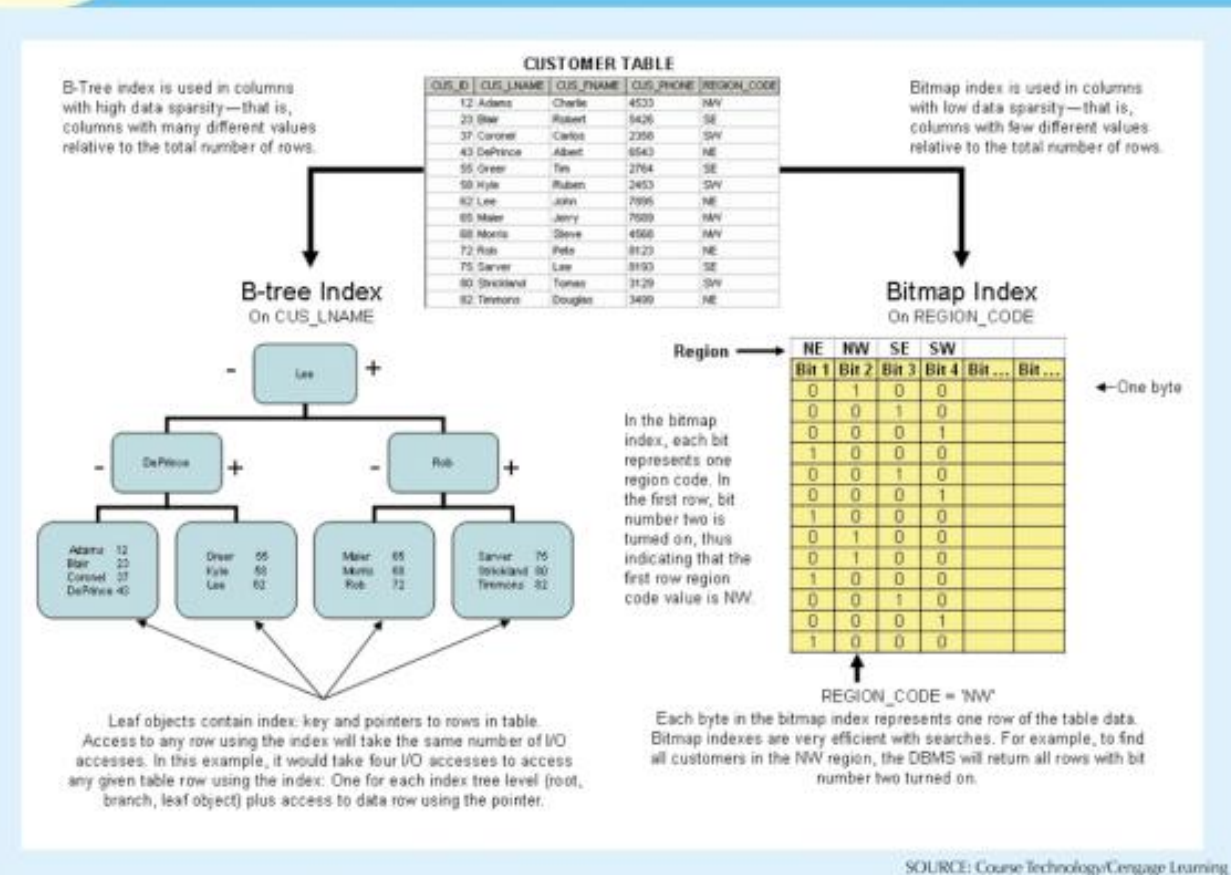
Database Systems, 13th Edition ²⁸

FIGURE 11.3 Index representation for the CUSTOMER table



FIGURE 11.4

B-tree and bitmap index representation



SOURCE: Course Technology/Cengage Learning

Database Systems, 13th Edition ³⁰

Optimizer Choices

- Rule-based optimizer
- Preset rules and points
- Rules assign a fixed cost to each operation
- Cost-based optimizer
- Algorithms based on statistics about objects being accessed
- Adds up processing cost, I/O costs, resource costs to derive total cost

Database Systems, 13th Edition ³¹

Optimizer Choices

- The **optimizer's objective** is to find alternative ways to execute a query—to evaluate the “cost” of each alternative and then to choose the one with the lowest cost.

- Example:

Assume that you want to list all products provided by a vendor based in Florida. To acquire that information, you could write the following query:

```
SELECT P_CODE, P_DESCRIPT, P_PRICE, V_NAME, V_STATE FROM PRODUCT, VENDOR
WHERE PRODUCT.V_CODE = VENDOR.V_CODE AND
VENDOR.V_STATE = 'FL';
```

Furthermore, assume that the database statistics indicate the following:

- The PRODUCT table has 7,000 rows.

- The VENDOR table has 300 rows.
- Ten vendors are located in Florida.
- One thousand products come from vendors in Florida.

Database Systems, 13th Edition ³²

TABLE 11.4 Comparing Access Plans and I/O Costs

PLAN	STEP	OPERATION	I/O OPERATIONS	I/O COST	RESULTING SET ROWS	TOTAL I/O COST
A	A1	Cartesian product (PRODUCT, VENDOR)	7,000 + 300	7,300	2,100,000	7,300
	A2	Select rows in A1 with matching vendor codes	2,100,000	2,100,000	7,000	2,107,300
	A3	Select rows in A2 with V_STATE = 'FL'	7,000	7,000	1,000	2,114,300
B	B1	Select rows in VENDOR with V_STATE = 'FL'	300	300	10	300
	B2	Cartesian Product (PRODUCT, B1)	7,000 + 10	7,010	70,000	7,310
	B3	Select rows in B2 with matching vendor codes	70,000	70,000	1,000	77,310

Database Systems, 13th Edition ³³

Using Hints to Affect

Optimizer Choices

- Optimizer might not choose best plan
- Makes decisions based on existing statistics – Statistics may be old

– Might choose less-efficient decisions

- Optimizer hints: special instructions for the optimizer embedded in the SQL command text

Database Systems, 13th Edition ³⁴

- Here are examples of optimizer hints in various contexts: • 1. Forcing a Full Table Scan (Oracle):

- This hint instructs the optimizer to perform a full scan of the employees table, even if an index might exist on department_id.

- `SELECT /*+ FULL(e) */ *`

FROM employees e

WHERE department_id = 10;

- 2. Forcing Index Usage (Oracle):

- This hint explicitly tells the optimizer to use the emp_dept_ix index on the employees table.

- `SELECT /*+ INDEX(employees emp_dept_ix) */ *`

FROM employees

WHERE department_id = 20;

- 3. Specifying Join Order (Oracle):

- The LEADING hint dictates the join order, ensuring that employees (aliased as e) is joined with departments (aliased as d) first.

- `SELECT /*+ LEADING(e d) */ e.name, d.location`

FROM employees e JOIN departments d ON e.department_id = d.id; Database Systems, 10th Edition 35

- Syntax:

- Hints are enclosed within /*+ and */ comments immediately following the SELECT, INSERT, UPDATE, or DELETE keyword.

- `SELECT /*+ ALL_ROWS */ employee_id, first_name FROM employees WHERE department_id = 10;`

- Categories of Optimizer Hints:

- **Optimization Approaches and Goals:**

- **ALL_ROWS**: Optimizes for best throughput (minimum total resource consumption).
- **FIRST_ROWS(n)**: Optimizes for best response time (minimum resource usage to return the first n rows).
- **RULE** (deprecated in newer versions): Forces rule-based optimization.
- **Access Paths**:
- **FULL(table_alias)**: Forces a full table scan.
- **INDEX(table_alias index_name)**: Forces the use of a specific index.
- **NO_INDEX(table_alias index_name)**: Prevents the use of a specific index.
- **Join Orders**:
- **ORDERED**: Forces the join order of tables as they appear in the FROM clause.
- **LEADING(table_alias)**: Specifies the driving table in a join.
- **Join Operations**:
- **USE_NL(table_alias)**: Forces a nested loop join.
- **USE_HASH(table_alias)**: Forces a hash join.
- **USE_MERGE(table_alias)**: Forces a sort-merge join. ³⁶

**TABLE
11.5**

Optimizer Hints

HINT	USAGE
ALL_ROWS	Instructs the optimizer to minimize the overall execution time—that is, to minimize the time needed to return all rows in the query result set. This hint is generally used for batch mode processes. For example: <pre>SELECT /*+ ALL_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
FIRST_ROWS	Instructs the optimizer to minimize the time needed to process the first set of rows—that is, to minimize the time needed to return only the first set of rows in the query result set. This hint is generally used for interactive mode processes. For example: <pre>SELECT /*+ FIRST_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
INDEX(name)	Forces the optimizer to use the P_QOH_NDX index to process this query. For example: <pre>SELECT /*+ INDEX(P_QOH_NDX) */ * FROM PRODUCT WHERE P_QOH < 10;</pre>

Database Systems, 13th Edition ³⁷

SQL Performance Tuning

- Evaluated from client perspective

- Most current relational DBMSs perform **automatic query optimization** at the server end
- Most **SQL performance optimization techniques are DBMS specific**
 - Does this mean that you should not worry about how a SQL query is written because the DBMS will always optimize it? No, because there is considerable room for improvement. (The DBMS uses *general* optimization techniques rather than focusing on specific techniques dictated by the special circumstances of the query execution.)
 - Majority of performance problems are related to poorly written SQL code
 - although a DBMS provides general optimizing services, a carefully written query almost always outperforms a poorly written one.

Database Systems, 13th Edition ³⁸

Index Selectivity

- Indexes are used when:
 - Indexed column appears by itself in search criteria of **WHERE** or **HAVING** clause
 - Indexed column appears by itself in **GROUP BY** or **ORDER BY** clause
 - **MAX** or **MIN** function is applied to indexed column
 - **Data sparsity** on indexed column is **high** • Measure of how likely an index will be used

Database Systems, 13th Edition ³⁹

Index Selectivity (cont'd.)

- General guidelines for indexes:
 - Create indexes for each attribute in **WHERE**, **HAVING**, **ORDER BY**, or **GROUP BY** clause
 - **Do not use in small tables or tables with low sparsity** – **Declare primary and foreign keys** so optimizer can use indexes in **join operations**
 - If you perform join operations on columns other than the primary and foreign keys, **Declare indexes in join columns.** – **How many indexes should you create?** You should not create an index for every column in a table. Too many indexes **will slow down INSERT, UPDATE, and DELETE operations, especially if the table contains many thousands of rows.** Furthermore, some **query optimizers will choose only one index to be the driving index for a query.**

Database Systems, 13th Edition ⁴⁰

Conditional Expressions

- Normally expressed within WHERE or HAVING clauses of SQL statement
- Restricts output of query to only rows matching conditional criteria

Database Systems, 13th Edition ⁴¹

Conditional Expressions (cont'd.)

- Common practices for efficient SQL:
 - Use simple columns or literals (constant) in conditionals. For example, $P_PRICE > 10.00$ is faster than $P_QOH > P_MIN * 1.10$
 - Numeric field comparisons are faster than character, date, and NULL comparisons. NULL conditions tend to be the slowest of all conditional operands.
 - Equality comparisons are faster than inequality ($>$, $>=$, $<$, $<=$).
 - Transform conditional expressions to use literals. For example, if your condition is $P_PRICE - 10 = 7$, change it to read $P_PRICE = 17$.

Also, if you have a composite condition such as:

$P_QOH < P_MIN$ AND $P_MIN = P_REORDER$ AND $P_QOH = 10$

change it to read:

$P_QOH = 10$ AND $P_MIN = P_REORDER$ AND $P_MIN > 10$

- Write equality conditions first
- *If you use multiple AND conditions, write the condition most likely to be false first. – When using multiple OR conditions, put the condition most likely to be true first. – Whenever possible, try to avoid the use of the NOT logical operator* For example:

$NOT (P_PRICE > 10.00)$ can be written as $P_PRICE <= 10.00$.

Also, $NOT (EMP_SEX = 'M')$ can be written as $EMP_SEX = 'F'$.

Database Systems, 13th Edition ⁴²

Query Formulation

1. Identify what columns and computations are required
2. Identify source tables

3. Determine how to join tables

4. Determine what selection criteria is needed:

- *a. Simple comparison.* In most cases, you will be comparing single values—for example, `P_PRICE > 10`.
- *b. Single value to multiple values.* If you are comparing a single value to multiple values, you might need to use an IN comparison operator—for example, `V_STATE IN ('FL', 'TN', 'GA')`.
- *c. Nested comparisons.* In other cases, you might need to have some nested selection criteria involving subqueries—for example, `P_PRICE >= (SELECT AVG (P_PRICE) FROM PRODUCT)`.
- *d. Grouped data selection.* On other occasions, the selection criteria might apply not to the raw data but to the aggregate data. In those cases, you need to use the HAVING clause.

5. Determine in what order to display output

- In those cases, you need to use the ORDER BY clause. Remember that the ORDER BY clause is one of the most resource-intensive operations for the DBMS.

Database Systems, 13th Edition ⁴³

DBMS Performance Tuning

- Includes managing DBMS processes in primary memory and structures in physical storage
- DBMS performance tuning at server end focuses on setting parameters used for the:
 - Data cache
 - SQL cache
 - Sort cache
 - Optimizer mode

Database Systems, 13th Edition ⁴⁴

DBMS Performance Tuning (cont'd.)

- Some general recommendations for creation of databases:
 - Use RAID (Redundant Array of Independent Disks) to provide balance between performance and fault tolerance
 - Minimize disk contention
 - Put high-usage tables in their own table spaces – Assign separate data files in separate storage volumes for indexes, system, and high-usage tables

Database Systems, 13th Edition ⁴⁵

DBMS Performance Tuning (cont'd.)

- Take advantage of table storage organizations in database
- Partition tables based on usage
- Use denormalized tables where appropriate – Store computed and aggregate attributes in tables

Database Systems, 13th Edition ⁴⁶

References

- Optimization Modes and Hints

Database Systems, 10th Edition 47.

.....

Database Systems:

Design, Implementation, and Management

13th Edition

Chapter 10 Transaction Management and Concurrency Control

What Is a Transaction?

- Logical **unit of work** that must be either entirely completed or aborted
- Successful transaction changes database from one **consistent state** to another
- One in which all ***data integrity constraints*** are satisfied
- Most real-world database transactions are formed by two or more database requests – Equivalent of a single SQL statement in an application program or transaction

- Will involve at least one read and write action₂

Database Systems, 13th Edition 3

What Is a Transaction?

To understand the concept of a transaction, suppose that you sell a product to a customer. Furthermore, suppose that the customer may charge the purchase to his or her account. Given that scenario, your sales transaction consists of at least the following parts:

- You must write a new customer invoice.
- You must reduce the quantity on hand in the product's inventory.
- You must update the account transactions.
- You must update the customer balance. 4

Evaluating Transaction Results

- Not all transactions update database.

```
SELECT CUST_NUMBER, CUST_BALANCE
```

```
FROM CUSTOMER
```

```
WHERE CUST_NUMBER = 10016;
```

- Although the query does not make any changes in the CUSTOMER table, the SQL code represents a transaction because it *accesses* the database

- Improper or incomplete transactions can have devastating effect on database integrity

– Some DBMSs provide means by which user can define enforceable constraints

– Other integrity rules are enforced automatically by the DBMS

Database Systems, 13th Edition ⁵

Evaluating Transaction Results

```
INSERT INTO INVOICE VALUES (...);
```

```
INSERT INTO LINE (...);
```

```
UPDATE PRODUCT SET ...
```

```
UPDATE CUSTOMER SET CUST_BALANCE..
```

```
INSERT INTO ACCT_TRANSACTION VALUES (...); COMMIT;
```

- Suppose the first 3 statements are executed and then power is lost with no backup –DBMS is in an inconsistent state unless transaction management is supported

- Invoice and Line rows were added and the QOH of Product table updated but the customer balance has not been increased nor has a new transaction been added to Acct_Transaction

Figure 9.2

Database Systems, 13th Edition ⁷

Transaction Properties (ACID)

- Atomicity

- All operations of a transaction must be completed or else the entire transaction is aborted

- Consistency

- Permanence of database's consistent state • Isolation

- Data used during transaction cannot be used by second transaction until the first is completed

Database Systems, 13th Edition ⁸

Transaction Properties (cont'd.)

- Durability

- Once transactions are committed, they cannot be undone

- Serializability

- Concurrent execution of several transactions yields consistent results

- Multiuser databases are subject to multiple concurrent transactions

Database Systems, 13th Edition ⁹

Transaction Management with SQL

- ANSI has defined standards that govern SQL database transactions

- Transaction support is provided by two SQL statements: COMMIT and ROLLBACK

- Transaction sequence must continue until:

- COMMIT statement is reached
- ROLLBACK statement is reached
- End of program is successfully reached
- Program is abnormally terminated. This action is equivalent to ROLLBACK.
- Some SQL implementations use BEGIN (or SET) TRANSACTION; to indicate the beginning of a new transaction

Transaction Management with SQL

- The use of COMMIT is illustrated in the following simplified sales example, which updates a product's quantity on hand (PROD_QOH) and the customer's balance when the customer buys two units of product 1558-QW1 priced at \$43.99 per unit (for a total of \$87.98) and charges the purchase to the customer's account:

UPDATE PRODUCT

SET PROD_QOH = PROD_QOH – 2

WHERE PROD_CODE = '1558-QW1';

UPDATE CUSTOMER

SET CUST_BALANCE = CUST_BALANCE + 87.98

WHERE CUST_NUMBER = '10011';

COMMIT;

The Transaction Log

- A DBMS uses a transaction log to keep track of all transactions that update the database.
- The DBMS uses the information stored in this log for a recovery requirement triggered by a ROLLBACK statement, a program's abnormal termination, or a system failure such as a network discrepancy or a disk crash.
- Some RDBMSs use the transaction log to recover a database *forward* to a currently consistent state. After a server failure, for example, Oracle automatically rolls back uncommitted transactions and rolls forward transactions that were committed but not yet written to the physical database.

- This behavior is required for transactional correctness and is typical of any transactional DBMS

Database Systems, 13th Edition ¹²

The Transaction Log

- While the DBMS executes transactions that modify the database, it also automatically updates the transaction log
- Although using a transaction log increases the processing overhead of a DBMS, the ability to restore a corrupted database is worth the price.

Database Systems, 13th Edition ¹³

The Transaction Log

- Transaction log stores:
 - A record for the beginning of transaction – For each transaction component:
 - Type of operation being performed (update, delete, insert)
 - Names of objects affected by transaction
 - “Before” and “after” values for updated fields • Pointers to previous and next transaction log entries for the same transaction
 - Ending (COMMIT) of the transaction

Database Systems, 13th Edition ¹⁴

The Transaction Log

- Next slide illustrates a simplified transaction log that reflects a transaction composed of two SQL UPDATE statements
- If a system failure occurs, the DBMS examines the log for all uncommitted or incomplete transactions and restores the database to its previous state
 - When the recovery process is completed, the DBMS will write in the log all committed transactions that were not physically written to the database before the failure occurred
 - Committed transactions are not rolled back

Concurrency Control

- Coordination of simultaneous transaction execution in a multiprocessing database
- Objective is to ensure serializability of transactions in a multiuser environment
- The simultaneous execution of transactions over a shared database can create several data integrity and consistency problems

- Three main problems:

- Lost updates

- Uncommitted data

- Inconsistent retrievals

Lost Updates

- Lost update problem:

- Two concurrent transactions update same data element

TABLE 10.2 Two Concurrent Transactions to Update QOH	
TRANSACTION	COMPUTATION
T1: Purchase 100 units	$PROD_QOH = PROD_QOH + 100$
T2: Sell 30 units	$PROD_QOH = PROD_QOH - 30$

- Table 10.3 shows the serial execution of the transactions under normal circumstances.

- Final result of $PROD_QOH = 105$ is correct

Lost Updates

- Table 10.4 shows what happens when a second transaction can read the $PROD_QOH$ value of a product before a previous

transaction has been committed for that same product

- One of the updates is lost by being overwritten by the other transaction

Uncommitted Data

- Uncommitted data phenomenon:
 - Two transactions are executed concurrently – First transaction rolled back after second already accessed uncommitted data
- Violates the isolation property

TABLE 10.5 Transactions Creating an Uncommitted Data Problem

TRANSACTION	COMPUTATION
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100 (Rolled back)
T2: Sell 30 units	PROD_QOH = PROD_QOH - 30

Database Systems, 13th Edition ²⁰

Uncommitted Data

- Serial execution of the transactions yields the correct answer under normal circumstances

Uncommitted Data

- Uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution

Inconsistent Retrievals

- Inconsistent retrievals:
 - First transaction accesses data
 - Second transaction alters the data
 - First transaction accesses the data again • Transaction might read some data before they are changed and other data after changed • Yields inconsistent results

Inconsistent Retrievals

- A sample of the PRODUCT table, showing the records being updated as specified in the UPDATE code

Inconsistent Retrievals

- Inconsistent retrievals are possible during the transaction execution

- The “after” summation shown below reflects that the value of 25 for product 1546-QQ2 was read after the WRITE statement was completed so the total up to that point is 65
- The “before” total reflects that the value of 23 for 1558-QQ1 was read before the next WRITE was completed (in which case the value would have been 13 instead of 23)
- After step 7, the total is 65+23 rather than 65+13 • The final total will be 102 instead of 92

Inconsistent Retrievals

The Scheduler

- The scheduler is a special DBMS process that establishes the order in which the operations are executed within concurrent transactions. The scheduler interleaves the execution of database operations to ensure serializability and isolation of transactions.
- To determine the appropriate order, the scheduler bases its actions on concurrency control algorithms, such as locking or time stamping methods, which are explained in the next sections.
- However, it is important to understand that not all transactions are serializable. • The DBMS determines what transactions are serializable and proceeds to interleave the execution of the transaction's operations. Generally, transactions that are not serializable are executed on a first-come, first-served basis by the DBMS.
- The scheduler's main job is to create a serializable schedule of a transaction's operations, in which the interleaved execution of the transactions (T1, T2, T3, etc.) yields the same results as if the transactions were executed in serial order (one after another).

The Scheduler

- FCFS is an inefficient method of allocating the CPU, since the CPU waits idly as a READ or WRITE operation completes
- The scheduler also facilitates data isolation to ensure that two transactions do not update the same data element at the same time
- If one or both of two concurrent transactions wants to perform a WRITE on the same data, then a conflict will result

Concurrency Control with Locking Methods • Lock

- Guarantees exclusive use of a data item to a current transaction
- Lock is acquired prior to access and released when the transaction is completed
- Required to prevent another transaction from reading inconsistent data
- Pessimistic locking
- Use of locks based on the assumption that conflict between transactions is likely
- Lock manager
- Responsible for assigning and policing the locks used by transactions

Lock Granularity

- Indicates level of lock use
- Locking can take place at following levels: – Database
- Table
- Page
- Row
- Field (attribute)

Database Systems, 13th Edition ³⁰

Lock Granularity

- Database-level lock
- Entire database is locked, prevents the use of any tables by transaction T2 while T1 is being executed
- Good for batch systems, not for multi-user systems

Lock Granularity

- Table-level lock
- Entire table is locked
- If a transaction requires access to several tables, each table may be locked
- Two transactions can access the database as long as they access different tables

– Not suitable for multi-user systems because all rows are locked even if different transactions access different rows

Database Systems, 13th Edition ³³

Database Systems, 13th Edition ³⁴

Lock Granularity

- Page-level lock

– Entire diskpage (or page) is locked

- Page is the equivalent of a diskblock - a directly addressable section of disk (e.g., 4K, 8K or 16K) • If you want to write 73 bytes to a 4K page, the entire page must be read from disk, updated in memory and written back to disk

– A table can span multiple pages and a page can contain several rows of one or more tables

- As long as two transactions do not access the same page, no waiting is required by the second transaction • currently the most frequently used locking method for multi-user DBMSs

Database Systems, 13th Edition ³⁶

Lock Granularity

- Row-level lock

– Allows concurrent transactions to access different rows of same table

- Even if rows are located on same page
- High overhead because a lock exists for each row of a table
- If the application requests multiple locks on the same page, a row level lock is automatically escalated to a page level lock.

Database Systems, 13th Edition ³⁷

Database Systems, 13th Edition ³⁸

Lock Granularity

- Field-level lock
 - Allows concurrent transactions to access same row
- Requires use of different fields (attributes) within the row
- Most flexible type of lock but requires an extremely high level of overhead – rarely implemented

Database Systems, 13th Edition ³⁹

Lock Types

- Binary lock
 - Two states: locked (1) or unlocked (0)
 - Every DB operation requires that the affected object be locked
 - Transaction must unlock the object after its termination
 - Locks and unlocks are performed automatically by the DBMS, not the user

Database Systems, 13th Edition ⁴⁰

Lock Types

- Applying locks to the “Lost Update Problem”
 - Lock/unlock eliminates problem because the lock is not released until the WRITE is completed
 - PROD_QOH can not be used until it has been properly updated – Binary locks are considered too restrictive as the DBMS will not allow two transactions to read the same object even though neither updates the database and no concurrency problems can occur

Lock Types

- Exclusive lock
 - Access is specifically reserved for transaction that locked object
 - Must be used when potential for conflict exists • i.e., when two operations are in conflict when they access the same data and at least one of them is performing a **WRITE**

- Shared lock
 - Concurrent transactions are granted read access on basis of a common lock
- Produces no conflict as long as all the concurrent transactions are read-only

Lock Types

- A shared lock is issued when a transaction wants to read data from the DB and no exclusive lock is held on that data item
- An exclusive lock is issued when a transaction wants to update (write) a data item and no locks are currently held on that data item by any other transaction
- Thus, under these concepts, a lock can have three states
 - Unlocked
 - Shared (read)
 - Exclusive (write) ⁴³

Lock Types

- If T1 has a shared lock on data item X, T2 may be issued a shared lock to read X
- If T2 wants to update X, an exclusive lock is required by T2
 - The exclusive lock is granted if and only if no other locks are held on the data item
 - If T1 has a shared lock on X. T2 can not get an exclusive lock on X – **mutual exclusive rule**
 - T2 must wait until T1 commits

Lock Types

- A shared/exclusive lock schema increases the lock manager's overhead
 - The type of lock held must be known before a lock can be granted
 - Three lock operations exist:
 - READ_LOCK to check the type of lock
 - WRITE_LOCK to issue the lock
 - UNLOCK to release the lock
 - The schema has been enhanced to allow a lock upgrade from shared to exclusive and a lock downgrade from exclusive to shared

Lock Types

- Although locks prevent serious data inconsistencies, they can lead to major problems – The resulting transaction schedule might not be serializable
 - The schedule might create deadlocks
 - A deadlock occurs when two transactions wait indefinitely for each other to unlock data
 - A database deadlock (similar to traffic gridlock) is caused when two or more transactions wait for each to unlock data
- ### Two-Phase Locking to Ensure Serializability
- Defines how transactions acquire and relinquish locks
 - Guarantees serializability, but does not prevent deadlocks
- Growing phase
 - Transaction acquires all required locks without unlocking any data
 - Shrinking phase
 - Transaction releases all locks and cannot obtain any new lock

Database Systems, 13th Edition ⁴⁷

Two-Phase Locking

to Ensure Serializability

- Governed by the following rules:
 - Two transactions cannot have conflicting locks – No unlock operation can precede a lock operation in the same transaction
 - No data are affected until all locks are obtained, meaning until the transaction is in its locked point

Database Systems, 13th Edition ⁴⁸

Two-Phase Locking to Ensure Serializability

- The transaction first acquires the two locks it needs
- When it has two locks, it reaches its locked point
- Next, the data are modified to conform to the transaction requirements
- Finally, the transaction is completed and it releases all of the locks it acquired in the first phase

- Two phase locking increases transaction processing and cost and might cause deadlocks

Deadlocks

- Condition that occurs when two transactions wait for each other to unlock data
 - T1 = access data items X and Y
 - T2 = access data items Y and X
- If T1 has not unlocked data item Y, T2 cannot begin • If T2 has not unlocked data item X, T1 cannot continue • T1 and T2 each wait for the other to unlock the required data item – **deadly embrace**
- Possible only if one of the transactions wants to obtain an exclusive lock on a data item
 - No deadlock condition can exist among shared locks 51

Deadlocks

Controlling Deadlock

- Prevention
 - A transaction requesting a new lock is aborted when there is the possibility that a deadlock can occur. – If the transaction is aborted, all changes made by this transaction are rolled back and all locks obtained by the transaction are released
 - The transaction is then rescheduled for execution – It avoids the condition that leads to deadlocking Controlling Deadlock
- Detection
 - The DBMS periodically tests the database for deadlocks
 - If found, the “victim” transaction is aborted (rolled back and restarted) and the other transaction continues • Avoidance
 - The transaction must obtain all of the locks it needs before it can be executed
 - This avoids the rolling back of conflicting transactions by requiring that locks be obtained in succession – Serial lock assignment increases action response times Deadlocks
- Choice of deadlock control method depends on database environment
 - Low probability of deadlock; detection recommended

– High probability; prevention recommended – If response time is not high on the system's priority list, deadlock avoidance might be employed • All current DBMSs support deadlock detection while some use a blend of prevention and avoidance techniques for other types of data, such as data warehouses or XML data

Concurrency Control with Time Stamping Methods

- Assigns global unique time stamp to each transaction
- Produces explicit order in which transactions are submitted to DBMS
- Properties of timestamps

– Uniqueness

- Ensures that no equal time stamp values can exist – Monotonicity
 - Ensures that time stamp values always increase
- #### Concurrency Control with Time Stamping Methods

- All database operations within the same transaction must have the same timestamp • The DBMS executes conflicting operations in timestamp order, thereby ensuring serializability of the transactions

- If two transactions conflict, one is stopped, rolled back, rescheduled and assigned a new timestamp value
- #### Concurrency Control with Time Stamping Methods

- Disadvantage - each value stored in the database requires two additional timestamp fields; one for the last time the field was read and one for the last update

– This increases memory needs and the database's processing overhead

- Timestamping demands a lot of system resources because many transactions might have to be stopped, rescheduled and restamped
- #### Wait/Die and Wound/Wait Schemes

- How to decide which transaction to rollback and which continues executing

– **Wait/die**

- Older transaction requests lock - it waits until younger is completes and locks released
- Younger transaction requests lock - it dies (rolls back) and is rescheduled using the same timestamp

– **Wound/wait**

- Older transaction requests lock, it preempts (wounds) the younger transaction by rolling it back and rescheduling it using the same timestamp
- Younger transaction requests lock, it will wait until the other transaction is completed and the locks released 59

Wait/Die and Wound/Wait Schemes

- Wait for lock request can cause some transactions to wait indefinitely, causing a deadlock
- To prevent a deadlock, each lock request has an associated time-out value
- If the lock is not granted before the time-out expires, the transaction is rolled back

Concurrency Control with Optimistic Methods

- Optimistic approach
- Based on assumption that majority of database operations do not conflict
- Does not require locking or time stamping techniques
- Transaction is executed without restrictions until it is committed
- Each transaction moves through two or three phases - read, validation, and write

Database Systems, 13th Edition ⁶¹

Concurrency Control with Optimistic Methods

- Read Phase
- Transaction reads the database, executes computations, makes updates to a private copy of the database values (temporary update file) not accessed by the remaining transactions
- Validation Phase
- Transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database • If validation test is positive, the transaction goes to the write phase
- If validation test is negative, transaction is restarted and the changes discarded
- Write Phase
- The changes are permanently applied to the database 62

Concurrency Control with Optimistic Methods

- This approach is acceptable for most read or query database systems that require few update transactions
- In a heavily used DBMS environment, one or more of the above techniques will be used • However, it may be necessary at times to employ database recovery techniques to restore the database to a consistent state Database Recovery Management
- Restores database to previous consistent state • Based on atomic transaction property – All portions of transaction are treated as single logical unit of work
- All operations are applied and completed to produce consistent database
- If transaction operation cannot be completed: – Transaction aborted
- Changes to database are rolled back

Database Systems, 13th Edition ⁶⁴

Database Recovery Management

- In addition to recovering transactions, recovery techniques also apply to the database and system after some type of critical error – Hardware/software failures
- Human caused incidents
- Natural disasters

Database Systems, 13th Edition ⁶⁵

Transaction Recovery

- Use data in the transaction log to recover a database from an inconsistent state to a consistent state
- Four concepts that affect the recovery process – **Write-ahead-log protocol**: ensures transaction logs are written before database data is updated • In case of a failure, the DB can be recovered using data in the transaction log
- **Redundant transaction logs**: ensure physical disk failure will not impair ability to recover

Transaction Recovery

- **Buffers**: temporary storage areas in primary memory
- To improve processing time, the DBMS software reads the data from the physical disk and stores a copy of it on a “buffer” in primary memory
- When a transaction updates data, it actually updates the copy of the data in the buffer

because that's faster than updating the physical disk

- Later, all buffers that contain data are written to a physical disk during a single operation thereby saving significant processing time

Transaction Recovery

– **Checkpoints:** operations in which DBMS writes all its updated buffers to disk

- While this is happening, the DBMS does not execute any other requests
- Scheduled by the DBMS several times per hour
- A checkpoint operation is also registered in the transaction log
- The physical database and the transaction log are in synch

– Synching is required because update operations update the copy of the data in the buffers and not in the physical database

Transaction Recovery

- **Deferred-write (deferred update) technique** – Only transaction log is updated

– DB is physically updated only after the transaction reaches its commit point

– If the transaction aborts before a commit, no rollback is needed

Transaction Recovery

- Recovery process for all started and committed transactions (before the failure) follows these steps

– identify last checkpoint (i.e., when transaction data was physically saved to disk)

– If transaction committed before checkpoint:

- Do nothing, the data was already saved

– If transaction committed after checkpoint:

- Use transaction log to redo the transaction using the “after” values

– Changes are made in oldest to newest order

– If transaction had ROLLBACK operation after the last checkpoint or that was left active (with neither a COMMIT nor a ROLLBACK) before the failure:

- Do nothing, DB was never updated

Transaction Recovery

- **Write-through technique**

- Database is immediately updated by transaction operations during transaction's execution even before the transaction reaches its commit point
- Recovery process: identify last checkpoint (when data were physically written to the disk)
- If transaction committed before checkpoint:
 - Do nothing; data already saved
- If transaction committed after last checkpoint:
 - DBMS redoes the transaction using "after" values in oldest to newest order
- If transaction had ROLLBACK or was left active (with neither a COMMIT nor a ROLLBACK) before failure occurred:
 - DBMS uses the transaction log to ROLLBACK using the "before" values in newest to oldest order 71

Transaction Recovery

Database Systems, 13th Edition 72

Transaction Recovery

- Transaction 101 consists of two UPDATE statements that reduce the QOH for 54778-2T and increase the customer balance for customer 10011 for a credit sale of 2 units of product 54778-27
- Transaction 106 represents the credit sale of 1 unit of 89-WRE-Q to customer 10016 for \$277.55. This transaction consists of 5 DML statement – 3 INSERTs and 2 UPDATEs
- Transaction 155 consists of 1 UPDATE that increases the QOH of 2232/QWE from 6 to 26
- A database checkpoint writes all updated database buffers for previously uncommitted transaction to disk. In this case, all changes made by 101. Transaction Recovery
- Using the deferred update method, the following will happen during a recovery
 1. Identify the last checkpoint – TRL ID 423
 2. Transaction 101 started and ended before the last checkpoint so no action needs to be taken
 3. For each transaction committed after TRL ID 423, the DBMS uses the transaction log to write changes to disk using the "after" values. For transaction 106
 - a. Find COMMIT (TRL ID 457)

- b. Use the previous pointer values to locate the start of the transaction (TRL ID 397)
 - c. Use the next pointer values to locate each DML and apply the “after” values (TRL ID 405, 415, 419, 427 and 431)
 - d. Repeat the process for transaction 155
4. Any other transaction will be ignored as no changes were written to disk⁷⁴
-

NOSQL DATABASES

Advantages of RDBMSs

- Strong theoretical basis
- Store information efficiently
- They guard against data duplication and redundancy
- Transactions in relational databases are well understood and offer strong guarantees of data consistency.
- The relational database engines themselves are mature, robust, efficient and fully featured pieces of software.

2.1 Assumptions behind relational databases and RDBMSs

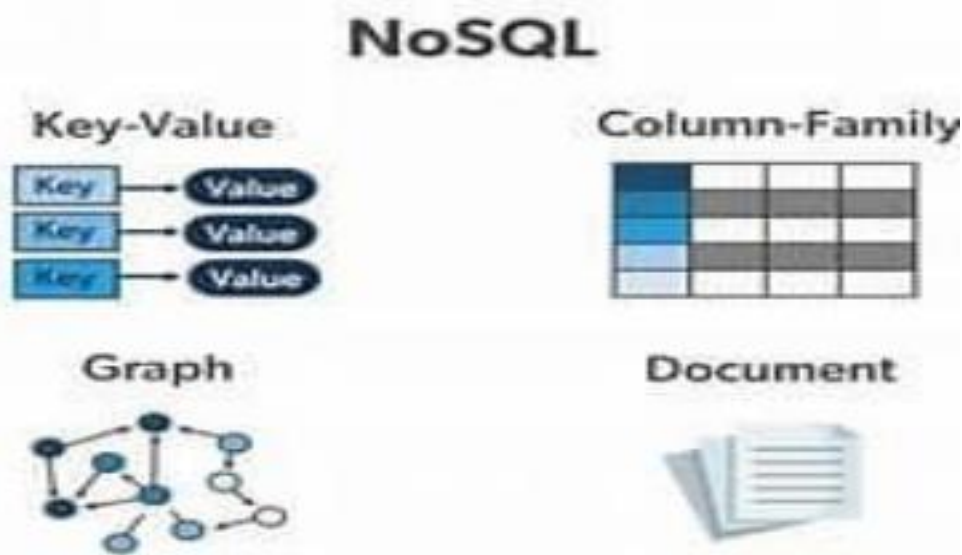
- data can be broken down into sets of records
- every record contains the same types of data (fields) • each field of each record holds atomic data
- records make sense on their own
- the data is more important than its relationships • data is regularly written to and read from the database • the database must always be internally consistent • the database is the ‘single point of truth’ and the results of interacting with the database must reflect that • the DBMS can see all of the data at once.

2.1 Assumptions behind relational databases and RDBMSs

- Some assumptions can be relaxed in practice: • atomic data
- each record fits the same schema
- But relational DBMSs work best if they fit these assumptions.
- these assumptions may no longer hold
- alternatives to the RDBMS may be more appropriate.

3 Alternatives to the relational model: NoSQL databases

- What is 'NoSQL'?
- an umbrella term for all non-relational DBMSs. • many alternatives to the relational way of organizing persistent data.



NoSQL databases

From www.nosql-database.org:

Next Generation Databases mostly addressing some of the points: being **non-relational**, **distributed**, **open-source** and **horizontal scalable**. The original intention has been **modern web-scale databases**. The movement began early 2009 and is growing rapidly. Often more characteristics apply as: **schema-free**, **easy replication support**, **simple API**, **eventually consistent** / **BASE** (not ACID), a **huge data amount**, and more.

- Many NoSQL databases are aggregate-oriented. • Relational databases are fragmented, normalised.
- NoSQL databases are Schema Free.
- Relational databases are schema based.

4 Examples of NoSQL databases

	Genre	Version	Datatypes	Data Relations
MongoDB	Document	2.0	Typed	None
CouchDB	Document	1.1	Typed	None
Riak	Key-value	1.0	Blob	Ad hoc (Links)
Redis	Key-value	2.4	Semi-typed	None
PostgreSQL	Relational	9.1	Predefined and typed	Predefined
Neo4j	Graph	1.7	Untyped	Ad hoc (Edges)
HBase	Columnar	0.90.3	Predefined and typed	None

5 Databases in the real world: pragmatics and performance

- Will consider real world demands of databases & their implications for database management.

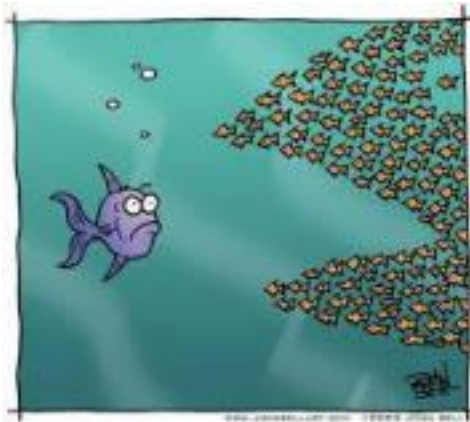
- The two main issues:
- **high volumes of data**
- **keeping data available and secure**

5 Databases in the real world: pragmatics and performance

Increasing capacity for relational DBMSs: • scaling up

- increasing the capacity of that computer (one server), and then distributing the database management features across several computers.
- Expensive

- Fragile: if a server goes down, **all** data is lost.
- Scaling out
- many small servers distributing the data load between them with redundancy.
- cheaper
- more robust



- Splitting one dataset across several servers is called sharding

Adv. & disadv. Of

scaling out

Advantages:

- More processing and storage capacity may be achieved for the same cost by buying many small 'commodity' computers and disks, rather than a few large ones.
- It also offers the advantages of concurrency, as jobs can be more easily distributed across the server farm.
- Data will be resilient (flexible); replicated copies of data will exist on several servers, meaning that the loss of one machine should not bring the entire operation to a grinding halt.

Disadvantages:

- More servers means more failures
- **clusters** should be designed with the presumption that something, somewhere, will fail fairly often.

Implications of scaling out

- Fundamental implications for how DBMSs operate, including what is meant by a 'transaction'.
- Pragmatic response to a network partition: allow the subsets to remain available but unsynchronised, reconcile the differences later.
- Atomicity, Consistency, Isolation, Durability

(ACID) – part 12

changes to ☐

- Basically Available, Soft state, Eventual consistency (**BASE**) – part 17.

Document databases: Mongo DB

Comparing Rel. & Document DBs

Relational DBs:

1. A relational database comprises a set of **tables**.
2. Each table consists of a set of **rows**
3. Each row has the same set of **fields**
4. Within a table, field names must be unique
5. Field values must be atomic, although this may be NULL. Documents DBs

1. A document database consists of a set of **collections**.
2. Each collection contains a set of **documents**
3. Each document contains a set of **key–value pairs** (also referred to as **fields**)
4. Within a document, keys must be unique.
5. Key-values may be non-atomic and may contain empty (null) values.

Relational vs. Document DB Terminologies

Relational DB	Document DB
Database	Database

Table / Relation	Collection
Record / Row / Tuple	Document
Field / Column name / Attribute	Key
Field value / attribute value	Key value

_id	name	address
1	John	Highway37
2	Peter	Lowstreet 27
3	Amy	NULL
4	Hannah	Mountain 21
5	Michael	NULL

```
{'_id': 1, 'name': 'John', 'address': 'Highway37'} {'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}
{'_id': 3, 'name': 'Amy'}
```

```
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'} {'_id': 5, 'name': 'Michael', 'address': null }
```

Relational Table MongoDB Collection

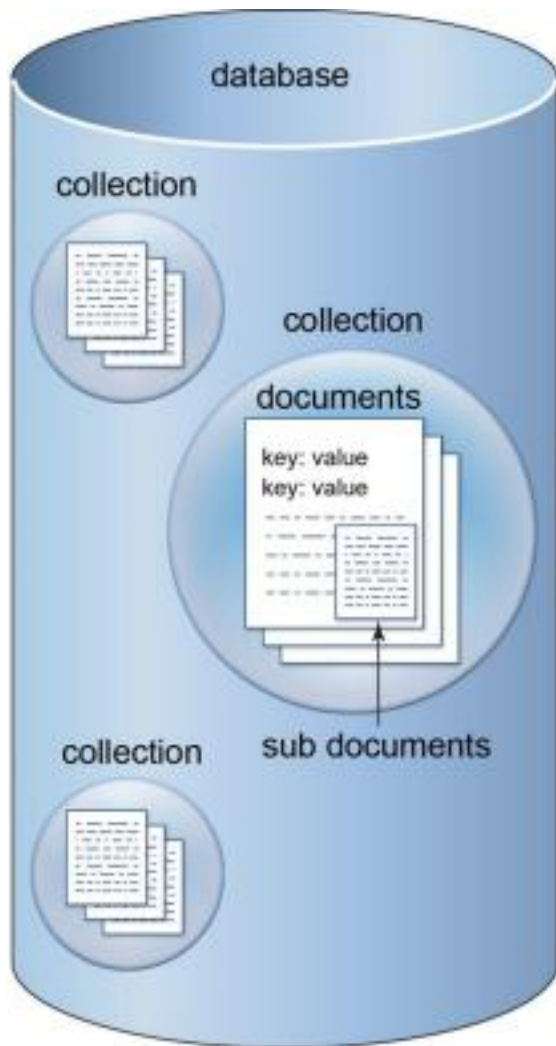


Figure 14.1 Document database concepts

Key difference

- The key departure from the relational model is what is inside the documents.
- Document databases are **schema-less**: each document in a collection can have a different set of keys.
- The values in the document need not be atomic: they can have complex structure, when the values are referred to sub-documents.
- In MongoDB, all **keys** must be valid JavaScript strings.

<u>Patient id</u>	Patient_name	doctor_id	doctor_name	<u>date</u>	<u>drug_code</u>	drug_name	dosage	duration
p001	Thronton	d06	Gibson	15-May-2014	T02378	Tramadol	50 mg 3 x day	As required
p001	Thronton	d06	Gibson	15-May-2014	O17663	Omeprazole	40 mg 1 x day	NULL
p001	Thronton	d06	Gibson	23-May-2014	S33558	Simvastatin	40 mg 1 x day	NULL
p001	Thronton	d06	Gibson	15-Jun-2014	A12458	Amitriptyline	10 mg 5 x day	As required

{'patient_id': 'p001',

'patient_name': 'Thornton', 'doctor_id': 'd06',

'doctor_name': 'Gibson', 'prescribed_drugs': [

List of

subdocuments

MongoDB

representation (as 1 document)

```
{'prescription_date': datetime.datetime(2014, 5, 15, 0, 0),
```

```
  'drug': {'code': 'T02378', 'drug_name': 'Tramadol'}, 'dosage': '50mg 3 x day',
```

```
  'duration': 'As required'},
```

sub-sub documents

```
{'prescription_date': datetime.datetime(2014, 5, 15, 0, 0), 'drug': {'code': 'O17663',  
'drug_name': 'Omeprazole'}, 'dosage': '40mg 1 x day'},
```

```
{'prescription_date': datetime.datetime(2014, 5, 23, 0, 0), 'drug': {'code': 'S33558', 'drug_name':  
'Simvastatin'}, 'dosage': '40mg 1 x day', 'duration': null },
```

```
{'prescription_date': datetime.datetime(2014, 6, 15, 0, 0), 'drug': {'code': 'A12458',  
'drug_name': 'Amitriptyline'}, 'dosage': '10mg 5 x day',
```

```
'duration': '30 days'}
```

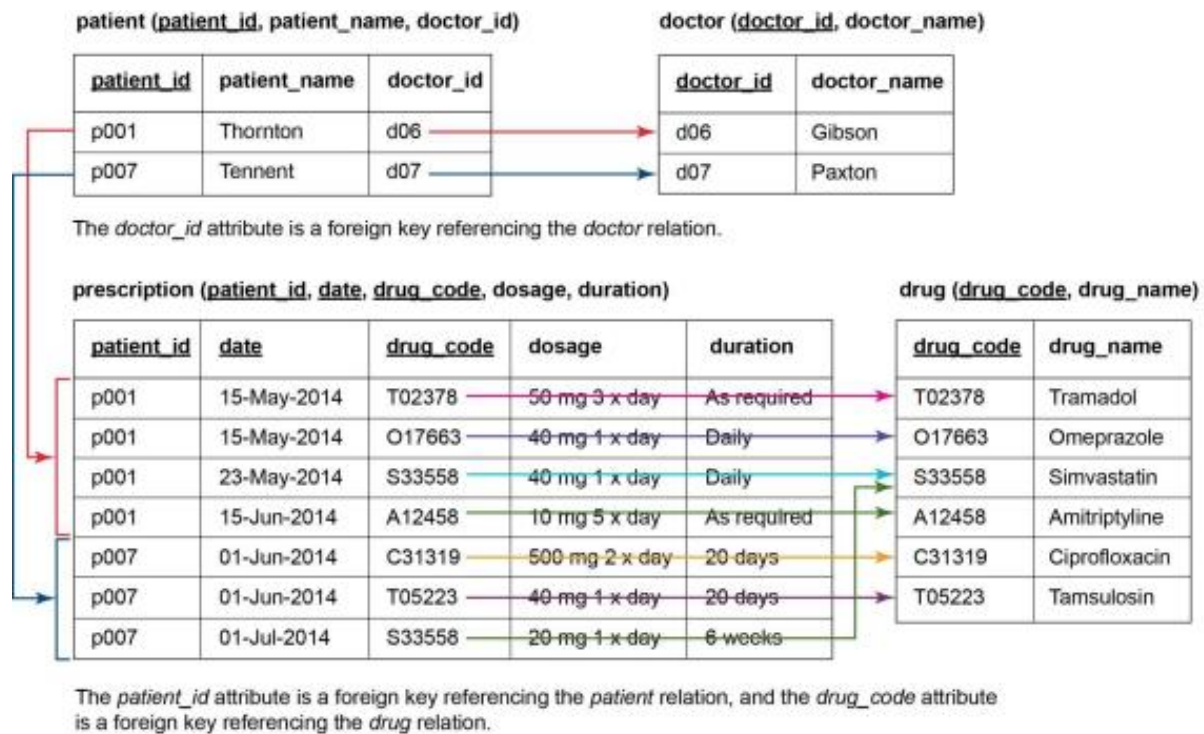
```
]
```

```
}
```

This document is shown in a format similar to how it would appear in the MongoDB console (as JSON), and in the PyMongo interface from Python to

MongoDB.

Normalised



Un normalized PyMongo

- PyMongo converts many MongoDB data structures between the MongoDB structures and native Python types.
- You may therefore find it helpful to understand MongoDB initially in those terms.
- PyMongo represents a document as a dict with several keys: *patient_id*, *patient_name*, *doctor_id*, and so on.
- Many keys have simple, atomic values – strings in this case.
- The *prescribed_drugs* key has a more complex value: it consists of a list of several sub-documents (themselves dicts).
- Each sub-document describes one prescription, which itself contains a sub-document describing the drug.

Example: from Exercise 14.6

SQL	PyMongo
<pre>select patient_name, date_of_birth from patient where gender = 'M' and weight > 75</pre>	<pre>patients.find({'gender':'M','weight':{'\$gt':75}}, ['patient_name', 'date_of_birth'])</pre>

find is a method that acts on a collection (patient, in this case). Its first argument is a query document that specifies which documents in the collection will be returned. For a document to be returned, it must contain the same keys as the query document (specified in the find) and those keys must have the specified values. Various operators, such as '\$gt' ('greater than'), for conditions other than equality are provided (see [*MonqoDB: The Definitive Guide*](#) for descriptions of these operators).

In this case, the query document in the find statement specifies that the returned documents must have gender and weight keys, the gender key must have a value of 'M', and the weight key must have a value greater than 75.

The second, optional, argument is a list of the keys (and the associated values) to be included in the returned documents. If this argument is missing, all keys of every matching document will be returned. This is the same functionality as SQL's select * from **PyMongo's find method**

collection query document > operator projection

```
patients.find(
{'gender':'M','weight':{'$gt':75}},
['patient_name',
```

```
'date_of_birth']
```

```
)
```

Note: The projection is options. If this argument is missing, all keys of every matching document will be returned. This is the same functionality as SQL's **select * from**

PyMongo's find method

```
patients.find(
```

```
{'gender':'M','weight':{
```

```
'$gt':75}},
```

```
['patient_name',
```

```
'date_of_birth']
```

```
)
```

collection

query

document

>

operator

projection

Note: The projection is options. If this argument is missing, all keys of every matching document will be returned. This is the same functionality as SQL's **select * from**

Sections:

Database 2

ORACLE ENVIRONMENT

LAB 1

Outline

Lab 1 Oracle Administration
1 Types of users in Oracle
2 Types of Oracle Clients
3 Connecting to SQLPLUS
4 Creating user and define user privileges
Lab 2 Privileges
1 Grant System Privilege / Object Privileges
2 Revoke System Privilege / Object Privileges
3 Lock and Expire Accounts

4 Oracle Users and User Table

Outline

Lab 3 SQL Developer
1 Create user using/ Create Table
2 Show Connected user and Disconnect
3 Connect Disconnected User
4 Roles and Granting

Types of Users in Oracle

Two user accounts are automatically created with the database and granted the DBA role. They are both created with the password that you supplied upon installation,

These two user accounts are:

- **SYS**
- **SYSTEM**

Admin user 'Sys'

- This account can perform all administrative functions. ▪ All base (underlying) tables and views for the database data dictionary are stored in the SYS schema.
- These base tables and views are critical for the operation of Oracle Database.
- To maintain the integrity of the data dictionary, tables in the SYS schema are manipulated only by the database.

- They should never be modified by any user or database administrator.
- You must not create any tables in the SYS schema.

→ **The SYS user is granted the SYSDBA privilege, which enables a user to perform high-level administrative tasks such as backup and recovery.**

Admin user

Administration tasks as

Shutdown Server.

Backup and Recovery.

Create any User.

Drop any table.

Disconnect any user.

Prevent any user from connecting. Export and import Data.

Regular User 'SYSTEM' Pre Installed User

This account can perform all administrative functions except the following:

- Backup and recovery
- Database upgrade

While this account can be used to perform day-to-day administrative tasks, Oracle strongly recommends creating named users account for administering the Oracle database to enable monitoring of database activity.

Three Clients of ORACLE

1. **SQL PLUS** : It's the Primary Client of ORACLE.

- **CMD** Command Line Based.

2. **ISQLPLUS**: It's an extension to SQLPLUS.

- **GUI** --> Graphical User Interface, It's a browser Based.

3. **ORACLE SQL Developer**

- Developed by Oracle.
- GUI Based

- Connect and get data to any other DBMS as from MYSQL server, SQL Server, SYBASE Server, Access, DB2.

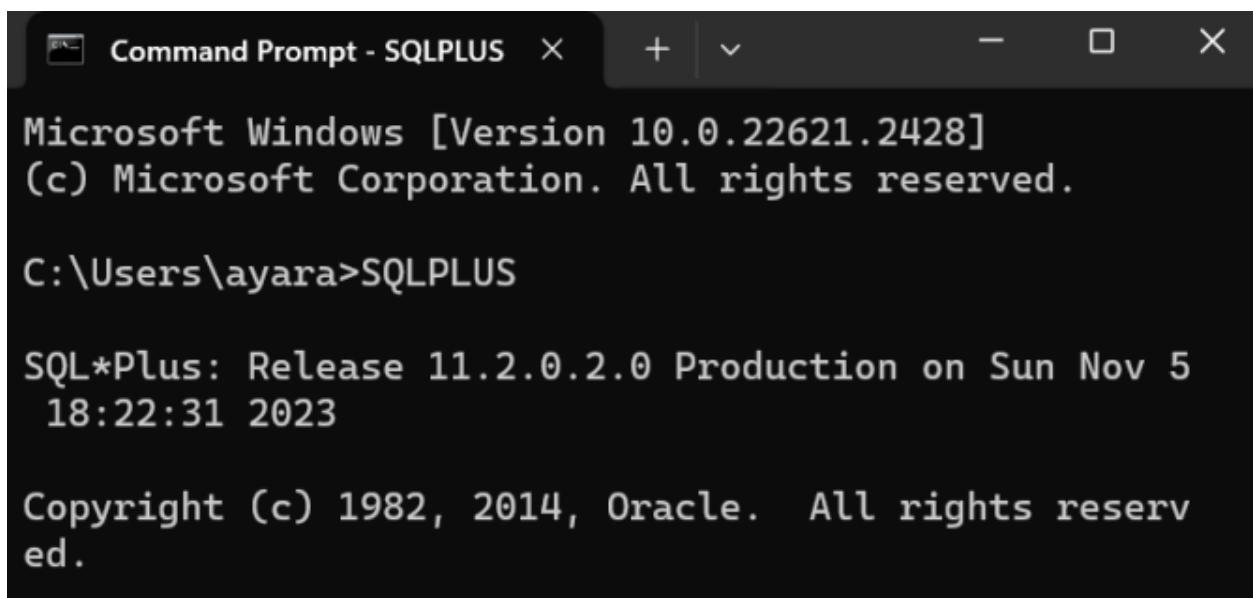
Connecting to SQLPLUS 1. CMD

2. SQLPLUS

3. RUN SQL COMMAND LINE

CMD

Open CMD



```
Microsoft Windows [Version 10.0.22621.2428]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ayara>SQLPLUS

SQL*Plus: Release 11.2.0.2.0 Production on Sun Nov 5
18:22:31 2023

Copyright (c) 1982, 2014, Oracle. All rights reserv
ed.
```

SQLPLUS

Open SQLPLUS.exe

```
C:\oracle\app\oracle\produ X + v
SQL*Plus: Release 11.2.0.2.0 Production on Sun Nov 5 18:30:44 2023
Copyright (c) 1982, 2014, Oracle. All rights reserved.
Enter user-name: SYS AS SYSDBA
Enter password:
Connected to:
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production
```

SQL COMMAND

```
SQL> CONN / AS SYSDBA
Connected.
SQL> CONN SYS AS SYSDBA
Enter password:
Connected.
SQL> CONN ALI / 123
Connected.
SQL> CONNECT ALI / 123
Connected.
```

LINE

Open Run

SQL Command Line ➤ **ADMIN USER**

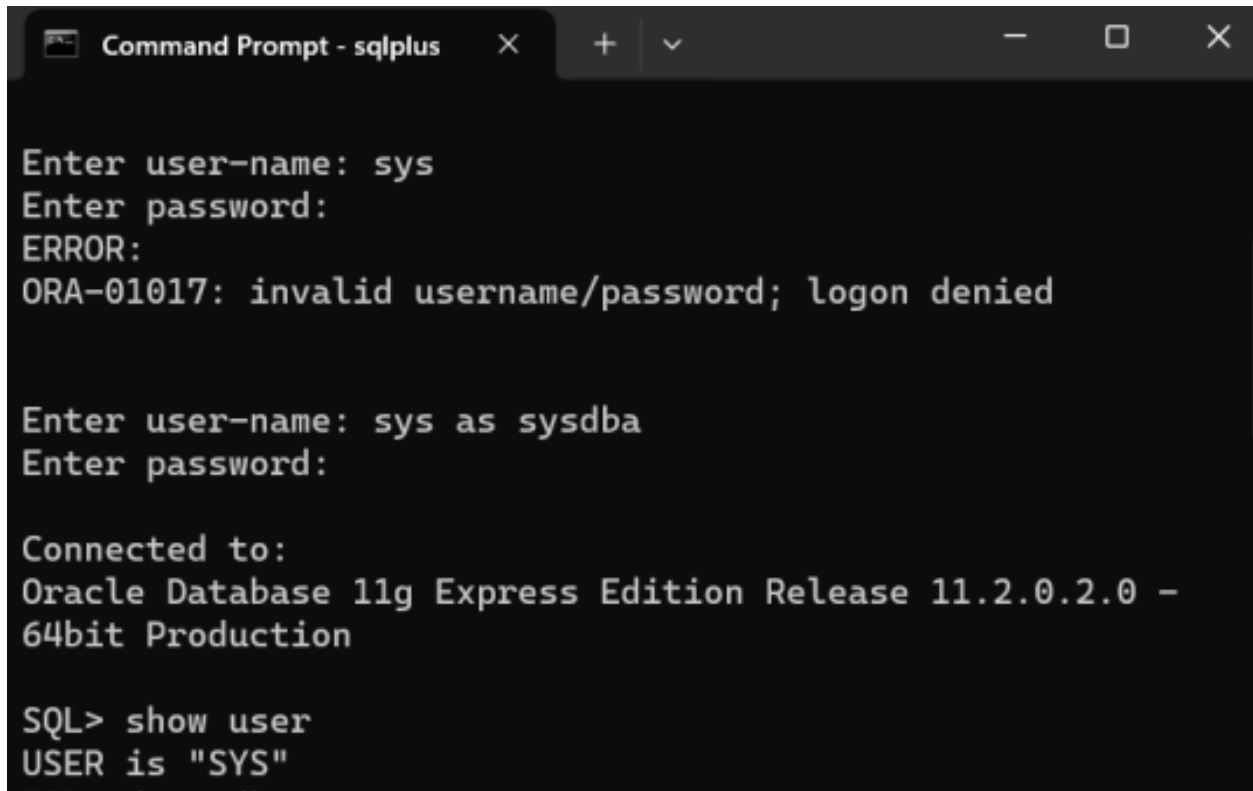
CONN / AS SYSDBA

CONN SYS AS SYSDBA

➤ **REGULAR USER**

CONN USER / PASSWORD

Login by User Admin



```
Command Prompt - sqlplus

Enter user-name: sys
Enter password:
ERROR:
ORA-01017: invalid username/password; logon denied

Enter user-name: sys as sysdba
Enter password:

Connected to:
Oracle Database 11g Express Edition Release 11.2.0.2.0 -
64bit Production

SQL> show user
USER is "SYS"
```

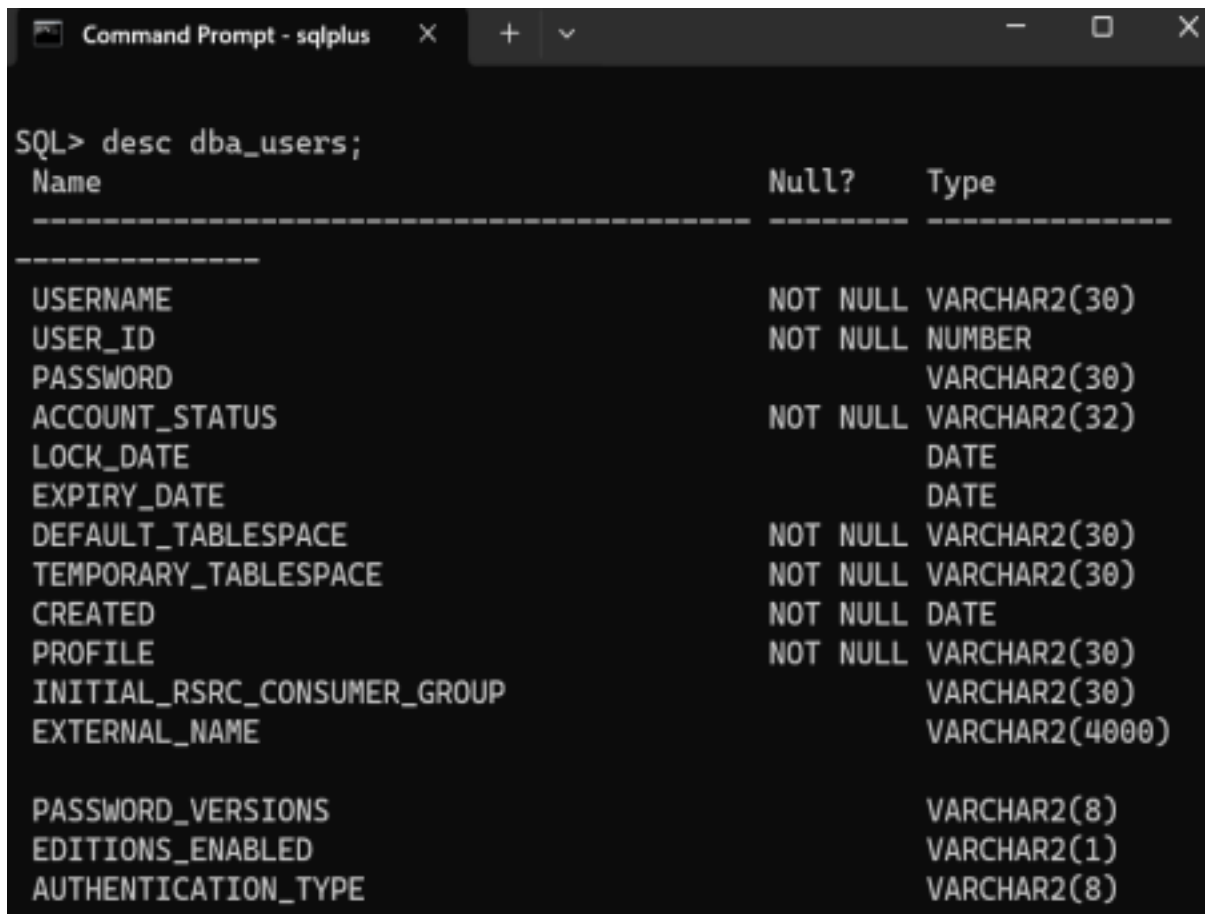
'SYS' USER NAME:

SYS AS SYSDBA

PASSWORD:

ADMIN

Show List of users table columns DESC DBA_USERS;



```
SQL> desc dba_users;
```

Name	Null?	Type
USERNAME	NOT NULL	VARCHAR2(30)
USER_ID	NOT NULL	NUMBER
PASSWORD		VARCHAR2(30)
ACCOUNT_STATUS	NOT NULL	VARCHAR2(32)
LOCK_DATE		DATE
EXPIRY_DATE		DATE
DEFAULT_TABLESPACE	NOT NULL	VARCHAR2(30)
TEMPORARY_TABLESPACE	NOT NULL	VARCHAR2(30)
CREATED	NOT NULL	DATE
PROFILE	NOT NULL	VARCHAR2(30)
INITIAL_RSRC_CONSUMER_GROUP		VARCHAR2(30)
EXTERNAL_NAME		VARCHAR2(4000)
PASSWORD_VERSIONS		VARCHAR2(8)
EDITIONS_ENABLED		VARCHAR2(1)
AUTHENTICATION_TYPE		VARCHAR2(8)

Show users from DBA

```
SQL> SELECT USERNAME, CREATED FROM DBA_USERS;
```

USERNAME	CREATED
SYS	29-MAY-14
SYSTEM	29-MAY-14
ANONYMOUS	29-MAY-14
APEX_PUBLIC_USER	29-MAY-14
FLows_FILES	29-MAY-14
APEX_040000	29-MAY-14
OUTLN	29-MAY-14
DIP	29-MAY-14
ORACLE_OCM	29-MAY-14
XS\$NULL	29-MAY-14
MDSYS	29-MAY-14

USERNAME	CREATED
CTXSYS	29-MAY-14
DBSNMP	29-MAY-14
XDB	29-MAY-14
APPQOSSYS	29-MAY-14
HR	29-MAY-14

```
16 rows selected.
```

SELECT

USERNAME, CREATED FROM

DBA_USERS;

Create User using SQLPLUS

CREATE USER ALI

IDENTIFIED BY 123

```
SQL> create user ali identified by 123;
```

```
User created.
```

Connecting User

User (ALI) can not connect to the server!

He needs privilege....

```
SQL> conn ali/123
ERROR:
ORA-01045: user ALI lacks CREATE SESSION privilege;
logon denied
```

Thank you!

Database 2

ORACLE ENVIRONMENT

LAB 2

User Privileges 1. System Privileges

Right access to specific users to do **administration tasks**.

Examples

Create, Drop, Disconnect User Recovery and Backups

Export and Import

2. Object Privileges

Right access to specific users to do **actions on objects**.

Examples

select, insert, delete, update

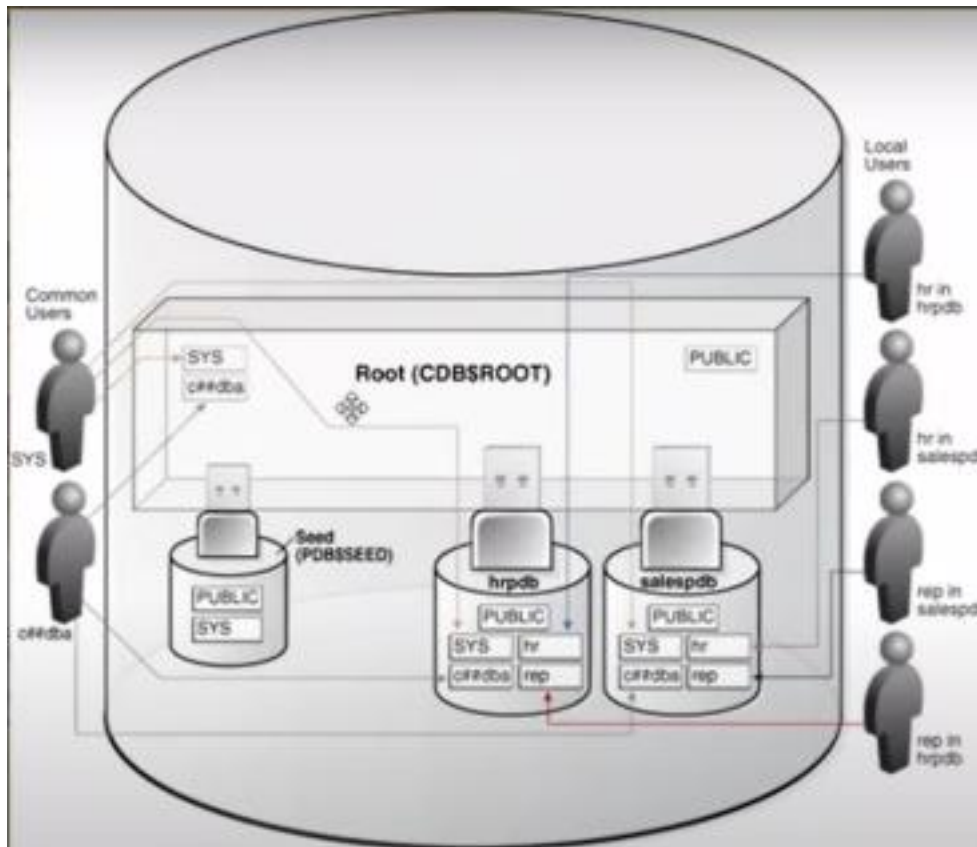
Schema

- A schema is a collection of database objects.
- A schema is owned by a database user and has the same name as that user.
- Schema objects are logical structures created by users.
- Objects such as tables or indexes hold data, or can consist of a definition only, such as a view or synonym.

Database Objects ▪ Tables

- Triggers
- Function
- procedures
- Indexes
- Sequence
- synonyms

Schema



PL/ SQL

PL/SQL is a procedural language designed specifically to embrace SQL statements within its syntax.

PL/SQL program units are compiled by the Oracle Database server and stored inside the database.

At run-time, both PL/SQL and SQL run within the same server process, bringing optimal efficiency.

PL/SQL automatically inherits the robustness, security, and portability of the Oracle Database.

Building and Managing PL/SQL Program Units

- **Building with Blocks**: PL/SQL is a block-structured language; familiarity with blocks is critical to writing good code.
- **Controlling the Flow of Execution**: Conditional branching and iterative processing in PL/SQL.
- **Wrap Your Code in a Neat Package**: Packages are the fundamental building blocks of any high quality PL/SQL-based application.

- **Picking Your Packages**: Concepts and benefits of PL/SQL packages.
- **Error Management**: An exploration of error management features in PL/SQL.

- **The Data Dictionary: Make Views Work for You**: Use several key data dictionary views to analyze and manage your code.

Building blocks of PL/SQL programs

PL/SQL is a **block-structured language**. A PL/SQL block is defined by the keywords **DECLARE**, **BEGIN**, **EXCEPTION**, and **END**, which break up the block into three sections:

1. Declarative: Statements that **declare variables**, constants, and other code elements, which can then be used within that block

2. Executable: Statements that are run when the block is executed

3. Exception handling: A specially structured section you can use to “catch,” or trap, any exceptions that are raised when the executable section runs.

A block itself is an executable statement, so you can nest blocks within other blocks.

Types of Blocks

PL/SQL blocks are units of code that can include declarations, executable statements, exception handlers, and other PL/SQL constructs.

There are several types of PL/SQL blocks, each serving a specific purpose.

1. Anonymous Block:

- Not stored in the database and has no name.
- Used for one-time execution, such as ad-hoc SQL scripts or quick procedural tasks.

2. Named Blocks:

- Stored in the database and can be executed many times.

Anonymous Block:

```
DECLARE
-- VARIABLES DECLARATION
BEGIN
-- Executable statements
EXCEPTION
-- Exception Handlers
END;
/
```

Example

```
SQL> set serveroutput on;
SQL> begin
  2  dbms_output.put_line('hello');
  3  end;
  4  /
hello
```

set serveroutput on: *enable the display of the output generated by DBMS_OUTPUT.PUT_LINE or other output commands within PL/SQL blocks.*

Named Blocks:

Block

Name

Stored

Procedure:

Description

A stored procedure is a named PL/SQL block that is stored in the database for reuse. It can be called explicitly by name.

Function: A function is similar to a stored procedure **but returns a value**. It is called as part of an expression.

Trigger: A trigger is a type of PL/SQL block associated with a table or view. It is automatically executed in response to specific events, such as INSERT, UPDATE, or DELETE.

Package: A package is a collection of related PL/SQL objects (procedures, functions, variables, etc.) grouped together.

It provides a way to organize and encapsulate code for modularity and reusability.

PL/SQL Variables Datatypes

- **Working with Strings in PL/SQL:** PL/SQL offers several different string datatypes for use in your applications
- **Working with Numbers in PL/SQL:** Learn about and how to use the different numeric types in PL/SQL.
- **Working with Dates in PL/SQL:** Dates are a relatively complex scalar datatype, in both SQL and PL/SQL
- **Working with Records:** A very common and useful composite type, PL/SQL's analogue to a table's row
- **Error Management:** PL/SQL's implementation of arrays plays a role in almost every key performance feature of PL/SQL
- **Working with Collections:** Use several key data dictionary views to analyze and manage your code

Numbers Data Type Declaration in PL/SQL

PL/SQL features various numeric types for different purposes, including:

NUMBER: A true decimal data type that is ideal for working with monetary amounts.

PLS_INTEGER: An integer type aligned with hardware's native representation, utilizing machine instructions for arithmetic. Not suitable for table storage; exclusive to PL/SQL.

SIMPLE_INTEGER.

BINARY_FLOAT and BINARY_DOUBLE.

Other numeric types like FLOAT, INTEGER, and DECIMAL are encountered, serving as subtypes of the core numeric types listed above.

Numbers Data Type

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2  new_number NUMBER:= 123.45;
  3  new_integer PLS_INTEGER:= 42.5;
  4  BEGIN
  5  dbms_output.put_line(new_number);
  6  dbms_output.put_line(new_integer);
  7  END;
  8  /
123.45
43
```

Numbers Calculations

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2  num1 INTEGER := 1;
  3  num2 INTEGER := 2;
  4  result INTEGER := num1+num2;
  5  BEGIN
  6  dbms_output.put_line('Result is : ' || result);
  7  END;
  8  /
Result is : 3

PL/SQL procedure successfully completed.
```

Nested Blocks / Global and Local Variable

```

SQL> DECLARE
  2  var1 varchar(20) := 'Golbal Variable';
  3  BEGIN
  4      DECLARE
  5      var2 varchar(20) := 'Local Variable';
  6      BEGIN
  7      dbms_output.put_line(var1);
  8      dbms_output.put_line(var2);
  9      END;
 10  END;
 11  /
Golbal Variable
Local Variable

```

Nested Blocks / Global and Local Variable

```

SQL> DECLARE
  2  var1 varchar(20) := 'Golbal Variable';
  3  BEGIN
  4      DECLARE
  5      var2 varchar(20) := 'Local Variable';
  6      BEGIN
  7      dbms_output.put_line(var2);
  8      END;
  9  dbms_output.put_line(var1);
 10  dbms_output.put_line(var2);
 11  END;
 12  /

```

```

dbms_output.put_line(var2);

```

*

ERROR at line 10:

ORA-06550: line 10, column 22:

PLS-00201: identifier 'VAR2' must be declared

ORA-06550: line 10, column 1:

PL/SQL: Statement ignored

Employee table contain following Data It will be used in the next examples

ID	NAME	DEP_NUM	SALARY
1	ali	1	70000
2	selim	1	40000
4	khaled	2	50000
5	karim	1	40000
3	rawan	2	55000

Retrieving Data from Table

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  idVar INTEGER;
  3  nameVar VARCHAR(20);
  4  BEGIN
  5  SELECT ID, NAME INTO idVar, nameVar
  6  FROM EMPLOYEE WHERE ID= 1;
  7  dbms_output.put_line(idVar || ' ' || nameVar);
  8  END;
  9  /
1 ali

PL/SQL procedure successfully completed.
```

SELECT INTO statement to retrieve data from a table and assign it to a variable.

SELECT COL_NAME

INTO VARIABLE

FROM TABLE_NAME;

PL / SQL Extension to SQL provide procedural commands like

Variables

Arrays

If Else Conditions

Loops

Functions

Procedures

Triggers

IF / ELSE Keyword

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  grade integer := 85;
  3  BEGIN
  4  if (grade >= 85 and grade <= 100)
  5  then
  6  dbms_output.put_line('A');
  7  elsif(grade < 85 and grade >= 75)
  8  then
  9  dbms_output.put_line('B');
 10  END IF;
 11  END;
 12  /
A
```

Procedure IF [(condition)]

THEN

< statement>

ELSIF [(condition)]

THEN

< statement>

ELSE [(condition)]

< statement>

END IF;

Employee table contain following Data It will be used in the next examples

ID	NAME	DEP_NUM	SALARY
1	ali	1	70000
2	selim	1	40000
4	khaled	2	50000
5	karim	1	40000
3	rawan	2	55000

IF / ELSE Keyword

Update Salary from Department 1 by increasing the rate by 10 %

```
SQL> CREATE OR REPLACE PROCEDURE updateSalary(dNumVar integer) AS
  2 BEGIN
  3 IF (dNumVar = 1)
  4 then
  5 update employee set salary= salary+salary *.1
  6 where DEP_NUM = 1;
  7 ELSE
  8 dbms_output.put_line('Nothing Updated!');
  9 END IF;
 10 END;
 11 /

Procedure created.
```

Executing Procedure

```
SQL> EXECUTE updateSalary(1);
```

```
PL/SQL procedure successfully completed.
```

```
SQL> select * from employee;
```

ID	NAME	DEP_NUM	SALARY
1	ali	1	77000
2	selim	1	44000
4	khaled	2	50000
5	karim	1	44000
3	rawan	2	55000

Thank you!

.....

Database 2

PL / SQL

LAB 4

Types of loops: **1. Basic Loop.**

2. For Loop.

3. While Loop.

Basic Loop

1-Basic Loop: With each iteration of the basic LOOP statement, its statements run and control returns to the top of the loop. The LOOP statement ends when a statement inside the loop transfers control outside the loop or raises an exception.

basic_loop_statement ::=



Basic Loop Syntax

Begin

Loop

-- Code to execute in each iteration -- increment statements

EXIT WHEN condition

END LOOP;

END;

Example

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  counter INTEGER :=1;
  3  BEGIN
  4  LOOP
  5  dbms_output.put_line(' Hello World  ' || counter );
  6  EXIT WHEN counter >= 10;
  7  counter := counter + 1;
  8  END LOOP;
  9  END;
 10  /
```

```
Hello World 1
Hello World 2
Hello World 3
Hello World 4
Hello World 5
Hello World 6
Hello World 7
Hello World 8
Hello World 9
Hello World 10
```

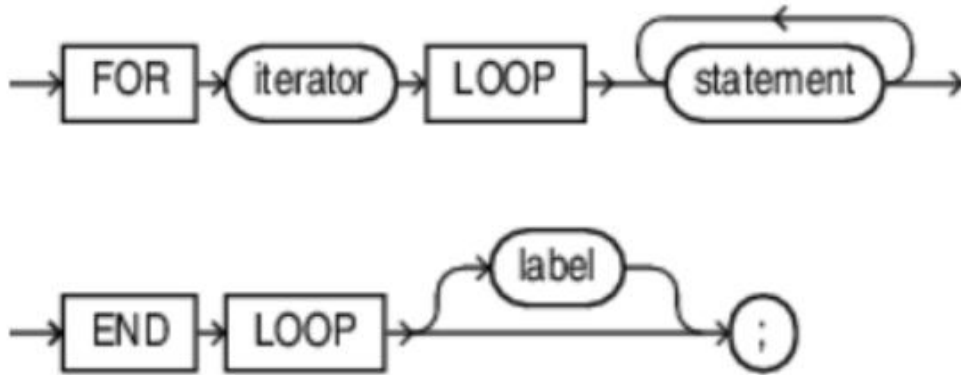
For Loop:

2. For Loop: With each iteration of the FOR LOOP statement, its statements run, its index is either incremented or decremented, and control returns to the top of the loop.

The FOR LOOP statement ends when its index reaches a specified value, or when a statement inside the loop transfers control outside the loop or raises an exception. An index is also called an iterand. Statements outside the loop cannot reference the iterand. After the FOR LOOP statement runs, the iterand is undefined.

For Loop:

for_loop_statement ::=



FOR LOOP

Begin

FOR counter_variable IN start_value .. end_value Loop

-- Code to execute in each iteration

END LOOP;

END;

FOR LOOP (Increment)

```
SQL> SET SERVEROUTPUT ON
SQL> BEGIN
  2  FOR counter in 1 .. 10
  3  LOOP
  4  dbms_output.put_line(' Hello World  ' || counter );
  5  END LOOP;
  6  END;
  7  /
```

```
Hello World 1
Hello World 2
Hello World 3
Hello World 4
Hello World 5
Hello World 6
Hello World 7
Hello World 8
Hello World 9
Hello World 10
```

FOR LOOP (REVERSE)

```

SQL> SET SERVEROUTPUT ON
SQL> BEGIN
  2  FOR counter in reverse 1 .. 10
  3  LOOP
  4  dbms_output.put_line(' Hello World  ' || counter );
  5  END LOOP;
  6  END;
  7  /

```

```

Hello World  10
Hello World   9
Hello World   8
Hello World   7
Hello World   6
Hello World   5
Hello World   4
Hello World   3
Hello World   2
Hello World   1

```

While Loop:

The WHILE LOOP statement runs one or more statements while a condition is TRUE.

The WHILE LOOP statement ends when the condition becomes FALSE or NULL, or when a statement inside the loop transfers control outside the loop or raises an exception.

while_loop_statement ::=



WHILE LOOP Syntax **Begin**

WHILE condition

LOOP

-- Code to be executed END LOOP;

END;

WHILE LOOP EXAMPLE

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2 counter NUMBER := 1;
  3 BEGIN
  4     WHILE counter <= 10
  5     LOOP
  6         DBMS_OUTPUT.PUT_LINE('Counter: ' || counter);
  7         counter := counter + 1;
  8     END LOOP;
  9     END;
10 /
```

```
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
Counter: 6
Counter: 7
Counter: 8
Counter: 9
Counter: 10
```

Functions in PL/SQL

CREATE [OR REPLACE FUNCTION] function_name

1. Built in System Function 2. User Defined Functions

[(parameter_name [IN | OUT | IN OUT] type [, ...])] RETURN function_return_value {IS | AS}
return_variable -- Function body goes here

BEGIN

-- Code goes here

RETURN some_value;

END function_name;

Allow user to create Procedure or Function

```
SQL> grant create procedure to ali;  
  
Grant succeeded.
```

Functions Example

```
SQL> CREATE OR REPLACE FUNCTION add_numbers(num1 NUMBER, num2 NUMBER)  
2 RETURN NUMBER IS result NUMBER;  
3 BEGIN  
4 result := num1 + num2;  
5 RETURN result;  
6 END add_numbers;  
7 /
```

Function created.

```
SQL> SELECT add_numbers(1,2) from dual;  
  
ADD_NUMBERS(1,2)  
-----  
3
```

DUAL Table

DUAL is a one-row, one-column pseudo table that is often used for selecting a constant value or

performing operations where a table reference is required but the actual data is not important.

```
SQL> select upper('test') from dual;  
  
UPPE  
----  
TEST
```

Employee table contain following Data It will be used in the next examples

Example

Get count of Employees in the same Department

Result In SQL

Result In PL/SQL

Bulletin Functions

```
select EMPNAME from Emp;
```

```
select lower(empname) from emp;
```

```
select Upper(empname) from emp;
```

```
select Initcap(empname) from emp;
```

```
update emp set empname=upper(empname);
```

```
select empname , substr(empname,2) from emp;--from 2 to end  
select empname , replace(empname , 'H' , 's')as rep from emp;  
SELECT empname , length(empname) from emp;
```

```
select concat(empname , salary) from emp;
```

```
select lpad(empname,10,'*')from emp;
```

```
select rpad(empname,10,'*')from emp;
```

Thank you!

.....

Database 2

PL / SQL

LAB 6

Outline

Lab 6	PL/SQL
1	Examples of Function and Procedural
2	Triggers

Employee table contain following Data It will be used in the next examples

ID	NAME	DEP_NUM	SALARY
1	ali	1	70000
2	selim	1	40000
4	khaled	2	50000
5	karim	1	40000
3	rawan	2	55000

Example

Get max salary of employee based on department number

```
SQL> CREATE OR REPLACE FUNCTION getMaxSalary (depNumVar number)
  2 RETURN NUMBER AS maxSalary NUMBER;
  3 BEGIN
  4 SELECT MAX(SALARY) INTO maxSalary FROM EMPLOYEE
  5 WHERE DEP_NUM= depNumVar;
  6 RETURN maxSalary;
  7 END;
  8 /
```

Function created.

Result In SQL

```
SQL> select NAME, DEP_NUM, countEmployee(DEP_NUM) as count,
  getMaxSalary(DEP_NUM) as max FROM EMPLOYEE;
```

NAME	DEP_NUM	COUNT	MAX
ali	1	3	70000
selim	1	3	70000
ahmed	2	2	55000
karim	1	3	70000
rawan	2	2	55000

Result In PL/SQL

```
SQL> BEGIN
  2 dbms_output.put_line(getMaxSalary(1));
  3 END;
  4 /
70000
```

PL/SQL procedure successfully completed.

IN | OUT | IN_OUT Parameter

Parameter Mode	Direction	Description	Example Use
IN	Input to the procedure/function	Used to pass a value from the caller to the PL/SQL block . The value cannot be changed inside the procedure.	emp_id IN NUMBER
OUT	Output from the procedure/function	Used to return a value from the procedure back to the caller . The variable is assigned inside the procedure.	total_salary OUT NUMBER
IN OUT	Both input and output	Used to send a value in and return an updated value out . The variable is read and modified inside the procedure.	Balance IN OUT NUMBER

IN/ OUT Parameters in Functions

```
SQL> CREATE OR REPLACE FUNCTION getMaxMinSalary(depNumVar in
integer, maxSalary out integer, minSalary out integer)
2 return integer as countEmp integer;
3 BEGIN
4 SELECT COUNT(*), MAX(SALARY), MIN(SALARY)
5 INTO countEmp, maxSalary, minSalary
6 FROM EMPLOYEE WHERE DEP_NUM= depNumVar;
7 RETURN countEmp;
8 END;
9 /
```

Function created.

Result In PL/SQL

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
2 maxSalary integer;
3 minSalary integer;
4 countEmp integer;
5 BEGIN
6 countEmp := getMaxMinSalary(1, maxSalary, minSalary);
7 dbms_output.put_line(maxSalary);
8 dbms_output.put_line(minSalary);
9 END;
10 /
70000
40000
```

Granting Function to other user

```
SQL> GRANT EXECUTE ON getMaxMinSalary TO ahmed;

Grant succeeded.
```

Granting Function to other user

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2  maxSalary integer;
  3  minSalary integer;
  4  countEmp integer;
  5  BEGIN
  6  countEmp := ali.getMaxMinSalary(1, maxSalary, minSalary);
  7  dbms_output.put_line(maxSalary);
  8  dbms_output.put_line(minSalary);
  9  END;
 10  /
70000
40000
```

Procedures in PL/SQL

CREATE OR REPLACE PROCEDURES

➤ Procedures is a database object that contains a set of PL/SQL code to perform specific task.

PROCEDURE_name

[(parameter_name [IN | OUT | IN OUT] type [, ...])]

{AS}

-- Procedure body goes here BEGIN

-- Code goes here

END;

Procedure Example

```
SQL> CREATE OR REPLACE PROCEDURE sayHello AS  
  2 BEGIN  
  3 dbms_output.put_line('Hello World');  
  4 END;  
  5 /
```

Procedure created.

Executing Procedure

```
SQL> execute sayHello;  
Hello World  
  
PL/SQL procedure successfully completed.  
  
SQL> exec sayHello;  
Hello World  
  
PL/SQL procedure successfully completed.
```

Results in PL/SQL


```
SQL> SET SERVEROUTPUT ON
SQL> BEGIN
  2  sayHello();
  3  END;
  4  /
Hello World

PL/SQL procedure successfully completed.
```

Procedure Example

```
SQL> create or replace PROCEDURE addNums(num1 integer ,
num2 integer) as result integer;
  2  BEGIN
  3  result := num1+num2;
  4  dbms_output.put_line(result);
  5  END;
  6  /

Procedure created.
```

Executing Procedure

```
SQL> execute addNums(1,2);
3

PL/SQL procedure successfully completed.
```

Results in PL/SQL

```
SQL> SET SERVEROUTPUT ON
```

```
SQL> BEGIN
```

```
2 addNums(1,2);
```

```
3 END;
```

```
4 /
```

```
3
```

```
PL/SQL procedure successfully completed.
```

DML Statement in Procedure Example

```
SQL> create or replace PROCEDURE updateDep(dNumVar integer ,  
nameVar varchar) AS
```

```
2 BEGIN
```

```
3 update employee set DEP_NUM=dNumVar where name =nameVar;
```

```
4 commit;
```

```
5 DBMS_OUTPUT.PUT_LINE(nameVar||' updated sus');
```

```
6 END;
```

```
7 /
```

```
Procedure created.
```

Executing Procedure

Employee table contain following Data It will be used in the next examples

IF / ELSE Keyword Procedure

Update Salary from Department 1 by increasing the rate by 10 %

Executing Procedure

Function VS Procedure

Function	Procedure
Functions always return a value after the execution of queries.	The procedure can return a value using "IN OUT" and "OUT" arguments.
In SQL, those functions having a DML statement can not be called from SQL statements. But autonomous transaction functions can be called from SQL queries.	A procedure can not be called using SQL queries.
A function can be operated in the SELECT statement.	While it can't be operated in the SELECT statement.
The function can be called using Stored Procedure.	While procedures cannot be called from function.

Triggers

Triggers in Oracle are database objects that automatically execute a specified block of code (a PL/SQL program) in response to certain events on a particular table, view, or schema.

These events can occur during operations such as **INSERT**, **UPDATE**, or **DELETE**.

Key Features of Triggers:

Event-driven: Automatically executed when a specified event occurs.

Timing: Can execute BEFORE or AFTER the event.

Scope: Can be defined for individual rows (FOR EACH ROW) or for the entire table.

Security: Enforces business rules and data integrity automatically.

Types of Triggers:

1.DML Triggers:

•**BEFORE Triggers:** Execute before the DML event. •**AFTER Triggers:** Execute after the DML event.

•**INSTEAD OF Triggers:** Execute instead of a DML operation (mainly for views).

•Example: Restrict changes to certain rows in a table during an update.

Types of Triggers:

2. DDL Triggers:

•Triggered by Data Definition Language (DDL) statements like CREATE, ALTER, DROP.

•Typically used for logging or enforcing rules on schema changes.

3. Database Triggers:

•Triggered by system-level events such as user logon, logoff, or errors.

•Useful for auditing and monitoring.

Syntax for DML Triggers:

Example 1: BEFORE INSERT Triggers: Prevent inserting records into a table if a specific condition is met.

Example 2: AFTER INSERT Triggers:

Thank you!

.....

Database 2

PL / SQL

LAB 6

Outline

Lab 6	PL/SQL
1	Transaction Concurrency Control
2	Block Session

Transaction Concurrency Control

SESSION 1 SESSION 2

```
SQL> SELECT * FROM TEST;
```

12 no rows selected

```
SQL> CREATE TABLE TEST (id INTEGER PRIMARY KEY,  
name varchar(20));
```

Table created.

```
SQL> SELECT * FROM TEST;
```

no rows selected

```
SQL> INSERT INTO TEST VALUES (1, 'A');
```

1 row created.

```
SQL> SELECT * FROM TEST;
```

ID	NAME
1	A

Transaction Concurrency Control

SESSION 1 SESSION 2

```
SQL> commit;
```

Commit complete.

```
SQL> SELECT * FROM TEST;
```

ID	NAME
1	A

Transaction Concurrency Control

SESSION 1 SESSION 2

```
SQL> SELECT * FROM TEST;
```

ID	NAME
1	A

```
SQL> rollback;
```

```
Rollback complete.
```

```
SQL> INSERT INTO TEST VALUES (2, 'B');
```

```
1 row created.
```

```
SQL> SELECT * FROM TEST;
```

ID	NAME
1	A
2	B

```
SQL> SELECT * FROM TEST;
```

ID	NAME
1	A

Transaction Concurrency Control

SESSION 1 SESSION 2

Transaction Concurrency Control SESSION 1 SESSION 2

Adding a new employee has been cancelled.

Salary modification is cancelled.

Transaction Concurrency Control SESSION 1 SESSION 2 ? Flashing!!

Transaction Concurrency Control SESSION 1 SESSION 2 ? Flashing!!

V\$LOCK Blocking and Waiting Sessions ID from

V\$LOCK contains

an entry for all

locks held as well

as locks being

waited on.

Blocking and Waiting Sessions ID

V\$SESSION Table;

V\$SESSION displays session information for each current session.

Thank you!

.....

DATABASE PROGRAMMING

Oracle SQL/PL-SQL

Skills and case statement Complete Guide to Database Cursor Programming

Oracle Database Training Series

What is an anchored data type?

Definition

Tell PL/SQL to set the datatype of your variable based on the datatype of an already defined data structure or table.

Key Characteristics

- ▶ Automatically Inherits Data Type
- ▶ Automatically Adapts to Schema Changes ▶ Reduces Errors and Increases Consistency

Anchored datatype like:

- %TYPE
- %ROWTYPE

Syntax

TABLE_NAME.COLUMN_NAME%TYPE

How to make a function return a record?

	ID	NAME	DEP_NUM	SALARY
1	1	ali	1	75000
2	2	selim	1	45000
3	3	rawan	2	50000

EMP TABLE

```
CREATE OR REPLACE FUNCTION fn_getEmpRow(emp_id EMP.ID%TYPE)
Return EMP%ROWTYPE
AS
v_record EMP%ROWTYPE;
Begin
    select * into v_record
    from emp
    where id = emp_id;
    return v_record;
End;
```

```
BEGIN
DBMS_OUTPUT.put_line(fn_getEmpRow(1));
END;
```

How to make a

function return a record?

Execute function

```
select fn_getEmpRow(1) from emp;
```

ERROR: can not select one record

```
ORA-06553: PLS-801: internal error [55018]
06553. 00000 - "PLS-%s: %s"
*Cause:   SQL compilation failed because of an illegal reference to a
          PL/SQL unit.
*Action:  If the PL/SQL error message does not provide a workaround, then
          try the SQL statement as a top-level call as the appropriate user
          with the appropriate privileges.
Error at Line: 28 Column: 8
```

Error report -

```
ORA-06550: line 2, column 1:
PLS-00306: wrong number or types of arguments in call to 'PUT_LINE'
ORA-06550: line 2, column 1:
PL/SQL: Statement ignored
06550. 00000 - "line %s, column %s:\n%s"
*Cause:    Usually a PL/SQL compilation error.
*Action:
```

How to make a function return a record?

The Solution

```
DECLARE
V_rec emp%rowtype;
BEGIN
V_rec := fn_getEmpRow(1);
DBMS_OUTPUT.put_line(V_rec.id || ' ' || V_rec.name);
END;
```

PL/SQL procedure successfully completed.

1 ali

	ID	NAME	DEP_NUM	SALARY
1	1	ali	1	75000
2	5	selim	1	45000
3	8	rawan	2	50000
4	9	khaled	2	55000
5	15	karim	1	48000
6	20	karim	3	33000

ROW_COUNT() FUNCTION

Definition

Make a serial_number/index on the data in a table.

Syntax

Select Row_Count() over (order by column_name [desc])

```
SELECT
    ROW_NUMBER() OVER (ORDER BY SALARY DESC) AS RN,
    ID, NAME, DEP_NUM
FROM EMP;
```

	RN	ID	NAME	DEP_NUM	SALARY
1	1	1	ali	1	75000
2	2	9	khaled	2	55000
3	3	8	rawan	2	50000
4	4	15	karim	1	48000
5	5	5	selim	1	45000
6	6	20	karim	3	33000

ROW_COUNT() FUNCTION

	RN	ID	NAME	DEP_NUM	SALARY
1	1	1	ali	1	75000
2	2	9	khaled	2	55000
3	3	8	rawan	2	50000
4	4	15	karim	1	48000
5	5	5	selim	1	45000
6	6	20	karim	3	33000

*

Find the third max salary

```
SELECT RN, ID, NAME, DEP_NUM, SALARY FROM (
    SELECT
        ROW_NUMBER() OVER (ORDER BY SALARY DESC) AS RN,
        ID, NAME, DEP_NUM, SALARY
    FROM EMP
) WHERE RN = 3;
```

	RN	ID	NAME	DEP_NUM	SALARY
1	3	8	rawan	2	50000

ROW_COUNT() FUNCTION

Find the 3, 5, 6 max salary

What is CASE in PL/SQL

- CASE is a conditional expression used to make decisions.
- Works similarly to switch-case in other programming languages.
- Replaces complex IF-ELSIF-ELSE structures.

- Can be used in:

- o SQL statements

- o PL/SQL blocks

- o SELECT, UPDATE, INSERT, DELETE

Two Types of CASE:

1. Simple CASE

- Compares one expression with multiple fixed values.

2. Searched CASE

- Uses conditions instead of matching single values.

Simple CASE Syntax

Syntax Example

CASE expression

WHEN value1 THEN result1 WHEN value2 THEN result2 ELSE default_result

END;

DECLARE

grade CHAR(1) := 'A';

BEGIN

CASE grade

WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent'); WHEN 'B' THEN
DBMS_OUTPUT.PUT_LINE('Very Good'); ELSE DBMS_OUTPUT.PUT_LINE('Needs
Improvement'); END CASE;

END;

Searched CASE Syntax

Syntax Example

CASE

WHEN condition1 THEN result1 WHEN condition2 THEN result2 ELSE default_result

END;

DECLARE

salary NUMBER := 4000;

BEGIN

CASE

WHEN salary > 5000 THEN DBMS_OUTPUT.PUT_LINE('Highsalary'); WHEN salary BETWEEN
3000 AND 5000 THEN

```
DBMS_OUTPUT.PUT_LINE('Medium salary');  
ELSE DBMS_OUTPUT.PUT_LINE('Low salary');  
END CASE;  
END;
```

CASE Inside a SQL Query Example

```
SELECT  
name,  
salary,  
CASE  
WHEN salary >= 50000 THEN 'High' WHEN salary >= 30000 THEN 'Medium' ELSE 'Low'  
END AS salary_level  
FROM emp;
```

CASE Statement

Why Use CASE?

- Makes code cleaner and easier to read.
- Replaces multiple IF statements.
- Works directly in SQL queries.
- Great for:
 - Categorizing data
 - Creating reports
 - Data transformation
 - Conditional formatting

Limitations of CASE

- Cannot execute multiple statements inside the CASE block.
- Returns only one value.

- Not suitable for complex logic → Use IF instead.

Thank You

Oracle Database Training Series

Questions and Answers:

1 What is the main goal of heuristic query optimization?

- A. To validate SQL syntax
- B. To reduce intermediate result size ☒
- C. To increase query complexity
- D. To ignore access paths

Explanation: Heuristic optimization focuses on reducing the size of intermediate results by reordering operations like SELECT and PROJECT.

2 Which operation is typically applied first in heuristic optimization?

- A. JOIN
- B. SELECT ☒
- C. UNION

D. GROUP BY

Explanation: SELECT is applied early to filter rows and reduce the data size before performing expensive operations like JOIN.

3 What does a query tree represent?

- A. A list of SQL keywords
- B. A relational algebra expression with execution order 
- C. A graph of user queries
- D. A set of database constraints

Explanation: A query tree shows the sequence of relational operations and how they are applied to input relations.

4 Which transformation rule allows breaking a compound SELECT condition into multiple operations? A. Commutativity of JOIN

- B. Cascade of SELECT 
- C. Cascade of PROJECT
- D. Commutativity of UNION

Explanation: Cascade of SELECT splits a conjunctive condition into a sequence of individual SELECT operations.

5 What is the benefit of applying PROJECT early in a query tree?

- A. To increase the number of attributes
- B. To reduce the number of attributes processed 

C. To simplify the WHERE clause

D. To avoid using indexes

Explanation: PROJECT reduces the number of columns, minimizing the size of intermediate results.

6 Which relational algebra operations are commutative?

A. JOIN and Cartesian Product 

B. PROJECT and SELECT

C. DIFFERENCE and UNION

D. SELECT and INTERSECTION

Explanation: JOIN and Cartesian Product are commutative, meaning the order of relations can be swapped without affecting the result.

7 What does the cost-based optimizer use to choose an execution plan?

A. Fixed rule points

B. SQL syntax

C. Estimated resource usage 

D. Table names

Explanation: Cost-based optimizers evaluate CPU, I/O, and memory usage to select the most efficient plan.

8 Which catalog information is used to calculate selection cardinality?

- A. Index levels and block factor
- B. Selectivity and number of records 
- C. CPU usage and memory
- D. Record size and number of blocks

Explanation: Selection cardinality is calculated as $s = sl \times r$, where sl is selectivity and r is the number of records.

9 What is the purpose of pipelined evaluation in query execution?

- A. To store intermediate results
- B. To delay execution
- C. To forward results directly to the next operator 
- D. To increase memory usage

Explanation: Pipelined evaluation avoids temporary storage by passing results directly between operations.

10 Which set operations are commutative?

- A. UNION and INTERSECTION 
- B. DIFFERENCE and PRODUCT
- C. JOIN and SELECT
- D. PROJECT and UNION

Explanation: UNION and INTERSECTION are commutative, meaning the order of operands does not affect the result.

1 1 What does associativity of JOIN allow?

- A. Changing the order of SELECT conditions**
- B. Reordering parentheses in JOIN operations **
- C. Ignoring PROJECT operations**
- D. Applying SELECT after UNION**

Explanation: Associativity allows changing the grouping of JOIN operations without affecting the result.

1 2 What is the effect of converting a SELECT after a Cartesian product into a JOIN? A. It increases result size

- B. It improves execution efficiency **
- C. It removes filtering**
- D. It changes the query result**

Explanation: Replacing Cartesian product and SELECT with JOIN avoids unnecessary combinations and improves performance.

1 3 In Oracle, why might a developer use hints in a query?

- A. To change SQL syntax**
- B. To guide the optimizer to a specific plan **
- C. To disable indexing**
- D. To increase query cost**

Explanation: Hints allow developers to influence the optimizer based on their knowledge of the data.

1 4 What is the main heuristic for algebraic optimization?

- A. Apply operations that increase result size
- B. Apply SELECT and PROJECT early 
- C. Apply JOIN before filtering
- D. Apply UNION before PROJECT

Explanation: Applying SELECT and PROJECT early reduces the size of intermediate results and improves performance.

1 5 What does a query graph represent?

- A. Execution order of operations
- B. Relationships between tables without execution order 
- C. SQL syntax structure
- D. Index usage

Explanation: A query graph shows how tables are related but does not specify the order of operations.

1 6 What does selectivity measure in query optimization?

- A. The number of columns in a table
- B. The percentage of rows satisfying a condition 
- C. The size of the index

D. The number of joins in a query

Explanation: Selectivity indicates how many rows match a condition, helping estimate query cost.

1 7 Which cost component reflects the time to read data from disk?

A. Computation cost

B. Access cost to secondary storage 

C. Memory usage cost

D. Communication cost

Explanation: Access cost refers to the time and resources needed to retrieve data from disk storage.

1 8 What is selection cardinality?

A. The number of columns selected

B. The number of rows returned after applying a condition 

C. The number of blocks in a file

D. The size of the index

Explanation: Selection cardinality is calculated as selectivity × number of records, representing the filtered result size.

1 9 Which optimizer type is being phased out in Oracle?

A. Cost-based optimizer

B. Rule-based optimizer 

C. Adaptive optimizer

D. Index-based optimizer

Explanation: Oracle is moving away from rule-based optimization in favor of cost-based strategies.

2 0 What does the blocking factor (bfr) represent?

A. Number of blocks per table

B. Number of records per block 

C. Number of indexes per column

D. Number of joins per query

Explanation: Blocking factor indicates how many records fit into one block, affecting storage and access cost.

2 1 Which evaluation strategy stores intermediate results as temporary relations? A. Pipelined evaluation

B. Materialized evaluation 

C. Indexed evaluation

D. Cached evaluation

Explanation: Materialized evaluation saves intermediate results, which can be reused or accessed later.

2 2 What is the advantage of pipelined evaluation?

A. It uses more memory

B. It delays execution

C. It avoids storing intermediate results 

D. It increases access cost

Explanation: Pipelined evaluation passes results directly to the next operation, improving performance.

2  Which catalog attribute helps estimate the size of a file?

A. Number of joins

B. Record size 

C. Index depth

D. Query length

Explanation: Record size helps determine how much space each row takes, contributing to total file size.

2  What does the number of distinct values (d) in an attribute affect?

A. Join order

B. Selectivity estimation 

C. Index depth

D. Query syntax

Explanation: The number of distinct values helps estimate how selective a condition is on that attribute.

2  Which transformation rule allows SELECT to be applied before JOIN if the condition only involves one relation?

A. Commutativity of SELECT

B. Commuting SELECT with JOIN 

C. Cascade of PROJECT

D. Associativity of UNION

Explanation: SELECT can be pushed before JOIN if the condition only references one of the joined tables.

2  What is the result of applying SELECT to both relations before UNION?

A. Increased result size

B. Reduced filtering

C. Efficient filtering before combining 

D. Removal of duplicates

Explanation: Applying SELECT early filters each relation before combining them, improving performance.

2  Which cost component is most affected by network latency in distributed databases? A. Storage cost

B. Communication cost 

C. Computation cost

D. Access cost

Explanation: Communication cost includes data transfer delays between systems, especially in distributed setups.

2 8 What does the number of index levels (x) indicate?

- A. The number of columns in a table
- B. The depth of a multilevel index 
- C. The number of rows in a block
- D. The number of joins in a query

Explanation: Index levels reflect how deep the index structure is, affecting lookup time.

2 9 Which operation is NOT commutative in relational algebra?

- A. UNION
- B. INTERSECTION
- C. DIFFERENCE 
- D. JOIN

Explanation: Unlike UNION and INTERSECTION, DIFFERENCE is not commutative; order matters.

3 0 What is the effect of applying the most restrictive SELECT condition first? A. It increases intermediate result size

- B. It simplifies the query syntax
- C. It reduces the number of rows early 
- D. It delays execution

Explanation: Applying restrictive filters first minimizes the data processed in later operations.

3 1 What does the cost function in query optimization help determine?

- A. The SQL syntax of the query
- B. The number of tables in the database
- C. The total estimated cost of executing a query 
- D. The number of attributes in a table

Explanation: The cost function aggregates resource usage to estimate the total cost of executing a query.

3 2 Which operation is associative in relational algebra?

- A. SELECT
- B. PROJECT
- C. JOIN 
- D. DIFFERENCE

Explanation: JOIN is associative, meaning the grouping of operations can be changed without affecting the result.

3 3 What is the purpose of using statistics in cost-based optimization?

- A. To validate query syntax
- B. To estimate resource usage and execution cost 
- C. To simplify SQL statements
- D. To remove redundant joins

Explanation: Statistics help the optimizer estimate how much data will be processed, guiding efficient execution plans.

3 4 Which transformation rule allows PROJECT to be applied before UNION? A. Commutativity of SELECT

B. Cascade of PROJECT

C. PROJECT with UNION 

D. SELECT with JOIN

Explanation: PROJECT can be pushed before UNION to reduce the number of attributes processed.

3 5 What does the number of blocks (b) in a file affect?

A. The number of joins

B. The access cost to storage 

C. The selectivity of conditions

D. The syntax of the query

Explanation: More blocks mean more disk I/O, increasing access cost.

3 6 Which optimizer type uses fixed rules without considering data statistics?

A. Cost-based optimizer

B. Rule-based optimizer 

C. Adaptive optimizer

D. Index-based optimizer

Explanation: Rule-based optimizers rely on predefined rules and ignore actual data statistics.

3 7 What does materialized evaluation require?

A. Storing intermediate results as temporary relations 

B. Forwarding results directly

C. Ignoring SELECT operations

D. Removing indexes

Explanation: Materialized evaluation stores results between operations, which can be reused or accessed later.

3 8 Which transformation improves performance by reducing data size before JOIN? **A. Applying PROJECT after JOIN**

B. Applying SELECT before JOIN 

C. Applying UNION before SELECT

D. Applying DIFFERENCE before PROJECT

Explanation: SELECT filters rows early, reducing the amount of data processed in JOIN.

3 9 What is the effect of pushing PROJECT operations down the query tree?

A. It increases the number of attributes

B. It reduces the number of attributes early 

C. It delays execution

D. It simplifies the WHERE clause

Explanation: PROJECT limits the columns involved, reducing intermediate result size.

4  What does the optimizer use to compare execution strategies?

A. SQL syntax

B. Estimated cost based on resource usage 

C. Table names

D. Query length

Explanation: The optimizer evaluates resource usage like CPU, I/O, and memory to compare strategies.

4  Which transformation replaces SELECT after Cartesian product with JOIN? A. SELECT with UNION

B. SELECT with PROJECT

C. SELECT with JOIN 

D. SELECT with INTERSECTION

Explanation: If the SELECT condition matches a join condition, it can be replaced with a JOIN for efficiency.

4  What does the number of first-level index blocks (b11) help estimate?

A. Query syntax complexity

B. Index access cost 

C. Number of joins

D. Memory usage

Explanation: b11 affects how quickly the index can be accessed, influencing total query cost.

4  Which operation is NOT associative in relational algebra?

A. UNION

B. INTERSECTION

C. DIFFERENCE 

D. JOIN

Explanation: DIFFERENCE is not associative; changing the grouping can affect the result.

4  What does the optimizer do with multiple execution strategies?

A. Chooses the one with the highest cost

B. Chooses randomly

C. Chooses the one with the lowest estimated cost 

D. Ignores all strategies

Explanation: The optimizer selects the strategy that minimizes resource usage and execution time.

4 **5** What does selectivity depend on?

A. The number of joins

B. The number of distinct values in an attribute 

C. The number of blocks

D. The query syntax

Explanation: Selectivity is influenced by how many unique values an attribute has, affecting how many rows match a condition.

4 **6** What does the optimizer use to estimate how many rows will be returned by a condition? A. Index depth

B. Selectivity 

C. Record size

D. Query length

Explanation: Selectivity measures the fraction of rows that satisfy a condition, helping estimate result size.

4 **7** Which transformation rule allows SELECT to be applied to each relation before performing a set operation?

A. SELECT with JOIN

B. SELECT with UNION 

C. SELECT with PROJECT

D. SELECT with DIFFERENCE

Explanation: SELECT can be pushed before UNION, INTERSECTION, or DIFFERENCE to filter data early.

4 8 What does the optimizer consider when choosing between multiple access paths? A. Query syntax

B. Estimated cost of each path 

C. Table names

D. SQL keywords

Explanation: The optimizer compares access paths based on estimated resource usage to choose the most efficient one.

4 9 Which catalog attribute affects how many blocks are needed to store a file? A. Blocking factor 

B. Selectivity

C. Index level

D. Join condition

Explanation: Blocking factor determines how many records fit in a block, influencing total block count.

5 0 What is the effect of applying PROJECT before JOIN when possible?

A. It increases the number of attributes

B. It reduces the number of attributes processed in JOIN 

C. It delays execution

D. It removes filtering

Explanation: PROJECT limits the columns involved, reducing the data size passed to JOIN operations.

5 1 Which optimizer type uses statistics to evaluate execution plans?

A. Rule-based optimizer

B. Cost-based optimizer 

C. Syntax-based optimizer

D. Tree-based optimizer

Explanation: Cost-based optimizers rely on statistics like table size and selectivity to estimate execution cost.

5 2 What does the number of distinct values in an attribute help estimate?

A. Index depth

B. Selectivity 

C. Join order

D. Query complexity

Explanation: More distinct values usually mean higher selectivity, affecting how many rows match a condition.

5 3 What is the purpose of reordering leaf nodes in a query tree?

A. To simplify SQL syntax

B. To apply restrictive operations earlier 

C. To increase result size

D. To remove redundant joins

Explanation: Reordering leaf nodes helps apply filters and joins that reduce data size early in the execution plan.

5  Which evaluation strategy is more memory-efficient in streaming queries?

A. Materialized evaluation

B. Pipelined evaluation 

C. Indexed evaluation

D. Cached evaluation

Explanation: Pipelined evaluation avoids storing intermediate results, making it suitable for streaming and memory-sensitive tasks.

5  What does the optimizer do when multiple execution strategies are available? **A. Chooses the one with the highest cost**

B. Chooses randomly

C. Chooses the one with the lowest estimated cost 

D. Ignores all strategies

Explanation: The optimizer selects the strategy that minimizes resource usage and execution time.

5 **6** Which transformation rule allows SELECT and PROJECT to be reordered under certain conditions? A. Commutativity of JOIN

B. SELECT with PROJECT 

C. PROJECT with UNION

D. SELECT with INTERSECTION

Explanation: SELECT and PROJECT can be reordered if the selection condition uses only projected attributes.

5 **7** What does the optimizer use to estimate I/O cost?

A. Number of joins

B. Number of blocks and access paths 

C. Query syntax

D. Index depth

Explanation: I/O cost depends on how many blocks need to be read and which access paths are used.

5 **8** What is the effect of applying SELECT to each relation before JOIN when conditions are split? A. It increases result size

B. It reduces data size before joining 

C. It delays execution

D. It removes filtering

Explanation: Applying SELECT to each relation individually filters data early, improving join efficiency.

5  Which transformation rule allows JOIN to be reordered without affecting results? A. Cascade of SELECT

B. Commutativity of JOIN 

C. SELECT with UNION

D. PROJECT with INTERSECTION

Explanation: JOIN is commutative, so the order of relations can be swapped without changing the result.

6  What does selection cardinality represent?

A. The number of columns selected

B. The number of rows returned after filtering 

C. The number of blocks in a file

D. The size of the index

Explanation: Selection cardinality is the expected number of rows that satisfy a condition, calculated as $\text{selectivity} \times \text{number of records}$.

.....

 Informal Design Guidelines (Q1–12)

1. What should each tuple in a relation represent?

- o A combination of entities**
- o One entity or relationship instance**
- o A set of attributes**
- o A foreign key reference**

Answer: One entity or relationship instance

Explanation: Each tuple should represent a single, meaningful unit to avoid confusion and redundancy.

2. Which of the following violates good relational design?

- o Using foreign keys**
- o Mixing attributes of EMPLOYEES and PROJECTs**
- o Separating entity and relationship attributes**
- o Keeping tuples atomic**

Answer: Mixing attributes of EMPLOYEES and PROJECTs

Explanation: Mixing unrelated entities in one relation leads to poor design and anomalies. 3. What is a common cause of update anomalies?

- o Using primary keys**
- o Redundant data across tuples**
- o Atomic attributes**

o Normalized relations

Answer: Redundant data across tuples

Explanation: When the same data is repeated, updates must be made in multiple places. 4. Which anomaly prevents inserting a project without an employee?

o Update anomaly

o Insert anomaly

o Delete anomaly

o Join anomaly

Answer: Insert anomaly

Explanation: Insert anomalies occur when required data is missing for insertion. 5. What is the result of deleting a project with employees?

o Insert anomaly

o Update anomaly

o Delete anomaly

o Redundancy

Answer: Delete anomaly

Explanation: Deleting a project may unintentionally remove all associated employees. 6. Which guideline helps avoid anomalies?

o Use composite attributes

- o Design schemas with minimal NULLs**
- o Design schemas that avoid insert, update, and delete anomalies**
- o Store all data in one relation**

Answer: Design schemas that avoid insert, update, and delete anomalies

Explanation: This ensures data integrity and reduces maintenance issues.

7. What should be done if anomalies are present?

- o Ignore them**
- o Document them for application handling**
- o Merge all relations**
- o Remove foreign keys**

Answer: Document them for application handling

Explanation: If anomalies can't be avoided, they should be noted for proper handling. 8. Which of the following is NOT a reason for NULL values?

- o Attribute not applicable**
- o Attribute value unknown**
- o Attribute value confidential**
- o Attribute value unavailable**

Answer: Attribute value confidential

Explanation: Confidentiality is not a technical reason for NULLs in design. 9. How should frequently NULL attributes be handled?

- o Deleted**
- o Moved to a separate relation**
- o Replaced with default values**
- o Ignored**

Answer: Moved to a separate relation

Explanation: This reduces NULLs and improves schema clarity.

10. What is the goal of minimizing NULLs?

- o Increase redundancy**
- o Improve query performance and integrity**
- o Allow nested relations**
- o Simplify joins**

Answer: Improve query performance and integrity

Explanation: Fewer NULLs lead to cleaner queries and better data quality. 11. Which of the following is a poor design choice?

- o Using atomic attributes**
- o Including many optional attributes in one relation**
- o Separating entities**
- o Using foreign keys**

Answer: Including many optional attributes in one relation

Explanation: This increases the chance of NULLs and violates design guidelines. **12. What is the bottom line of informal design guidelines?**

- o Use composite keys**
- o Avoid normalization**
- o Keep relations clean and focused**
- o Store all data in one table**

Answer: Keep relations clean and focused

Explanation: Good design avoids mixing entities and ensures clarity.

** Functional Dependencies (Q13–30)**

13. What does $X \rightarrow Y$ mean?

- o X and Y are equal**
- o X determines Y**
- o Y determines X**
- o X and Y are unrelated**

Answer: X determines Y

Explanation: Functional dependency means X uniquely determines Y. **14. Which is a valid FD?**

- o $ENAME \rightarrow SSN$**
- o $SSN \rightarrow ENAME$**

o $\text{HOURS} \rightarrow \{\text{SSN}, \text{PNUMBER}\}$

o $\text{PNAME} \rightarrow \text{PNUMBER}$

Answer: $\text{SSN} \rightarrow \text{ENAME}$

Explanation: SSN uniquely identifies the employee name.

15. What must hold for $X \rightarrow Y$ to be valid?

o X must be a foreign key

o Y must be a primary key

o Tuples with same X must have same Y

o X and Y must be composite

Answer: Tuples with same X must have same Y

Explanation: This is the definition of functional dependency.

16. What do FDs help evaluate?

o Query speed

o Schema quality

o UI design

o Indexing

Answer: Schema quality

Explanation: FDs help assess the correctness and efficiency of schema design. 17. If K is a key, what does $K \rightarrow \text{all attributes}$ mean?

o K is a foreign key

- o K is a superkey
- o K is a candidate key
- o K is composite

Answer: K is a superkey

Explanation: A superkey uniquely identifies all attributes in a relation. 18. Which FD is partial?

- o $\{SSN, PNUMBER\} \rightarrow HOURS$
- o $\{SSN, PNUMBER\} \rightarrow ENAME$
- o $SSN \rightarrow ENAME$
- o $PNUMBER \rightarrow PNAME$

Answer: $\{SSN, PNUMBER\} \rightarrow ENAME$

Explanation: ENAME depends only on SSN, not the full composite key. 19. What is a full functional dependency?

- o Y depends on part of X
- o Y depends on all of X
- o Y is a candidate key
- o Y is a foreign key

Answer: Y depends on all of X

Explanation: Removing any part of X breaks the dependency.

20. Which FD is transitive?

- o $SSN \rightarrow ENAME$

o $SSN \rightarrow DNUMBER \rightarrow DMGRSSN$

o $PNUMBER \rightarrow PNAME$

o $SSN \rightarrow HOURS$

Answer: $SSN \rightarrow DNUMBER \rightarrow DMGRSSN$

Explanation: SSN indirectly determines DMGRSSN via DNUMBER.

21. What does it mean if $X \rightarrow Y$ is a full functional dependency?

- X determines Y only partially
- Y depends on all of X
- Y is a candidate key
- X is a foreign key

☒ Answer: Y depends on all of X

?? Explanation: Full dependency means removing any part of X breaks the dependency.

22. Which of the following is a partial dependency?

- $\{SSN, PNUMBER\} \rightarrow HOURS$
- $\{SSN, PNUMBER\} \rightarrow ENAME$
- $SSN \rightarrow ENAME$
- $PNUMBER \rightarrow PNAME$

☒ Answer: $\{SSN, PNUMBER\} \rightarrow ENAME$

?? Explanation: ENAME depends only on SSN, not the full composite key.

23. What condition must be met for a relation to be in Second Normal Form (2NF)?

- All attributes must be atomic
- All non-prime attributes must be fully dependent on the whole primary key
- All attributes must be candidate keys
- All attributes must be foreign keys

✓ Answer: All non-prime attributes must be fully dependent on the whole primary key ?? Explanation: This removes partial dependencies and ensures proper normalization.

24. Which of the following violates Second Normal Form?

- Full dependency
- Partial dependency
- Atomic attributes
- Composite keys

✓ Answer: Partial dependency

?? Explanation: 2NF eliminates partial dependencies on part of the primary key.

25. What is a prime attribute?

- An attribute that is always NULL

- An attribute that is part of a candidate key
- An attribute that is foreign
- An attribute that is composite

 **Answer: An attribute that is part of a candidate key**

 **Explanation: Prime attributes are those included in any candidate key.**

26. What is a non-prime attribute?

- An attribute not part of any candidate key
- An attribute that is always NULL
- An attribute that is a foreign key
- An attribute that is composite

 **Answer: An attribute not part of any candidate key**

 **Explanation: Non-prime attributes are not involved in any candidate key.**

27. Which of the following is true about full functional dependency?

- Y depends on part of X
- Y depends on all of X
- Y is a candidate key
- Y is a foreign key

 **Answer: Y depends on all of X**

?? Explanation: Full dependency means the entire set X is needed to determine Y.

28. What is the purpose of identifying full vs partial dependencies?

- To define foreign keys
- To determine normalization level
- To create indexes
- To simplify user interfaces

✅ Answer: To determine normalization level

?? Explanation: Full and partial dependencies help decide if a relation is in 2NF.

29. Which of the following is NOT a valid reason to normalize a relation?

- To reduce redundancy
- To improve data integrity
- To increase anomalies
- To simplify updates

✅ Answer: To increase anomalies

?? Explanation: Normalization reduces anomalies, not increases them.

30. What does Second Normal Form (2NF) build upon?

- First Normal Form
- Third Normal Form
- Boyce-Codd Normal Form
- Denormalization

 **Answer: First Normal Form**

 **Explanation: 2NF starts from 1NF and adds the condition of full dependency on the whole key.**

31. Which of the following is disallowed in First Normal Form?

- Atomic attributes
- Composite attributes
- Primary keys
- Foreign keys

Answer: Composite attributes

Explanation: 1NF requires each attribute to hold a single atomic value, not a structured or composite one.

32. What is a multivalued attribute?

- An attribute with multiple meanings
- An attribute that holds multiple values in one cell
- An attribute that is a foreign key
- An attribute that is part of a composite key

Answer: An attribute that holds multiple values in one cell

Explanation: Multivalued attributes violate 1NF by storing multiple values in a single field. 33. Which normal form removes partial dependencies?

- First Normal Form
- Second Normal Form
- Third Normal Form
- Boyce-Codd Normal Form

Answer: Second Normal Form

Explanation: 2NF eliminates partial dependencies on part of a composite primary key.

34. Which of the following is a requirement for 2NF?

- All attributes must be atomic
- All non-prime attributes must be fully dependent on the whole primary key
- All attributes must be candidate keys
- All attributes must be foreign keys

Answer: All non-prime attributes must be fully dependent on the whole primary key Explanation: This ensures no partial dependency exists.

35. What is a prime attribute?

- An attribute that is always NULL
- An attribute that is part of a candidate key
- An attribute that is foreign

- An attribute that is composite

Answer: An attribute that is part of a candidate key

Explanation: Prime attributes are those included in any candidate key. 36. What is a non-prime attribute?

- An attribute not part of any candidate key
- An attribute that is always NULL
- An attribute that is a foreign key
- An attribute that is composite

Answer: An attribute not part of any candidate key

Explanation: Non-prime attributes are not involved in any candidate key. 37. Which of the following violates 2NF?

- Full functional dependency
- Partial dependency
- Atomic attributes
- Composite keys

Answer: Partial dependency

Explanation: 2NF removes partial dependencies on part of the primary key. 38. What is a transitive dependency?

- A dependency between unrelated attributes
- A dependency that goes through another attribute
- A dependency on a foreign key

- A dependency on a composite key

Answer: A dependency that goes through another attribute

Explanation: Transitive dependency occurs when $X \rightarrow Y$ and $Y \rightarrow Z$ implies $X \rightarrow Z$.

39. Which normal form removes transitive dependencies?

- First Normal Form
- Second Normal Form
- Third Normal Form
- Fourth Normal Form

Answer: Third Normal Form

Explanation: 3NF eliminates transitive dependencies on non-prime attributes. **40. Which of the following is NOT required for 3NF?**

- Relation must be in 2NF
- No transitive dependency on non-prime attributes
- All attributes must be candidate keys
- All non-prime attributes must depend only on the key

Answer: All attributes must be candidate keys

Explanation: Only non-prime attributes must follow the dependency rules. **41. What is the condition for a relation to be in 1NF?**

- All attributes depend on the whole key
- All attributes are atomic
- All attributes are foreign keys
- All attributes are composite

Answer: All attributes are atomic

Explanation: 1NF requires atomicity of attribute values.

42. What does 2NF add beyond 1NF?

- Removal of transitive dependencies
- Removal of partial dependencies
- Use of composite keys
- Use of nested relations

Answer: Removal of partial dependencies

Explanation: 2NF builds on 1NF by eliminating partial dependencies. 43. What does 3NF add beyond 2NF?

- Removal of multivalued attributes
- Removal of composite attributes
- Removal of transitive dependencies
- Removal of foreign keys

Answer: Removal of transitive dependencies

Explanation: 3NF ensures non-prime attributes are not transitively dependent on the key.

44. Which of the following is a valid candidate key?

- A set of attributes that uniquely identifies tuples
- A foreign key
- A composite attribute
- A multivalued attribute

Answer: A set of attributes that uniquely identifies tuples

Explanation: Candidate keys are minimal sets that uniquely identify rows. 45. What distinguishes a key from a superkey?

- A key is a minimal superkey
- A key must be composite
- A key must be foreign
- A key must be atomic

Answer: A key is a minimal superkey

Explanation: Removing any attribute from a key makes it no longer a superkey. 46. Which of the following is true about denormalization?

- It increases redundancy
- It removes anomalies
- It enforces strict normal forms
- It simplifies schema design

Answer: It increases redundancy

Explanation: Denormalization trades off redundancy for performance or simplicity. 47. Why might a designer choose denormalization?

- To reduce anomalies
- To improve query performance
- To enforce 3NF
- To eliminate foreign keys

Answer: To improve query performance

Explanation: Denormalization can reduce join operations and speed up queries. 48. Which normal form is considered sufficient for most practical designs?

- 1NF
- 2NF
- 3NF
- 5NF

Answer: 3NF

Explanation: 3NF is commonly used as a balance between normalization and performance.

49. What is the role of functional dependencies in normalization?

- They define foreign keys
- They determine normal forms

- They create indexes
- They define user interfaces

Answer: They determine normal forms

Explanation: FDs are used to assess and enforce normalization rules. 50. Which of the following is NOT a normal form?

- First Normal Form
- Second Normal Form
- Fourth Normal Form
- Redundancy Normal Form

Answer: Redundancy Normal Form

Explanation: There is no such thing as Redundancy Normal Form. 51. What does atomic mean in the context of 1NF?

- The attribute is encrypted
- The attribute holds a single value
- The attribute is a key
- The attribute is composite

Answer: The attribute holds a single value

Explanation: Atomicity means no multi-part or nested values. 52. Which of the following is a violation of 3NF?

- A non-prime attribute transitively dependent on the key
- A prime attribute dependent on a candidate key

- A foreign key referencing another table
- A composite key used in a relation

Answer: A non-prime attribute transitively dependent on the key

Explanation: 3NF prohibits transitive dependencies on non-prime attributes.

53. What is the minimum requirement for a relation to be in 2NF?

- It must be in 1NF
- It must have no transitive dependencies
- It must have composite keys
- It must have foreign keys

Answer: It must be in 1NF

Explanation: 2NF builds on 1NF by removing partial dependencies.

54. Which of the following is true about candidate keys?

- They are always composite
- They are chosen randomly
- They uniquely identify tuples
- They must be foreign keys

Answer: They uniquely identify tuples

Explanation: Candidate keys are minimal sets that ensure

uniqueness. **55. Which of the following is NOT a benefit of normalization?**

- Reduces redundancy
- Improves data integrity
- Simplifies joins
- Increases anomalies

Answer: Increases anomalies

Explanation: Normalization reduces anomalies, not increases them. 56. What is the purpose of a primary key?

- To allow NULLs
- To uniquely identify tuples
- To store multiple values
- To reference other tables

Answer: To uniquely identify tuples

Explanation: Primary keys ensure each row is unique.

57. Which of the following is a valid superkey?

- Any set of attributes that uniquely identifies tuples
- Only the primary key
- Only foreign keys
- Only atomic attributes

Answer: Any set of attributes that uniquely identifies tuples

Explanation: Superkeys may include extra attributes beyond the minimal key.

58. What is the difference between a candidate key and a primary key?

- Candidate keys are chosen randomly
- Primary key is the selected candidate key
- Candidate keys are always foreign
- Primary keys allow duplicates

Answer: Primary key is the selected candidate key

Explanation: One candidate key is chosen to be the primary key.

59. Which of the following is true about normalization?

- It always improves performance
- It always reduces joins
- It improves schema quality
- It increases redundancy

Answer: It improves schema quality

Explanation: Normalization improves structure and reduces anomalies.

60. What is the informal definition of 3NF?

- All attributes depend on the key
- All attributes depend on the whole key
- All attributes depend on nothing but the key
- All attributes are foreign keys

Answer: All attributes depend on nothing but the key

Explanation: 3NF ensures no transitive dependency; attributes depend directly on the key.

.....

✓ Informal Design Guidelines (Q1–12)

1. What should each tuple in a relation represent?

- o A combination of entities**
- o One entity or relationship instance**
- o A set of attributes**
- o A foreign key reference**

Answer: One entity or relationship instance

Explanation: Each tuple should represent a single, meaningful unit to avoid confusion and redundancy.

2. Which of the following violates good relational design?

- o Using foreign keys**
- o Mixing attributes of EMPLOYEES and PROJECTs**
- o Separating entity and relationship attributes**

o Keeping tuples atomic

Answer: Mixing attributes of EMPLOYEES and PROJECTs

Explanation: Mixing unrelated entities in one relation leads to poor design and anomalies. 3. What is a common cause of update anomalies?

o Using primary keys

o Redundant data across tuples

o Atomic attributes

o Normalized relations

Answer: Redundant data across tuples

Explanation: When the same data is repeated, updates must be made in multiple places. 4. Which anomaly prevents inserting a project without an employee?

o Update anomaly

o Insert anomaly

o Delete anomaly

o Join anomaly

Answer: Insert anomaly

Explanation: Insert anomalies occur when required data is missing for insertion. 5. What is the result of deleting a project with employees?

o Insert anomaly

- o Update anomaly**
- o Delete anomaly**
- o Redundancy**

Answer: Delete anomaly

Explanation: Deleting a project may unintentionally remove all associated employees. 6. Which guideline helps avoid anomalies?

- o Use composite attributes**
- o Design schemas with minimal NULLs**
- o Design schemas that avoid insert, update, and delete anomalies**
- o Store all data in one relation**

Answer: Design schemas that avoid insert, update, and delete anomalies

Explanation: This ensures data integrity and reduces maintenance issues.

7. What should be done if anomalies are present?

- o Ignore them**
- o Document them for application handling**
- o Merge all relations**
- o Remove foreign keys**

Answer: Document them for application handling

Explanation: If anomalies can't be avoided, they should be noted for proper handling. 8. Which of the following is NOT a reason for NULL values?

- o Attribute not applicable**
- o Attribute value unknown**
- o Attribute value confidential**
- o Attribute value unavailable**

Answer: Attribute value confidential

Explanation: Confidentiality is not a technical reason for NULLs in design. 9. How should frequently NULL attributes be handled?

- o Deleted**
- o Moved to a separate relation**
- o Replaced with default values**
- o Ignored**

Answer: Moved to a separate relation

Explanation: This reduces NULLs and improves schema clarity.

10. What is the goal of minimizing NULLs?

- o Increase redundancy**
- o Improve query performance and integrity**
- o Allow nested relations**
- o Simplify joins**

Answer: Improve query performance and integrity

Explanation: Fewer NULLs lead to cleaner queries and better data quality. 11. Which of the following is a poor design choice?

- o Using atomic attributes**
- o Including many optional attributes in one relation**
- o Separating entities**
- o Using foreign keys**

Answer: Including many optional attributes in one relation

Explanation: This increases the chance of NULLs and violates design guidelines. 12. What is the bottom line of informal design guidelines?

- o Use composite keys**
- o Avoid normalization**
- o Keep relations clean and focused**
- o Store all data in one table**

Answer: Keep relations clean and focused

Explanation: Good design avoids mixing entities and ensures clarity.

Functional Dependencies (Q13–30)

13. What does $X \rightarrow Y$ mean?

- o X and Y are equal**

- o X determines Y
- o Y determines X
- o X and Y are unrelated

Answer: X determines Y

Explanation: Functional dependency means X uniquely determines Y. 14. Which is a valid FD?

- o $ENAME \rightarrow SSN$
- o $SSN \rightarrow ENAME$
- o $HOURS \rightarrow \{SSN, PNUMBER\}$
- o $PNAME \rightarrow PNUMBER$

Answer: $SSN \rightarrow ENAME$

Explanation: SSN uniquely identifies the employee name.

15. What must hold for $X \rightarrow Y$ to be valid?

- o X must be a foreign key
- o Y must be a primary key
- o Tuples with same X must have same Y
- o X and Y must be composite

Answer: Tuples with same X must have same Y

Explanation: This is the definition of functional dependency.

16. What do FDs help evaluate?

- o Query speed
- o Schema quality
- o UI design
- o Indexing

Answer: Schema quality

Explanation: FDs help assess the correctness and efficiency of schema design. 17. If K is a key, what does $K \rightarrow \text{all attributes}$ mean?

- o K is a foreign key
- o K is a superkey
- o K is a candidate key
- o K is composite

Answer: K is a superkey

Explanation: A superkey uniquely identifies all attributes in a relation. 18. Which FD is partial?

- o $\{\text{SSN}, \text{PNUMBER}\} \rightarrow \text{HOURS}$
- o $\{\text{SSN}, \text{PNUMBER}\} \rightarrow \text{ENAME}$
- o $\text{SSN} \rightarrow \text{ENAME}$
- o $\text{PNUMBER} \rightarrow \text{PNAME}$

Answer: $\{\text{SSN}, \text{PNUMBER}\} \rightarrow \text{ENAME}$

Explanation: ENAME depends only on SSN, not the full composite key.

19. What is a full functional dependency?

- o Y depends on part of X**
- o Y depends on all of X**
- o Y is a candidate key**
- o Y is a foreign key**

Answer: Y depends on all of X

Explanation: Removing any part of X breaks the dependency.

20. Which FD is transitive?

- o SSN $\rightarrow$ ENAME**
- o SSN $\rightarrow$ DNUMBER $\rightarrow$ DMGRSSN**
- o PNUMBER $\rightarrow$ PNAME**
- o SSN $\rightarrow$ HOURS**

Answer: SSN $\rightarrow$ DNUMBER $\rightarrow$ DMGRSSN

Explanation: SSN indirectly determines DMGRSSN via DNUMBER.

21. What does it mean if $X \rightarrow Y$ is a full functional dependency?

- X determines Y only partially**
- Y depends on all of X**
- Y is a candidate key**
- X is a foreign key**

✅ Answer: Y depends on all of X

?? Explanation: Full dependency means removing any part of X breaks the dependency.

22. Which of the following is a partial dependency?

- {SSN, PNUMBER} → HOURS
- {SSN, PNUMBER} → ENAME
- SSN → ENAME
- PNUMBER → PNAME

✅ Answer: {SSN, PNUMBER} → ENAME

?? Explanation: ENAME depends only on SSN, not the full composite key.

23. What condition must be met for a relation to be in Second Normal Form (2NF)?

- All attributes must be atomic
- All non-prime attributes must be fully dependent on the whole primary key
- All attributes must be candidate keys
- All attributes must be foreign keys

✅ Answer: All non-prime attributes must be fully dependent on the whole primary key ?? Explanation: This removes partial dependencies and ensures proper normalization.

24. Which of the following violates Second Normal Form?

- Full dependency
- Partial dependency
- Atomic attributes
- Composite keys

✓ Answer: Partial dependency

?? Explanation: 2NF eliminates partial dependencies on part of the primary key.

25. What is a prime attribute?

- An attribute that is always NULL
- An attribute that is part of a candidate key
- An attribute that is foreign
- An attribute that is composite

✓ Answer: An attribute that is part of a candidate key

?? Explanation: Prime attributes are those included in any candidate key.

26. What is a non-prime attribute?

- An attribute not part of any candidate key
- An attribute that is always NULL
- An attribute that is a foreign key
- An attribute that is composite

✓ Answer: An attribute not part of any candidate key

?? Explanation: Non-prime attributes are not involved in any candidate key.

27. Which of the following is true about full functional dependency?

- Y depends on part of X
- Y depends on all of X
- Y is a candidate key
- Y is a foreign key

✓ Answer: Y depends on all of X

?? Explanation: Full dependency means the entire set X is needed to determine Y.

28. What is the purpose of identifying full vs partial dependencies?

- To define foreign keys
- To determine normalization level
- To create indexes
- To simplify user interfaces

✓ Answer: To determine normalization level

?? Explanation: Full and partial dependencies help decide if a relation is in 2NF.

29. Which of the following is NOT a valid reason to normalize a relation?

- To reduce redundancy
- To improve data integrity
- To increase anomalies
- To simplify updates

 **Answer: To increase anomalies**

 **Explanation: Normalization reduces anomalies, not increases them.**

30. What does Second Normal Form (2NF) build upon?

- First Normal Form
- Third Normal Form
- Boyce-Codd Normal Form
- Denormalization

 **Answer: First Normal Form**

 **Explanation: 2NF starts from 1NF and adds the condition of full dependency on the whole key.**

31. Which of the following is disallowed in First Normal Form?

- Atomic attributes
- Composite attributes
- Primary keys

- Foreign keys

Answer: Composite attributes

Explanation: 1NF requires each attribute to hold a single atomic value, not a structured or composite one.

32. What is a multivalued attribute?

- An attribute with multiple meanings
- An attribute that holds multiple values in one cell
- An attribute that is a foreign key
- An attribute that is part of a composite key

Answer: An attribute that holds multiple values in one cell

Explanation: Multivalued attributes violate 1NF by storing multiple values in a single field. 33. Which normal form removes partial dependencies?

- First Normal Form
- Second Normal Form
- Third Normal Form
- Boyce-Codd Normal Form

Answer: Second Normal Form

Explanation: 2NF eliminates partial dependencies on part of a composite primary key.

34. Which of the following is a requirement for 2NF?

- All attributes must be atomic
- All non-prime attributes must be fully dependent on the whole primary key
- All attributes must be candidate keys
- All attributes must be foreign keys

Answer: All non-prime attributes must be fully dependent on the whole primary key
Explanation: This ensures no partial dependency exists.

35. What is a prime attribute?

- An attribute that is always NULL
- An attribute that is part of a candidate key
- An attribute that is foreign
- An attribute that is composite

Answer: An attribute that is part of a candidate key

Explanation: Prime attributes are those included in any candidate key.

36. What is a non-prime attribute?

- An attribute not part of any candidate key
- An attribute that is always NULL
- An attribute that is a foreign key
- An attribute that is composite

Answer: An attribute not part of any candidate key

Explanation: Non-prime attributes are not involved in any candidate key. 37. Which of the following violates 2NF?

- Full functional dependency
- Partial dependency
- Atomic attributes
- Composite keys

Answer: Partial dependency

Explanation: 2NF removes partial dependencies on part of the primary key. 38. What is a transitive dependency?

- A dependency between unrelated attributes
- A dependency that goes through another attribute
- A dependency on a foreign key
- A dependency on a composite key

Answer: A dependency that goes through another attribute

Explanation: Transitive dependency occurs when $X \rightarrow Y$ and $Y \rightarrow Z$ implies $X \rightarrow Z$.

39. Which normal form removes transitive dependencies?

- First Normal Form
- Second Normal Form
- Third Normal Form
- Fourth Normal Form

Answer: Third Normal Form

Explanation: 3NF eliminates transitive dependencies on non-prime attributes. 40. Which of the following is NOT required for 3NF?

- Relation must be in 2NF
- No transitive dependency on non-prime attributes
- All attributes must be candidate keys
- All non-prime attributes must depend only on the key

Answer: All attributes must be candidate keys

Explanation: Only non-prime attributes must follow the dependency rules. 41. What is the condition for a relation to be in 1NF?

- All attributes depend on the whole key
- All attributes are atomic
- All attributes are foreign keys
- All attributes are composite

Answer: All attributes are atomic

Explanation: 1NF requires atomicity of attribute values.

42. What does 2NF add beyond 1NF?

- Removal of transitive dependencies
- Removal of partial dependencies

- Use of composite keys
- Use of nested relations

Answer: Removal of partial dependencies

Explanation: 2NF builds on 1NF by eliminating partial dependencies. 43. What does 3NF add beyond 2NF?

- Removal of multivalued attributes
- Removal of composite attributes
- Removal of transitive dependencies
- Removal of foreign keys

Answer: Removal of transitive dependencies

Explanation: 3NF ensures non-prime attributes are not transitively dependent on the key.

44. Which of the following is a valid candidate key?

- A set of attributes that uniquely identifies tuples
- A foreign key
- A composite attribute
- A multivalued attribute

Answer: A set of attributes that uniquely identifies tuples

Explanation: Candidate keys are minimal sets that uniquely identify rows. 45. What distinguishes a key from a superkey?

- A key is a minimal superkey

- A key must be composite
- A key must be foreign
- A key must be atomic

Answer: A key is a minimal superkey

Explanation: Removing any attribute from a key makes it no longer a superkey. 46. Which of the following is true about denormalization?

- It increases redundancy
- It removes anomalies
- It enforces strict normal forms
- It simplifies schema design

Answer: It increases redundancy

Explanation: Denormalization trades off redundancy for performance or simplicity. 47. Why might a designer choose denormalization?

- To reduce anomalies
- To improve query performance
- To enforce 3NF
- To eliminate foreign keys

Answer: To improve query performance

Explanation: Denormalization can reduce join operations and speed up queries. 48. Which normal form is considered sufficient for most practical designs?

- 1NF
- 2NF
- 3NF
- 5NF

Answer: 3NF

Explanation: 3NF is commonly used as a balance between normalization and performance.

49. What is the role of functional dependencies in normalization?

- They define foreign keys
- They determine normal forms
- They create indexes
- They define user interfaces

Answer: They determine normal forms

Explanation: FDs are used to assess and enforce normalization rules. 50. Which of the following is NOT a normal form?

- First Normal Form
- Second Normal Form
- Fourth Normal Form

- Redundancy Normal Form

Answer: Redundancy Normal Form

Explanation: There is no such thing as Redundancy Normal Form.

51. What does atomic mean in the context of 1NF?

- The attribute is encrypted
- The attribute holds a single value
- The attribute is a key
- The attribute is composite

Answer: The attribute holds a single value

Explanation: Atomicity means no multi-part or nested values. 52.

Which of the following is a violation of 3NF?

- A non-prime attribute transitively dependent on the key
- A prime attribute dependent on a candidate key
- A foreign key referencing another table
- A composite key used in a relation

Answer: A non-prime attribute transitively dependent on the key

Explanation: 3NF prohibits transitive dependencies on non-prime attributes.

53. What is the minimum requirement for a relation to be in 2NF?

- It must be in 1NF
- It must have no transitive dependencies

- It must have composite keys
- It must have foreign keys

Answer: It must be in 1NF

Explanation: 2NF builds on 1NF by removing partial dependencies.

54. Which of the following is true about candidate keys?

- They are always composite
- They are chosen randomly
- They uniquely identify tuples
- They must be foreign keys

Answer: They uniquely identify tuples

Explanation: Candidate keys are minimal sets that ensure uniqueness. 55. Which of the following is NOT a benefit of normalization?

- Reduces redundancy
- Improves data integrity
- Simplifies joins
- Increases anomalies

Answer: Increases anomalies

Explanation: Normalization reduces anomalies, not increases them. 56. What is the purpose of a primary key?

- To allow NULLs

- To uniquely identify tuples
- To store multiple values
- To reference other tables

Answer: To uniquely identify tuples

Explanation: Primary keys ensure each row is unique.

57. Which of the following is a valid superkey?

- Any set of attributes that uniquely identifies tuples
- Only the primary key
- Only foreign keys
- Only atomic attributes

Answer: Any set of attributes that uniquely identifies tuples

Explanation: Superkeys may include extra attributes beyond the minimal key.

58. What is the difference between a candidate key and a primary key?

- Candidate keys are chosen randomly
- Primary key is the selected candidate key
- Candidate keys are always foreign
- Primary keys allow duplicates

Answer: Primary key is the selected candidate key

Explanation: One candidate key is chosen to be the primary key.

59. Which of the following is true about normalization?

- It always improves performance
- It always reduces joins
- It improves schema quality
- It increases redundancy

Answer: It improves schema quality

Explanation: Normalization improves structure and reduces anomalies.

60. What is the informal definition of 3NF?

- All attributes depend on the key
- All attributes depend on the whole key
- All attributes depend on nothing but the key
- All attributes are foreign keys

Answer: All attributes depend on nothing but the key

Explanation: 3NF ensures no transitive dependency; attributes depend directly on the key.

.....

Faculty of Computers and Artificial Intelligence Final Exam (1) – Advanced Databases

50 MCQs | Doctor-Level | Comprehensive

Department: Data Science	Total Marks: 50 Marks
Course Name: Advanced Database	Duration: 2 hours

Q1.

Which relational design practice should be avoided to ensure semantic clarity of relations?

- a. Using foreign keys to reference related entities**
- b. Separating entity attributes from relationship attributes**
- c. Mixing attributes of different entities in the same relation**
- d. Designing schemas that can be explained relation by relation**

Correct Answer: c

Explanation: Mixing attributes of multiple entities violates semantic clarity and leads to anomalies.

Q2.

Redundant data in a relation primarily leads to:

- a. Increased encryption overhead**
- b. Reduced disk I/O**

- c. Update, insertion, and deletion anomalies**
- d. Improved query performance**

Correct Answer: c

Explanation: Redundancy causes anomalies during insert, delete, and update operations.

Q3. (*Scenario-based*)

A project cannot be inserted into a table unless an employee is assigned to it. This situation illustrates:

- a. Deletion anomaly**
- b. Update anomaly**
- c. Insert anomaly**
- d. Functional dependency violation**

Correct Answer: c

Explanation: Insert anomaly occurs when certain data cannot be added without unrelated data.

Q4.

Which of the following is NOT a valid reason for NULL values in a relation?

- a. Attribute value exists but is unknown**
- b. Attribute is not applicable**

- c. Attribute value exists but is unavailable**
- d. Attribute requires special security clearance**

Correct Answer: d

Explanation: Security clearance is not a conceptual reason for NULL values.

Q5.

Functional dependencies are primarily used to:

- a. Improve storage allocation**
- b. Measure query execution cost**
- c. Define the correctness of relational designs**
- d. Encrypt database attributes**

Correct Answer: c

Explanation: FDs evaluate design quality and define normal forms.

Q6.

If $X \rightarrow Y$ holds, which statement is true?

- a. Y uniquely determines X**
- b. X determines a unique value of Y**
- c. X and Y must have the same number of attributes**
- d. X and Y must be candidate keys**

Correct Answer: b

Explanation: FD means identical X values imply identical Y values.

Q7.

Which concept is used alongside functional dependencies to define normal forms?

- a. Indexes**
- b. Triggers**
- c. Keys**
- d. Views**

Correct Answer: c

Explanation: Keys and FDs together define normal forms.

Q8.

If K is a key of relation R, then:

- a. K functionally determines some attributes**
- b. K is always a single attribute**
- c. K functionally determines all attributes in R**
- d. K must be a foreign key**

Correct Answer: c

Explanation: A key uniquely determines all attributes.

Q9.

Normalization primarily aims to:

- a. Increase redundancy for faster reads**
- b. Convert bad relations into acceptable ones**
- c. Eliminate indexes**
- d. Optimize only query execution time**

Correct Answer: b

Explanation: Normalization removes anomalies via decomposition.

Q10.

Which violates First Normal Form (1NF)?

- a. A column with atomic values**
- b. A column containing a list of phone numbers**
- c. A primary key column**
- d. A column with NULL values**

Correct Answer: b

Explanation: 1NF forbids multi-valued attributes.

Q11.

A relation is in Third Normal Form (3NF) if it:

- a. Is in 1NF only**
- b. Is in 2NF with no partial dependencies**

c. Is in 2NF with no transitive dependencies

d. Has no functional dependencies

Correct Answer: c

Explanation: 3NF removes transitive dependencies.

Q12.

Which normal form requires all non-key attributes to depend on the entire primary key?

a. 1NF

b. 2NF

c. 3NF

d. BCNF

Correct Answer: b

Explanation: 2NF eliminates partial dependency.

Q13.

Best approach to convert a relation with repeating groups into 1NF?

a. Add more attributes

b. Remove repeating groups entirely

c. Create a separate relation for repeating groups

d. Merge repeating groups with another table

Correct Answer: c

Explanation: Repeating groups must be decomposed.

Q14.

An index in database terminology is best described as:

- a. A primary key**
- b. A foreign key**
- c. A composite key**
- d. An access path**

Correct Answer: d

Explanation: Indexes provide alternative access paths.

Q15.

The main purpose of using indexes is to:

- a. Reduce physical database size**
- b. Create backup copies**
- c. Improve search efficiency**
- d. Enforce referential integrity**

Correct Answer: c

Explanation: Indexes speed up data retrieval.

Q16.

Which distinguishes a dense index from a sparse index?

- a. Dense index has fewer entries**
- b. Dense index has entries for every search key value**
- c. Dense index is only used on primary keys**
- d. Dense index is unordered**

Correct Answer: b

Explanation: Dense index contains one entry per record.

Q17.

A primary index is classified as:

- a. Dense**
- b. Sparse**
- c. Secondary**
- d. Alternate**

Correct Answer: b

Explanation: Primary index has one entry per block.

Q18.

A clustering index is defined on:

- a. A key field only**
- b. A non-ordering field**
- c. A non-key ordering field**

d. A foreign key

Correct Answer: c

Explanation: Clustering index orders records by a non-key field.

Q19.

Which statement about secondary indexes is correct?

- a. Only one secondary index can exist**
- b. Secondary index must be a candidate key**
- c. Secondary indexes are always sparse**
- d. A data file can have multiple secondary indexes**

Correct Answer: d

Explanation: Multiple secondary indexes are allowed.

Q20.

Secondary indexes are typically:

- a. Sparse**
- b. Dense**
- c. Multilevel only**
- d. Block-anchored**

Correct Answer: b

Explanation: Secondary indexes have entries for every record.

Q21.

Which query processing step checks SQL syntax?

- a. Scanning**
- b. Parsing**
- c. Validating**
- d. Optimizing**

Correct Answer: b

Explanation: Parsing ensures syntactic correctness.

Q22.

Which step verifies relation and attribute names?

- a. Scanning**
- b. Parsing**
- c. Validating**
- d. Code generation**

Correct Answer: c

Explanation: Validation checks semantics and schema correctness.

Q23.

Query optimization is best described as:

- a. Executing SQL code**

b. Choosing the most efficient execution plan

c. Parsing queries

d. Translating SQL to relational algebra

Correct Answer: b

Explanation: Optimization selects the best execution strategy.

Q24.

A query tree corresponds to:

a. Relational calculus

b. SQL syntax

c. Relational algebra

d. Physical storage

Correct Answer: c

Explanation: Query trees represent relational algebra expressions.

Q25.

Which relational algebra operation is NOT commutative?

a. Union

b. Intersection

c. Difference

d. Join

Correct Answer: c

Explanation: $A - B \neq B - A$.

Q26.

The main heuristic in query optimization is to:

- a. Perform joins first**
- b. Reduce intermediate result size early**
- c. Delay selections**
- d. Increase Cartesian products**

Correct Answer: b

Explanation: Smaller intermediates improve performance.

Q27.

Which optimizer uses database statistics?

- a. Rule-based optimizer**
- b. Heuristic optimizer**
- c. Cost-based optimizer**
- d. Syntax optimizer**

Correct Answer: c

Explanation: Cost-based optimization relies on statistics.

Q28.

Which is NOT a phase of query processing?

- a. Parsing**
- b. Execution**
- c. Fetching**
- d. Normalization**

Correct Answer: d

Explanation: Normalization is a design activity.

Q29.

A transaction is best defined as:

- a. A single SQL SELECT statement**
- b. A logical unit of work**
- c. A physical disk operation**
- d. A database schema**

Correct Answer: b

Explanation: Transactions group operations logically.

Q30.

Which ACID property ensures all-or-nothing execution?

- a. Consistency**
- b. Isolation**

c. Atomicity

d. Durability

Correct Answer: c

Explanation: Atomicity guarantees complete or abort.

Q31.

Which property ensures committed data is not lost?

a. Atomicity

b. Consistency

c. Isolation

d. Durability

Correct Answer: d

Explanation: Durability preserves committed changes.

Q32.

Which concurrency problem occurs when one update overwrites another?

a. Dirty read

b. Inconsistent retrieval

c. Lost update

d. Phantom read

Correct Answer: c

Explanation: Lost update occurs due to overwriting.

Q33.

Reading uncommitted data violates which ACID property?

- a. Atomicity**
- b. Consistency**
- c. Isolation**
- d. Durability**

Correct Answer: c

Explanation: Isolation prevents access to uncommitted data.

Q34.

The transaction log is mainly used for:

- a. Query optimization**
- b. Index creation**
- c. Database recovery**
- d. Schema validation**

Correct Answer: c

Explanation: Logs support rollback and recovery.

Q35.

Which lock granularity offers the highest concurrency?

- a. Database-level**
- b. Table-level**
- c. Page-level**
- d. Row-level**

Correct Answer: d

Explanation: Finer granularity increases concurrency.

Q36.

Which DBMS component manages locks?

- a. Scheduler**
- b. Optimizer**
- c. Lock manager**
- d. Listener**

Correct Answer: c

Explanation: Lock manager controls concurrency.

Q37.

Which NoSQL characteristic contrasts ACID?

- a. Strong consistency**
- b. BASE properties**
- c. Schema enforcement**

d. Vertical scaling

Correct Answer: b

Explanation: NoSQL often follows BASE model.

Q38.

Which is NOT a NoSQL database type?

a. Document

b. Key-value

c. Graph

d. Relational

Correct Answer: d

Explanation: Relational is not NoSQL.

Q39.

Scaling out primarily means:

a. Using a bigger server

b. Increasing RAM

c. Distributing data across servers

d. Vertical partitioning

Correct Answer: c

Explanation: Scaling out uses multiple servers.

Q40.

Sharding refers to:

- a. Replicating indexes**
- b. Splitting data across servers**
- c. Encrypting data**
- d. Vertical scaling**

Correct Answer: b

Explanation: Sharding distributes data horizontally.

Q41.

Document databases differ because:

- a. They require fixed schemas**
- b. Values must be atomic**
- c. Documents can have different structures**
- d. They enforce strict normalization**

Correct Answer: c

Explanation: Document DBs are schema-less.

Q42.

In document databases, relationships are typically:

- a. Strongly enforced**
- b. Fully normalized**

- c. Embedded or application-managed**
- d. Based on foreign keys**

Correct Answer: c

Explanation: Relationships are often embedded.

Q43.

Which database model is best suited for highly connected data?

- a. Relational**
- b. Document**
- c. Key-value**
- d. Graph**

Correct Answer: d

Explanation: Graph DBs handle relationships efficiently.

1 1

Q44.

Which is a disadvantage of scaling out?

- a. Higher cost**
- b. Less concurrency**
- c. Increased failure probability**
- d. Limited availability**

Correct Answer: c

Explanation: More nodes increase failure chances.

Q45.

Eventual consistency means:

- a. Data is always consistent**
- b. Data becomes consistent over time**
- c. No replication occurs**
- d. ACID is fully enforced**

Correct Answer: b

Explanation: Consistency is achieved eventually.

Q46.

Which NoSQL model stores data as <key, value> pairs?

- a. Graph**
- b. Column-family**
- c. Document**
- d. Key-value**

Correct Answer: d

Explanation: Key-value stores are simplest NoSQL type.

Q47.

Which DBMS process chooses execution plans?

- a. Scheduler**

- b. Optimizer**
- c. Listener**
- d. Lock manager**

Correct Answer: b

Explanation: Optimizer selects best access paths.

Q48.

Which cache stores compiled SQL plans?

- a. Data cache**
- b. Index cache**
- c. SQL cache**
- d. Disk cache**

Correct Answer: c

Explanation: SQL cache stores processed SQL.

Q49.

Minimizing I/O operations improves performance because:

- a. CPU is faster than RAM**
- b. Disk access is slower than memory**
- c. Network is faster than disk**
- d. Indexes eliminate I/O**

Correct Answer: b

Explanation: Disk I/O is the main bottleneck.

Q50.

Good database performance starts with:

- a. Index tuning**
- b. Query optimization**
- c. Good database design**
- d. Hardware upgrades**

Correct Answer: c

Explanation: Poor design cannot be fixed by tuning.

.....

Faculty of Computers and Artificial Intelligence Final Exam (2) – Advanced Databases

50 MCQs | Doctor-Level | Comprehensive

Department: Data Science	Total Marks: 50 Marks
Course Name: Advanced Database	Duration: 2 hours

Q1.

Which situation most strongly justifies decomposing a relation during normalization?

- a. Presence of foreign keys**
- b. Existence of functional dependencies**
- c. High query execution cost**
- d. Lossless but dependency-preserving decomposition**

Correct Answer: b

Explanation: Decomposition is driven by problematic functional dependencies causing anomalies.

Q2. (*Scenario-based*)

A relation R(A,B,C) has FDs: $A \rightarrow B$, $B \rightarrow C$. Which normal form does R violate?

- a. 1NF**

- b. 2NF**
- c. 3NF**
- d. BCNF**

Correct Answer: c

Explanation: $B \rightarrow C$ is a transitive dependency, violating 3NF.

Q3.

Which condition must always hold for a decomposition to be acceptable?

- a. It must increase the number of relations**
- b. It must eliminate all functional dependencies**
- c. It must be lossless**
- d. It must preserve all indexes**

Correct Answer: c

Explanation: Lossless decomposition guarantees no information loss.

1

Q4.

A relation is in BCNF if:

- a. It is in 3NF**
- b. Every determinant is a superkey**
- c. There are no transitive dependencies**

d. It has no foreign keys

Correct Answer: b

Explanation: BCNF is stricter than 3NF.

Q5.

Which trade-off is commonly introduced by aggressive normalization?

a. Reduced integrity

b. Increased redundancy

c. Increased join cost

d. Loss of atomicity

Correct Answer: c

Explanation: Normalization may increase joins, impacting performance.

Q6.

Why might a database designer intentionally denormalize a schema?

a. To violate ACID properties

b. To simplify integrity constraints

c. To reduce join operations

d. To remove functional dependencies

Correct Answer: c

Explanation: Denormalization is often used for performance optimization.

Q7.

Which index type is most sensitive to record insertions and deletions?

- a. Primary index**
- b. Clustering index**
- c. Dense secondary index**
- d. Sparse index**

Correct Answer: c

Explanation: Dense secondary indexes require frequent updates.

Q8.

A sparse index is only possible when:

- a. The data file is unordered**
- b. The index is multilevel**
- c. The data file is ordered on the search key**
- d. The index is secondary**

Correct Answer: c

Explanation: Sparse indexes rely on ordering in the data file.

Q9.

Which index minimizes disk I/O for range queries?

- a. Hash index**
- b. Dense secondary index**
- c. B+-tree index**
- d. Bitmap index**

Correct Answer: c

Explanation: B+-trees efficiently support range queries.

Q10.

Why are B+-trees preferred over B-trees in DBMSs?

- a. Smaller tree height**
- b. Faster insertion**
- c. All data pointers stored at leaf level**
- d. No need for rebalancing**

Correct Answer: c

Explanation: Leaf-level storage improves sequential access.

Q11.

Which factor most influences index blocking factor?

- a. Disk latency**
- b. Record size**
- c. Index entry size**

d. File organization

Correct Answer: c

Explanation: Blocking factor depends on entry size vs block size.

Q12. (*Calculation logic*)

Increasing index entry size will generally:

- a. Increase blocking factor**
- b. Reduce number of index blocks**
- c. Increase number of index blocks**
- d. Eliminate binary search**

Correct Answer: c

Explanation: Larger entries reduce entries per block.

Q13.

Which statement about clustering indexes is TRUE?

- a. They must be dense**
- b. Only one clustering index can exist per file**
- c. They require unique values**
- d. They cannot be sparse**

Correct Answer: b

Explanation: Only one physical ordering is possible.

Q14.

Secondary indexes differ from primary indexes because they:

- a. Are always sparse**
- b. Define physical record order**
- c. Do not affect file ordering**
- d. Require sorted files**

Correct Answer: c

Explanation: Secondary indexes provide alternate access paths.

Q15.

Which query processing phase transforms SQL into an internal form?

- a. Parsing**
- b. Validation**
- c. Execution**
- d. Fetching**

Correct Answer: a

Explanation: Parsing converts SQL into internal representations.

Q16.

Which task is performed after query optimization?

- a. Syntax checking**
- b. Access plan selection**
- c. Physical execution**
- d. Schema validation**

Correct Answer: c

Explanation: Execution follows optimization.

Q17.

Which optimization technique pushes selections as early as possible?

- a. Cost-based optimization**
- b. Heuristic optimization**
- c. Dynamic optimization**
- d. Rule disabling**

Correct Answer: b

Explanation: Heuristics aim to reduce intermediate results.

Q18.

Why are statistics critical for cost-based optimization?

- a. They define SQL syntax**
- b. They estimate resource usage**
- c. They enforce constraints**

d. They replace indexes

Correct Answer: b

Explanation: Cost estimates rely on statistics.

Q19.

Which statistic is MOST relevant for join cost estimation?

a. Number of attributes

b. Cardinality of relations

c. Attribute names

d. Data types

Correct Answer: b

Explanation: Cardinality directly affects join size.

Q20.

Dynamic query optimization differs from static optimization because it:

a. Occurs at compile time

b. Ignores statistics

c. Uses runtime information

d. Avoids indexes

Correct Answer: c

Explanation: Dynamic optimization adapts at execution time.

Q21.

Which operation typically dominates query execution cost?

- a. CPU arithmetic**
- b. Disk I/O**
- c. Memory allocation**
- d. Network latency**

Correct Answer: b

Explanation: Disk I/O is the primary bottleneck.

Q22.

A query tree primarily represents:

- a. Physical execution order**
- b. SQL grammar**
- c. Relational algebra expression**
- d. Disk access sequence**

Correct Answer: c

Explanation: Query trees model algebraic operations.

Q23.

Which transformation is always safe during algebraic optimization?

- a. Join commutativity**
- b. Selection pushdown**
- c. Cartesian product expansion**
- d. Difference reordering**

Correct Answer: b

Explanation: Selection pushdown preserves semantics.

Q24.

Which relational algebra operator is most expensive?

- a. Selection**
- b. Projection**
- c. Join**
- d. Rename**

Correct Answer: c

Explanation: Joins often generate large intermediates.

Q25.

Which ACID property is most challenged in distributed NoSQL systems?

- a. Atomicity**
- b. Consistency**

c. Isolation

d. Durability

Correct Answer: b

Explanation: Consistency is often relaxed (BASE).

Q26.

Which concurrency problem leads to inconsistent aggregate results?

a. Lost update

b. Dirty read

c. Inconsistent retrieval

d. Phantom write

Correct Answer: c

Explanation: Data changes during repeated reads.

Q27.

Which mechanism prevents dirty reads?

a. Timestamp ordering

b. Write-ahead logging

c. Strict locking

d. Indexing

Correct Answer: c

Explanation: Strict locking enforces isolation.

Q28.

What is the primary role of the scheduler?

- a. Optimize SQL syntax**
- b. Assign locks**
- c. Order concurrent transaction execution**
- d. Manage buffer cache**

Correct Answer: c

Explanation: Scheduler ensures serializability.

Q29.

Which lock granularity provides the lowest concurrency?

- a. Row-level**
- b. Page-level**
- c. Table-level**
- d. Database-level**

Correct Answer: d

Explanation: Database-level locking blocks all access.

Q30.

Why is write-ahead logging essential?

- a. Speeds up queries**
- b. Guarantees durability**
- c. Eliminates deadlocks**
- d. Reduces index size**

Correct Answer: b

Explanation: WAL ensures committed changes survive failures.

Q31.

Which failure recovery action rolls back uncommitted transactions?

- a. Forward recovery**
- b. Checkpointing**
- c. Undo operation**
- d. Redo operation**

Correct Answer: c

Explanation: Undo restores previous values.

Q32.

Which NoSQL feature enables horizontal scalability?

- a. Fixed schema**
- b. Vertical partitioning**

c. Sharding

d. Normalization

Correct Answer: c

Explanation: Sharding distributes data across nodes.

Q33.

Which NoSQL model best supports nested data?

a. Key-value

b. Graph

c. Document

d. Column-family

Correct Answer: c

Explanation: Document DBs support nested structures.

Q34.

Eventual consistency implies that:

a. Data is always correct

b. Replicas may diverge temporarily

c. Transactions are atomic

d. Isolation is guaranteed

Correct Answer: b

Explanation: Consistency is achieved over time.

Q35.

Which scenario favors NoSQL over RDBMS?

- a. Strong transactional consistency**
- b. Highly structured data**
- c. Massive distributed workloads**
- d. Complex joins**

Correct Answer: c

Explanation: NoSQL excels at scale and distribution.

Q36.

Which component stores recently accessed data blocks?

- a. SQL cache**
- b. Data cache**
- c. Log buffer**
- d. Index file**

Correct Answer: b

Explanation: Data cache minimizes disk access.

Q37.

Why is minimizing I/O central to performance tuning?

- a. CPU is slower than disk**

- b. Disk access dominates response time**
- c. Memory is unlimited**
- d. Network is irrelevant**

Correct Answer: b

Explanation: Disk I/O is orders of magnitude slower.

Q38.

Which tuning activity is client-side?

- a. Index creation**
- b. SQL rewriting**
- c. Buffer allocation**
- d. Lock scheduling**

Correct Answer: b

Explanation: SQL tuning is client-side.

Q39.

Which tuning activity is server-side?

- a. Query formulation**
- b. Schema design**
- c. Buffer cache configuration**
- d. Join ordering**

Correct Answer: c

Explanation: Memory and cache are server-side.

Q40.

Which metric best reflects query selectivity?

- a. Number of columns**
- b. Percentage of rows retrieved**
- c. Data type**
- d. Index depth**

Correct Answer: b

Explanation: Selectivity measures row filtering.

Q41.

Which index is least useful for high-cardinality attributes?

- a. B+-tree**
- b. Hash**
- c. Bitmap**
- d. Secondary index**

Correct Answer: c

Explanation: Bitmap indexes suit low cardinality.

Q42.

Which transformation reduces join input size?

- a. Projection before join**
- b. Join before selection**
- c. Cartesian product**
- d. Duplicate elimination**

Correct Answer: a

Explanation: Early projection reduces tuple width.

Q43.

Which operation benefits most from indexing?

- a. Full table scan**
- b. Equality selection**
- c. Cartesian product**
- d. Rename**

Correct Answer: b

Explanation: Indexes speed up equality predicates.

Q44.

Which scenario risks lost updates?

- a. Read-only transactions**
- b. Concurrent writes on same data**
- c. Serializable schedule**

d. Strict two-phase locking

Correct Answer: b

Explanation: Lost updates occur during concurrent writes.

Q45.

Which isolation level is weakest?

a. Serializable

b. Repeatable read

c. Read committed

d. Read uncommitted

Correct Answer: d

Explanation: Read uncommitted allows dirty reads.

Q46.

Which index structure supports logarithmic search time?

a. Linear index

b. Hash table

c. B+-tree

d. Bitmap

Correct Answer: c

Explanation: B+-tree height grows logarithmically.

Q47.

Which condition breaks dependency preservation?

- a. Lossless join**
- b. Incorrect decomposition**
- c. Excessive normalization**
- d. Foreign key usage**

Correct Answer: b

Explanation: Poor decomposition may lose FDs.

Q48.

Why might dependency preservation be sacrificed?

- a. To reduce storage**
- b. To achieve BCNF**
- c. To eliminate keys**
- d. To avoid joins**

Correct Answer: b

Explanation: BCNF may break dependency preservation.

Q49.

Which NoSQL database type models relationships explicitly?

- a. Document**
- b. Column-family**

c. Key-value

d. Graph

Correct Answer: d

Explanation: Graph DBs store nodes and edges.

Q50.

Final-level conclusion: the best overall DB design balances:

a. Normalization only

b. Performance only

c. Consistency only

d. Integrity, performance, and scalability

Correct Answer: d

Explanation: Real systems require trade-offs across all dimensions.

.....

Faculty of Computers and Artificial Intelligence Final Exam (3) – Advanced Databases

50 MCQs | Doctor-Level | Comprehensive

Department: Data Science	Total Marks: 50 Marks
Course Name: Advanced Database	Duration: 2 hours

Q1.

Which property guarantees that a database moves from one consistent state to another after a transaction?

- a. Atomicity**
- b. Isolation**
- c. Consistency**
- d. Durability**

Correct Answer: c

Explanation: Consistency ensures all integrity constraints remain satisfied.

Q2. (*Scenario-based*)

A transaction updates three tables. Two updates succeed, then the system crashes before COMMIT. Which mechanism restores consistency?

- a. Index rebuilding**

- b. Write-Ahead Logging**
- c. Query optimization**
- d. Buffer flushing**

Correct Answer: b

Explanation: WAL allows undo of uncommitted updates.

Q3.

Which concurrency problem occurs when a transaction reads different values for the same row during execution?

- a. Dirty read**
- b. Lost update**
- c. Inconsistent retrieval**
- d. Phantom read**

Correct Answer: c

Explanation: Data changes mid-transaction cause inconsistent reads.

Q4.

Which isolation level completely prevents phantom reads?

- a. Read committed**
- b. Repeatable read**
- c. Serializable**

d. Read uncommitted

Correct Answer: c

Explanation: Serializable enforces full isolation.

Q5.

Which transaction scheduling strategy ensures serializability by design?

a. First-Come-First-Served

b. Two-Phase Locking

c. Timestamp ordering

d. Round-robin scheduling

Correct Answer: b

Explanation: 2PL guarantees serializable schedules.

Q6.

Which phase of Two-Phase Locking allows no new locks to be acquired?

a. Growing phase

b. Shrinking phase

c. Commit phase

d. Validation phase

Correct Answer: b

Explanation: Locks are only released during shrinking.

Q7.

Which problem is most likely with coarse lock granularity?

- a. Deadlocks**
- b. Lost updates**
- c. Low concurrency**
- d. Dirty reads**

Correct Answer: c

Explanation: Coarse locks reduce parallelism.

Q8.

Which operation typically causes the highest lock contention?

- a. Projection**
- b. Selection**
- c. Join**
- d. Rename**

Correct Answer: c

Explanation: Joins access multiple relations.

Q9.

Which database object is most affected by write-heavy workloads?

- a. Buffer cache**
- b. Indexes**
- c. SQL cache**
- d. Query optimizer**

Correct Answer: b

Explanation: Indexes require frequent maintenance.

Q10.

Which index type is least suitable for frequent insert operations?

- a. Hash index**
- b. Sparse primary index**
- c. Dense secondary index**
- d. Clustering index**

Correct Answer: c

Explanation: Dense secondary indexes update per record.

Q11.

A sparse index reduces storage overhead because it:

- a. Stores one entry per record**
- b. Stores one entry per block**
- c. Eliminates pointers**
- d. Uses hashing**

Correct Answer: b

Explanation: Sparse index references blocks, not records.

Q12.

Which assumption must hold for a primary index to be effective?

- a. File is unordered**
- b. File is ordered on key**
- c. Index is multilevel**
- d. Index is dense**

Correct Answer: b

Explanation: Primary indexes rely on file ordering.

Q13.

Why can only one clustering index exist per file?

- a. DBMS limitation**
- b. Clustering indexes are dense**
- c. Physical ordering can only follow one attribute**
- d. Clustering indexes require unique keys**

Correct Answer: c

Explanation: Data can be physically ordered once.

Q14.

Which index type best supports equality-based lookups?

- a. Bitmap index**
- b. Hash index**
- c. B+-tree**
- d. Secondary sparse index**

Correct Answer: b

Explanation: Hashing provides $O(1)$ equality access.

Q15.

Why are hash indexes poor for range queries?

- a. High storage cost**
- b. Collision handling**
- c. Lack of key ordering**
- d. Complex pointer structures**

Correct Answer: c

Explanation: Hashing does not preserve order.

Q16.

Which cost component dominates large join operations?

- a. CPU cycles**
- b. Memory allocation**

c. Disk I/O

d. Network latency

Correct Answer: c

Explanation: Reading large relations from disk is costly.

Q17.

Which heuristic optimization rule is most effective?

a. Push projections down

b. Push selections down

c. Delay joins

d. Expand Cartesian products

Correct Answer: b

Explanation: Early selection reduces tuple count.

Q18.

Which relational algebra transformation preserves semantics?

a. Reordering difference

b. Pushing selection below join (when safe)

c. Replacing join with Cartesian product

d. Eliminating predicates

Correct Answer: b

Explanation: Safe selection pushdown is semantics-preserving.

Q19.

Which optimizer requires the most accurate statistics?

- a. Rule-based**
- b. Heuristic-based**
- c. Cost-based**
- d. Static optimizer**

Correct Answer: c

Explanation: Cost estimates depend on statistics.

Q20.

Outdated statistics mainly cause:

- a. Incorrect query results**
- b. Poor execution plans**
- c. Schema corruption**
- d. Transaction failure**

Correct Answer: b

Explanation: Optimizer decisions become suboptimal.

Q21.

Which statistic most affects index selectivity estimation?

- a. Attribute name**

- b. Data type**
- c. Number of distinct values**
- d. Attribute position**

Correct Answer: c

Explanation: Cardinality impacts selectivity.

Q22.

Which cache improves repeated execution of the same query?

- a. Data cache**
- b. SQL cache**
- c. Index cache**
- d. Log buffer**

Correct Answer: b

Explanation: SQL cache stores compiled execution plans.

Q23.

Which DBMS component coordinates concurrent execution?

- a. Optimizer**
- b. Listener**
- c. Scheduler**
- d. Buffer manager**

Correct Answer: c

Explanation: Scheduler orders transaction operations.

Q24.

Which operation benefits LEAST from indexing?

- a. Equality selection**
- b. Range selection**
- c. Full table scan**
- d. Join condition**

Correct Answer: c

Explanation: Full scans ignore indexes.

Q25.

Which condition is necessary for lossless decomposition?

- a. Dependency preservation**
- b. Common attribute forming a superkey**
- c. BCNF compliance**
- d. Absence of transitive dependencies**

Correct Answer: b

Explanation: Common attribute must determine one relation.

Q26.

Which normal form may sacrifice dependency preservation?

- a. 1NF**
- b. 2NF**
- c. 3NF**
- d. BCNF**

Correct Answer: d

Explanation: BCNF prioritizes anomaly removal.

Q27.

Which anomaly is eliminated by 2NF but not 1NF?

- a. Insertion anomaly**
- b. Partial dependency anomaly**
- c. Transitive dependency anomaly**
- d. Multivalued dependency**

Correct Answer: b

Explanation: 2NF removes partial dependencies.

Q28.

Which schema property ensures each fact is stored once?

- a. Redundancy**
- b. Normalization**
- c. Denormalization**

d. Replication

Correct Answer: b

Explanation: Normalization minimizes redundancy.

Q29.

Why might a designer prefer 3NF over BCNF?

- a. Stronger constraints**
- b. Fewer joins**
- c. Dependency preservation**
- d. Better indexing**

Correct Answer: c

Explanation: 3NF preserves dependencies.

Q30.

Which NoSQL principle replaces strict ACID consistency?

- a. Event-driven design**
- b. BASE**
- c. Sharding**
- d. Vertical scaling**

Correct Answer: b

Explanation: BASE relaxes consistency guarantees.

Q31.

Which NoSQL model best supports flexible schemas?

- a. Key-value**
- b. Graph**
- c. Document**
- d. Column-family**

Correct Answer: c

Explanation: Document DBs are schema-less.

Q32.

Which NoSQL system best models many-to-many relationships?

- a. Document**
- b. Key-value**
- c. Column-family**
- d. Graph**

Correct Answer: d

Explanation: Graph DBs store edges explicitly.

Q33.

Which scaling strategy increases fault tolerance?

- a. Scaling up**
- b. Vertical partitioning**

c. Scaling out

d. Normalization

Correct Answer: c

Explanation: Distributed replicas improve resilience.

Q34.

Which risk increases with scaling out?

a. Single point of failure

b. Schema rigidity

c. Network partitioning

d. Disk contention

Correct Answer: c

Explanation: Distributed systems face partition issues.

Q35.

Eventual consistency is acceptable when:

a. Financial accuracy is critical

b. Strong transactions are required

c. Temporary inconsistency is tolerable

d. Updates are rare

Correct Answer: c

Explanation: Many web systems tolerate delays.

Q36.

Which DBMS action minimizes disk reads?

- a. Increasing CPU speed**
- b. Increasing buffer cache**
- c. Reducing indexes**
- d. Normalizing schemas**

Correct Answer: b

Explanation: More cache reduces disk I/O.

Q37.

Which tuning activity is least effective for poor schema design?

- a. Index tuning**
- b. Query rewriting**
- c. Hardware upgrades**
- d. Normalization**

Correct Answer: c

Explanation: Hardware cannot fix bad design.

Q38.

Which factor most impacts join ordering decisions?

- a. Attribute names**

b. Join selectivity

c. SQL syntax

d. Index depth

Correct Answer: b

Explanation: Selectivity affects result size.

Q39.

Which join strategy is best for large relations without indexes?

a. Nested loop

b. Index nested loop

c. Hash join

d. Merge join

Correct Answer: c

Explanation: Hash join avoids index dependence.

Q40.

Which join strategy requires sorted inputs?

a. Hash join

b. Merge join

c. Nested loop

d. Cartesian join

Correct Answer: b

Explanation: Merge join works on sorted data.

Q41.

Which operation increases intermediate result size the most?

- a. Selection**
- b. Projection**
- c. Join**
- d. Rename**

Correct Answer: c

Explanation: Joins combine tuples.

Q42.

Which failure recovery step reapplies committed updates?

- a. Undo**
- b. Redo**
- c. Checkpoint**
- d. Validation**

Correct Answer: b

Explanation: Redo reapplies committed changes.

Q43.

Why are checkpoints important?

- a. Eliminate deadlocks**
- b. Reduce recovery time**
- c. Improve indexing**
- d. Enforce consistency**

Correct Answer: b

Explanation: Checkpoints limit log scanning.

Q44.

Which operation violates isolation if allowed?

- a. Reading committed data**
- b. Writing locked data**
- c. Reading uncommitted data**
- d. Writing committed data**

Correct Answer: c

Explanation: Dirty reads violate isolation.

Q45.

Which concurrency issue results in missing rows during repeated queries?

- a. Lost update**
- b. Dirty read**

- c. Phantom read
- d. Inconsistent retrieval

Correct Answer: c

Explanation: New rows appear between reads.

Q46.

Which index is most suitable for low-cardinality attributes?

- a. B+-tree
- b. Hash
- c. Bitmap
- d. Sparse index

Correct Answer: c

Explanation: Bitmap indexes excel with few distinct values.

Q47.

Which design choice most improves analytical query performance?

- a. Full normalization
- b. Denormalization
- c. Removing indexes
- d. Strict locking

Correct Answer: b

Explanation: Denormalization reduces joins.

Q48.

Which workload benefits most from column-family databases?

- a. OLTP**
- b. Graph traversal**
- c. Time-series analytics**
- d. Strong ACID transactions**

Correct Answer: c

Explanation: Column-family suits analytical queries.

Q49.

Which DBMS design goal conflicts most with scalability?

- a. Availability**
- b. Consistency**
- c. Partition tolerance**
- d. Performance**

Correct Answer: b

Explanation: CAP theorem highlights this trade-off.

Q50.

Final-level insight: a real-world DBMS must optimize for:

- a. Theory only
- b. Performance only
- c. Consistency only
- d. Trade-offs among correctness, performance, and scale

Correct Answer: d

Explanation: Practical systems balance competing goals.

.....

Faculty of Computers and Artificial Intelligence Final Exam (4) – Advanced Databases

50 MCQs | Doctor-Level | Comprehensive

Department: Data Science	Total Marks: 50 Marks
Course Name: Advanced Database	Duration: 2 hours

Q1.

Which of the following best defines a functional dependency $X \rightarrow Y$?

- a. X and Y must have the same attributes**
- b. Y uniquely determines X**
- c. X uniquely determines Y**
- d. X and Y must be keys**

Correct Answer: c

Explanation: If two tuples agree on X, they must agree on Y.

Q2.

Functional dependencies are mainly used to:

- a. Optimize queries**
- b. Design indexes**
- c. Normalize database schemas**
- d. Encrypt data**

Correct Answer: c

Explanation: FDs are the foundation of normalization.

Q3.

Which normal form eliminates repeating groups?

- a. 1NF**

- b. 2NF**
- c. 3NF**
- d. BCNF**

Correct Answer: a

Explanation: First Normal Form requires atomic attributes.

Q4.

Partial dependency exists when:

- a. A non-key attribute depends on part of a composite key b. A key depends on a non-key attribute**
- c. Two keys determine each other**
- d. An attribute depends on itself**

Correct Answer: a

Explanation: Partial dependency violates 2NF.

Q5.

A relation in 2NF must be:

- a. In 1NF only**
- b. In 1NF with no partial dependencies**
- c. In 3NF**
- d. Free of transitive dependencies**

Correct Answer: b

Explanation: 2NF removes partial dependencies.

Q6.

Which dependency violates Third Normal Form?

- a. Partial dependency**
- b. Transitive dependency**
- c. Multivalued dependency**
- d. Join dependency**

Correct Answer: b

Explanation: 3NF eliminates transitive dependencies.

Q7.

Which statement about BCNF is correct?

- a. It is weaker than 3NF**
- b. It allows transitive dependencies**
- c. Every determinant must be a superkey**
- d. It preserves all dependencies**

Correct Answer: c

Explanation: This is the defining rule of BCNF.

Q8.

Normalization mainly helps to:

- a. Increase redundancy**
- b. Reduce anomalies**
- c. Speed up joins**
- d. Increase disk usage**

Correct Answer: b

Explanation: Normalization reduces insert, update, and delete anomalies.

Q9.

Which is an example of an insertion anomaly?

- a. Losing data after deletion**
- b. Updating multiple rows for one change**
- c. Inability to insert data without unrelated data**
- d. Incorrect query results**

Correct Answer: c

Explanation: Insert anomaly blocks data insertion.

Q10.

Which type of anomaly occurs when deleting a record removes important information?

- a. Update anomaly**
- b. Insert anomaly**

c. Delete anomaly

d. Join anomaly

Correct Answer: c

Explanation: Deletion anomaly causes unintended data loss.

Q11.

An index in a database is mainly used to:

a. Enforce constraints

b. Reduce storage size

c. Speed up data retrieval

d. Store backup data

Correct Answer: c

Explanation: Indexes improve search efficiency.

Q12.

Which index contains an entry for every record?

a. Sparse index

b. Dense index

c. Primary index

d. Clustering index

Correct Answer: b

Explanation: Dense index has one entry per record.

Q13.

A primary index is built on:

- a. An unordered file**
- b. A non-key attribute**
- c. An ordered key field**
- d. A foreign key**

Correct Answer: c

Explanation: Primary index requires ordering on a key.

Q14.

Which index type can be defined on a non-key ordering field?

- a. Primary index**
- b. Secondary index**
- c. Clustering index**
- d. Hash index**

Correct Answer: c

Explanation: Clustering index orders data on non-key fields.

Q15.

Only one clustering index is allowed per file because:

- a. It must be dense**

- b. DBMS restriction**
- c. Physical ordering is single**
- d. It must be unique**

Correct Answer: c

Explanation: Data can be physically ordered once.

Q16.

Which index does NOT affect the physical order of records?

- a. Primary**
- b. Clustering**
- c. Secondary**
- d. Sparse**

Correct Answer: c

Explanation: Secondary indexes are logical access paths.

Q17.

Secondary indexes are usually:

- a. Sparse**
- b. Dense**
- c. Multilevel only**
- d. Block-based**

Correct Answer: b

Explanation: They contain one entry per record.

Q18.

Which index is best for range queries?

- a. Hash index**
- b. Bitmap index**
- c. B+-tree index**
- d. Dense index**

Correct Answer: c

Explanation: B+-trees preserve order.

Q19.

Which phase of query processing checks SQL syntax?

- a. Parsing**
- b. Validation**
- c. Execution**
- d. Fetching**

Correct Answer: a

Explanation: Parsing checks syntax.

Q20.

Which phase checks table and attribute names?

- a. Parsing**
- b. Validation**
- c. Optimization**
- d. Execution**

Correct Answer: b

Explanation: Validation ensures schema correctness.

Q21.

Query optimization aims to:

- a. Rewrite SQL syntax**
- b. Reduce execution time**
- c. Increase memory usage**
- d. Enforce constraints**

Correct Answer: b

Explanation: Optimization improves performance.

Q22.

Which optimization technique uses rules rather than statistics?

- a. Cost-based optimization**
- b. Dynamic optimization**
- c. Rule-based optimization**

d. Adaptive optimization

Correct Answer: c

Explanation: Rule-based relies on heuristics.

Q23.

Which operation is usually the most expensive?

a. Selection

b. Projection

c. Join

d. Rename

Correct Answer: c

Explanation: Joins generate large intermediate results.

Q24.

Which heuristic reduces intermediate results early?

a. Perform joins first

b. Delay selections

c. Push selections down

d. Increase projections

Correct Answer: c

Explanation: Early selection minimizes data size.

Q25.

A query tree represents:

- a. SQL syntax**
- b. Physical storage**
- c. Relational algebra expression**
- d. Disk layout**

Correct Answer: c

Explanation: Query trees model algebra operations.

Q26.

A transaction is best described as:

- a. A single SQL command**
- b. A group of related operations**
- c. A disk access**
- d. A database file**

Correct Answer: b

Explanation: Transactions group logical operations.

Q27.

Which ACID property ensures all operations succeed or none do?

- a. Consistency**
- b. Isolation**

c. Atomicity

d. Durability

Correct Answer: c

Explanation: Atomicity enforces all-or-nothing behavior.

Q28.

Which property ensures committed data is not lost after failure?

a. Atomicity

b. Consistency

c. Isolation

d. Durability

Correct Answer: d

Explanation: Durability preserves committed changes.

Q29.

Which concurrency problem occurs when one update overwrites another?

a. Dirty read

b. Lost update

c. Phantom read

d. Inconsistent retrieval

Correct Answer: b

Explanation: Lost update happens during concurrent writes.

Q30.

Which mechanism helps recover from system crashes?

- a. Indexing**
- b. Query optimization**
- c. Transaction log**
- d. Normalization**

Correct Answer: c

Explanation: Logs support rollback and recovery.

Q31.

Which lock level provides the highest concurrency?

- a. Database-level**
- b. Table-level**
- c. Page-level**
- d. Row-level**

Correct Answer: d

Explanation: Finer locks allow more parallelism.

Q32.

Which DBMS component manages locks?

- a. Optimizer**
- b. Scheduler**
- c. Lock manager**
- d. Listener**

Correct Answer: c

Explanation: Lock manager enforces concurrency control.

Q33.

NoSQL databases are mainly designed for:

- a. Strong ACID guarantees**
- b. Vertical scaling**
- c. Distributed scalability**
- d. Strict schemas**

Correct Answer: c

Explanation: NoSQL focuses on horizontal scalability.

Q34.

Which is a key feature of document databases?

- a. Fixed schema**
- b. Schema-less structure**
- c. Atomic values only**

d. Strong joins

Correct Answer: b

Explanation: Documents may have different structures.

Q35.

Which NoSQL model stores data as key–value pairs?

a. Graph

b. Column-family

c. Document

d. Key-value

Correct Answer: d

Explanation: Key-value stores use simple pairs.

Q36.

Scaling out means:

a. Buying a larger server

b. Adding more disks to one machine

c. Distributing data across many servers

d. Increasing CPU speed

Correct Answer: c

Explanation: Scaling out uses multiple servers.

Q37.

Sharding refers to:

- a. Data replication**
- b. Data encryption**
- c. Horizontal data partitioning**
- d. Index creation**

Correct Answer: c

Explanation: Sharding splits data across nodes.

Q38.

Which consistency model allows temporary inconsistency?

- a. ACID**
- b. Serializable**
- c. BASE**
- d. Strict consistency**

Correct Answer: c

Explanation: BASE allows eventual consistency.

Q39.

Which DBMS cache stores recently accessed data blocks?

- a. SQL cache**
- b. Data cache**

c. Log buffer

d. Index file

Correct Answer: b

Explanation: Data cache minimizes disk I/O.

Q40.

Reducing disk I/O improves performance because:

a. Disk is faster than RAM

b. Disk access is slower than memory

c. CPU depends on disk

d. Queries avoid optimization

Correct Answer: b

Explanation: Disk I/O is the main bottleneck.

Q41.

Which tuning activity is client-side?

a. Buffer configuration

b. Index maintenance

c. SQL query rewriting

d. Lock scheduling

Correct Answer: c

Explanation: SQL formulation happens at client side.

Q42.

Which tuning activity is server-side?

- a. Query formulation**
- b. Buffer cache tuning**
- c. SQL syntax checking**
- d. Attribute naming**

Correct Answer: b

Explanation: Memory tuning is server-side.

Q43.

Which statistic is most useful for selectivity estimation?

- a. Number of attributes**
- b. Number of distinct values**
- c. Attribute names**
- d. Data type**

Correct Answer: b

Explanation: Cardinality impacts selectivity.

Q44.

Which index is best for low-cardinality attributes?

- a. Hash**
- b. B+-tree**
- c. Bitmap**
- d. Sparse**

Correct Answer: c

Explanation: Bitmap indexes suit low-cardinality data.

Q45.

Which join method requires sorted input?

- a. Hash join**
- b. Merge join**
- c. Nested loop**
- d. Index join**

Correct Answer: b

Explanation: Merge join operates on sorted relations.

Q46.

Which join is best when no indexes exist?

- a. Index nested loop**
- b. Hash join**
- c. Merge join**
- d. Cartesian join**

Correct Answer: b

Explanation: Hash join does not require indexes.

Q47.

Which operation benefits least from indexing?

- a. Equality selection**
- b. Range selection**
- c. Full table scan**
- d. Join**

Correct Answer: c

Explanation: Full scan ignores indexes.

Q48.

Which failure recovery step reapplies committed updates?

- a. Undo**
- b. Redo**
- c. Abort**
- d. Validation**

Correct Answer: b

Explanation: Redo reapplies committed changes.

Q49.

Which isolation level allows dirty reads?

- a. Serializable**
- b. Repeatable read**
- c. Read committed**
- d. Read uncommitted**

Correct Answer: d

Explanation: Dirty reads are allowed here.

Q50.

A well-designed database balances:

- a. Normalization only**
- b. Performance only**
- c. Consistency only**
- d. Correctness, performance, and scalability**

Correct Answer: d

Explanation: Practical systems require balanced trade-offs.

